



SCHOOL OF COMPUTING SCIENCE AND DIGITAL MEDIA

MSc Big Data Analytics

Student Name: HIRAN MADHUSHANKA ABEYWARDHANA	Matriculation Number:
Supervisor: MR PUMUDU FERNANDO	Second Marker:
Project Title: CROSS-DOMAIN MACHINE LEARNING FRAMEWORK FOR SCALABLE GUI ELEMENT DETECTION AND ADAPTATION IN DESKTOP ENVIRONMENTS	
	Start Date: 23/09/2025
	Submission Date: 03/08/2026

CONSENT

I agree ☒
I do not agree ☐

That the University shall be entitled to use any results, materials or other outcomes arising from my project work for the purposes of non-commercial teaching and research, including collaboration.

DECLARATION

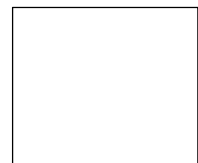
I confirm:

- That the work contained in this document has been composed solely by myself and that I have not made use of any unauthorised assistance.
- That the work has not been accepted in any previous application for a degree.
- All sources of information have been specifically acknowledged and all verbatim extracts are distinguished by quotation marks.

Student Signature: 	Date Signed: 03/08/2026
--	----------------------------



SYMBOL OF SUCCESS



MSC PROJECT REPORT

CROSS-DOMAIN MACHINE LEARNING FRAMEWORK FOR SCALABLE GUI ELEMENT DETECTION AND ADAPTATION IN DESKTOP ENVIRONMENTS

HIRAN MADHUSHANKA ABEYWARDHANA

2026

A REPORT SUBMITTED AS PART OF THE REQUIREMENTS FOR THE DEGREE OF
MSC BIG DATA ANALYTICS AT ROBERT GORDON UNIVERSITY, ABERDEEN, SCOTLAND

Declaration

I confirm that the work contained in this MSc project report has been composed solely by myself and has not been accepted in any previous application for a degree. All sources of information have been specifically acknowledged, and all verbatim extracts are distinguished by quotation marks.

A handwritten signature in black ink, appearing to be 'J. R.' with a stylized flourish.

Signed.....

03/08/2026

Date.....

The aim of the University's *Research Ethics Policy* is to establish and promote good ethical practice in the conduct of academic research. The questionnaire is intended to enable researchers to undertake an initial self-assessment of ethical issues in their research. Ethical conduct is not primarily a matter of following fixed rules; it depends on researchers developing a considered, flexible and thoughtful practice.

The questionnaire aims to engage researchers discursively with the ethical dimensions of their work and potential ethical issues, and the main focus of any subsequent review is not to 'approve' or 'disapprove' of a project but to make sure that this process has taken place.

The *Research Ethics Policy* is available at www.rgu.ac.uk/research-ethics-policy

Student Name	Hiran Madhushanka Abeywardhana
Supervisor	Mr Pumudu Fernando
Project Title	Cross-Domain Machine Learning Framework for Scalable GUI Element Detection and Adaptation in Desktop Environments
Course of Study	MSc Big Data Analytics
School/Department	School of Computing

PART 1: DESCRIPTIVE QUESTIONS

1.	Does the research involve, or does information in the research relate to: [see Guidance Note 1]	Yes	No
	(a) individual human subjects		X
	(b) groups (e.g. families, communities, crowds)		X
	(c) organisations		X
	(d) animals?		X
	(e) genetically-modified organisms www.rgu.ac.uk/hr/healthsafety/page.cfm?pge=26027#122628		X
	Please provide further details:		
2.	Will the research deal with information which is private or confidential? [see Guidance Note 2]	Yes	No
			X
Please provide further details:			

PART 2: THE IMPACT OF THE RESEARCH

3.	In the process of doing the research, is there any potential for harm to be done to, or costs to be imposed on: [see Guidance Note 3(i)]	Yes	No
	(a) research participants?		X
	(b) research subjects? [see Guidance Note 3(ii)]		X
	(c) you, as the researcher?		X
	(d) third parties? [see Guidance Note 3(iii)]		X
	Please state what you believe are the implications of the research:		
4.	When the research is complete, could negative consequences follow:	Yes	No
	(a) for research subjects		X
	(b) or elsewhere? [see Guidance Note 4]		X
	Please state what you believe are the consequences of the research:		

PART 3: ETHICAL PROCEDURES

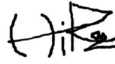
5.	Does the research require informed consent or approval from: [see Guidance Note 5(i)]	Yes	No
	(a) research participants?		X
	(b) research subjects? [see Guidance Note 5(ii)]		X
	(c) external bodies? [see Guidance Note 5(iii)]		X
	If you answered yes to any of the above, please explain your answer:		
6.	Are there reasons why research subjects may need safeguards or protection? [see Guidance Note 6]	Yes	No
			X

	If you answered yes to the above, please state the reasons and indicate the measures to be taken to address them:		
7.	Does the research involve any “regulated work with children” and/or “regulated work with protected adults”, therefore requiring membership of the <i>Protecting Vulnerable Groups (PVG) Scheme</i> ? [see Guidance Note 7]	Yes	No
			X
[Please note: if the research potentially involves “regulated work”, this MUST be raised with your HR Business Partner immediately. In this instance, the Human Resources Department will conduct a detailed assessment and will confirm whether or not PVG Membership is required.]			
(a) PVG membership is not required.		X	
(b) PVG membership may be required for working with children.			X
(c) PVG membership may be required for working with protected adults.			X
(d) PVG membership may be required for working with both children and protected adults.			X
If you answered yes to (b), (c) or (d) above, please give further information about the work you will be required to undertake and the nature of the contact with these groups. Please provide as much detail as possible:			
		Yes	No
Are you already a PVG member?			X
If yes, please provide your PVG Scheme number:			
8.	Are specified procedures or safeguards required for recording, management, or storage of data? [see Guidance Note 8]	Yes	No
			X
If you answered yes to any of the above, please give details:			

PART 4: THE RESEARCH RELATIONSHIP			
9.	Does the research require you to give or make undertakings to research participants or subjects about the use of data? [see Guidance Note 9]	Yes	No
			X

	If you answered yes to the above, please outline the likely undertakings:		
10.	Is the research likely to be affected by the relationship with a sponsor, funder or employer? [see Guidance Note 10]	Yes	No
			X
	If you answered yes to the above, please identify how the research may be affected:		

Part 5: Other Issues			
11.	Are there any other ethical issues not covered by this form which you believe you should raise?	Yes	No
			X

Statement by Student			
I believe that the information I have given in this form is correct, and that I have addressed the ethical issues as fully as possible at this stage.			
Signature:		Date:	06/11/2025

If any ethical issues arise during the course of the research, students should complete a further Student Project Ethical Review (SPER) form.

The Research Ethics Policy is available at www.rgu.ac.uk/research-ethics-policy

PART 6: TO BE COMPLETED BY THE SUPERVISOR			
12.	Does the research have potentially negative implications for the University? [see Guidance Note 11]	Yes	No
			X
	If you answered yes to the above, please explain your answer:		
13.	Are any potential conflicts of interest likely to arise in the course of the research? [see Guidance Note 12]	Yes	No
			X

	If you answered yes to the above, please identify the potential conflicts:		
14.	Are you satisfied that the student has engaged adequately with the ethical implications of the work? [see Guidance Note 13]	Yes	No
		x	
	If you answered no to the above, please identify the potential issues:		
15.	Appraisal: Please select one of the following		
	i. The research project should proceed in its present form – no further action is required	x	
	ii. The research project requires ethical approval by the School Ethics Review Panel (SERP) (or equivalent)		
	iii. The research project requires ethical review by the University's Research Ethics Sub-Committee		
	iv. The project needs to be returned to the student for modification prior to further action		
	v. The research project requires ethical review by an external body (N.B. Question 5 above). If this applies, please give these details:		
	Title of External Body providing ethical review		
	Address of External Body		
	Anticipated date when External Body may consider project		

AFFIRMATION BY SUPERVISOR

I have read the student's responses and have discussed ethical issues arising with the student. I can confirm that, to the best of my understanding, the information presented by the student is correct and appropriate to allow an informed judgement on whether further ethical approval is required.

Signature:	Pumudu Fernando	Date:	07/11/2025
-------------------	------------------------	--------------	-------------------

ABSTRACT

Automating tasks on desktop applications is useful in many practical scenarios, from repetitive office work to accessibility support. Existing tools, however, have well-known weaknesses. Bitmap-based tools stop working when the visual style of an application change. Accessibility-tree tools cannot see controls that the application does not expose to the operating system. Large vision-language models handle both cases but require computing and data budgets far beyond what a single-author MSc project can afford. This research proposes a lightweight, computer-vision-driven desktop UI automation solution built under a small labelled-data budget. The solution combines an object detector, pre-trained on mobile UI screenshots and then adapted to the desktop through transfer learning, with a text-matching layer and a refusal rule that reports uncertainty instead of clicking on the wrong control. The prototype, named VisClick, was tested on a fifteen-task functional suite against three classical automation baselines. It matched the strongest baseline on task-success rate while producing more predictable behavior on unfamiliar screens. A working prototype has been developed as proof of concept.

Keywords: GUI automation, transfer learning, computer vision, desktop UI, resource-constrained learning

ACKNOWLEDGEMENT

I would sincerely like to thank all those who supported me through this dissertation. First, my project supervisor, Mr. Pumudu Fernando, guided me through the design choices, the reporting protocol, and the many rounds of feedback that shaped the final report. My thanks also go to the module leaders and academic staff of the MSc Big Data Analytics program at Robert Gordon University for the teaching material that this project stands on.

I am grateful to my colleagues at Synopsys who took part in the expert evaluation - for making time in their working schedule to run the prototype on their own machines and for the honest, detailed feedback that grounded the deployment discussion in real production experience.

Finally, I want to thank my family for their patience over the two years of the program, and my friends and peers whose encouragement kept the writing on track through the difficult months. This dissertation would not be in the form it is without their support.

TABLE OF CONTENT

Contents

CHAPTER 01 - INTRODUCTION	1
1.1 CHAPTER OVERVIEW	1
1.2 PROJECT BACKGROUND.....	1
1.3 PROBLEM STATEMENT	2
1.4 RESEARCH AIM	3
1.5 RESEARCH QUESTIONS.....	3
1.6 RESEARCH GAP	3
1.7 RESEARCH OBJECTIVES	4
1.8 OPERATIONAL OBJECTIVES	5
1.8.1 DATA ENGINEERING AND BASELINE ESTABLISHMENT	5
1.8.2 MODEL ADAPTATION EXPERIMENTS	5
1.8.3 PROTOTYPE INTEGRATION.....	5
1.8.4 END-TO-END EVALUATION AND THESIS COMPOSITION	5
1.9 PROPOSED SOLUTION	6
1.10 SCOPE OF THE PROJECT.....	6
1.11 CHAPTER SUMMARY	7
CHAPTER 02 – LITERATURE REVIEW	8
2.1 CHAPTER OVERVIEW	8
2.2 EXISTING GUI ELEMENT DATASETS	8
2.3 PRE-PROCESSING FOR UI ELEMENT DETECTION.....	10
2.4 CLASSICAL AUTOMATION APPROACHES	11
2.5 DEEP-LEARNING APPROACHES FOR UI ELEMENT DETECTION	13
2.6 DOMAIN ADAPTATION METHODOLOGIES.....	14
2.7 GUI AGENTS AND VISUAL GROUNDING FRAMEWORKS	16
2.8 EVALUATION METRICS IN THE UI DOMAIN	17
2.9 RESEARCH GAP AND POSITIONING OF CURRENT STUDY	18
2.10 CHAPTER SUMMARY	19

CHAPTER 03 - REQUIREMENT ANALYSIS	20
3.1 CHAPTER OVERVIEW	20
3.2 STAKEHOLDER ANALYSIS	20
3.2.1 THE ONION MODEL	20
3.2.2 STAKEHOLDER VIEWPOINTS	21
3.2.3 A NOTE ON PRIMARY DATA GATHERING	22
3.3 REQUIREMENT GATHERING TECHNIQUES	23
3.4 METHODOLOGY FOR OBTAINING DATASETS	23
3.5 USE CASE DIAGRAMS (UML)	24
3.6 USE CASE DESCRIPTIONS	25
3.7 FUNCTIONAL REQUIREMENTS	26
3.8 NON-FUNCTIONAL REQUIREMENTS.....	27
3.9 CHAPTER SUMMARY	28
CHAPTER 04 - PROJECT MANAGEMENT.....	29
4.1 CHAPTER OVERVIEW	29
4.2 RESEARCH METHODOLOGY	29
4.3 SOFTWARE DESIGN METHODOLOGY.....	30
4.4 SOFTWARE DEVELOPMENT METHODOLOGY	31
4.4.1 WATERFALL WITH ITERATION WITHIN PHASES.....	31
4.4.2 PROGRAMMING PARADIGM - FUNCTIONAL AND MODULE-ORIENTED	32
4.5 PROJECT MANAGEMENT METHODOLOGY	33
4.5.1 CHOICE OF PROJECT MANAGEMENT FRAMEWORK	33
4.5.2 PRINCE2 PRINCIPLES APPLIED.....	33
4.5.3 TRACKING ARTEFACTS	34
4.6 RISK MITIGATION PLAN	34
4.7 PROJECT PLAN	36
4.7.1 JUSTIFICATION OF THE PHASED GANTT STRUCTURE.....	36
4.7.2 PHASE DETAILS.....	37
4.8 CHAPTER SUMMARY	37

CHAPTER 05 - DESIGN.....	38
5.1 CHAPTER OVERVIEW	38
5.2 RESEARCH DESIGN	38
5.2.1 CONTRIBUTION TO THE DESIGN	38
5.2.2 EXPERIMENTAL MATRIX.....	39
5.3 SYSTEM ARCHITECTURE.....	40
5.4 PROCESS FLOW	41
5.5 MODULE DESIGN	43
5.6 REPOSITORY LAYOUT AND DATA STORAGE DESIGN	43
5.7 ALGORITHM DESIGN	45
5.7.1 THE MATCHER.....	45
5.7.2 THE REFUSAL RULE.....	46
5.8 GUI WIREFRAMES.....	46
5.9 CHAPTER SUMMARY	47
CHAPTER 06 - IMPLEMENTATION.....	48
6.1 CHAPTER OVERVIEW	48
6.2 DEVELOPMENT ENVIRONMENT AND TOOLCHAIN	48
6.3 DATASET IMPLEMENTATION	49
6.3.1 SOURCE-DOMAIN CORPUS.....	49
6.3.2 UNLABELLED TARGET CORPUS	50
6.3.3 LABELLED TARGET TEST CORPUS	50
6.4 SOURCE-DOMAIN DETECTOR TRAINING	50
6.4.1 YOLOV8S SOURCE BASELINE.....	50
6.4.2 DETR-R50 SOURCE BASELINE	51
6.5 ADAPTATION METHODS.....	52
6.5.1 M0 ZERO-SHOT TRANSFER (CONTROL)	52
6.5.2 FEW-SHOT FINE-TUNE (SAMPLE-EFFICIENCY CURVE).....	52
6.5.3 SELF-SUPERVISED PRE-TRAINING + FINE-TUNE	54
6.5.4 ADAPTIVE TEACHER (UDA)	54

6.5.5 SHOT (UDA)	55
6.6 PRE-PROCESSING PIPELINE	56
6.7 PROTOTYPE IMPLEMENTATION	57
6.7.1 VISCLICK.CAPTURE	57
6.7.2 VISCLICK.DETECT	57
6.7.3 VISCLICK.OCR	58
6.7.4 VISCLICK.MATCH	58
6.7.5 VISCLICK.ACT	58
6.7.6 VISCLICK.BOT	59
6.7.7 VISCLICK.GUI	59
6.8 OCR INTEGRATION.....	60
6.9 MATCHING ALGORITHM IMPLEMENTATION.....	60
6.10 ACTION LAYER AND MULTI-MONITOR SUPPORT.....	61
6.11 EVALUATION HARNESS IMPLEMENTATION	61
6.12 CHAPTER SUMMARY	63
CHAPTER 07 - TESTING.....	64
7.1 CHAPTER OVERVIEW	64
7.2 TESTING STRATEGIES USED.....	64
7.3 DETECTOR TRAINING AND ADAPTATION RESULTS	66
7.3.1 SOURCE-DOMAIN BACKBONES	66
7.3.2 FEW-SHOT SAMPLE-EFFICIENCY CURVE	67
7.3.3 SSP+FT ABLATION.....	67
7.3.4 ADAPTIVE TEACHER	68
7.3.5 SHOT	69
7.4 GROUNDING ACCURACY (CPV).....	69
7.4.1 HAND-CORRECTED EIGHT-IMAGE SET	69
7.4.2 SCREENSPOT DESKTOP SLICE.....	70
7.4.3 PROTOCOL DISAGREEMENT.....	71
7.5 FOUR-METHOD COMPARISON ON T01-T15.....	71

7.5.1 AGGREGATE TASK SUCCESS RATE	71
7.5.2 PER-TASK VERDICT MATRIX.....	71
7.6 NON-FUNCTIONAL REQUIREMENT VERIFICATION.....	72
7.6.1 ACCURACY.....	73
7.6.2 LATENCY	73
7.6.3 MEMORY, RELIABILITY, AND THE REMAINING NFRS.....	73
7.7 CHAPTER SUMMARY	74
CHAPTER 08 - EVALUATION AND DISCUSSION	75
8.1 CHAPTER OVERVIEW	75
8.2 RQ1 - VISCLICK VERSUS CLASSICAL BASELINES	75
8.2.1 EVALUATION-PROTOCOL DISAGREEMENT.....	76
8.2.2 PER-TASK ERROR ANALYSIS.....	77
8.3 RQ2 - HOW WELL DOES ADAPTATION CLOSE THE GAP?	79
8.3.1 CATASTROPHIC FORGETTING AT $K = 1$	79
8.3.2 WHY ADAPTIVE TEACHER FULL UNDERPERFORMS ADAPTIVE TEACHER SHORT.....	80
8.3.3 THE SHOT FREEZE-DEPTH ABLATION	80
8.3.4 SSP+FT ON MOBILE UI HURTS DESKTOP TRANSFER.....	81
8.3.5 CROSS-METHOD SYNTHESIS - WHEN TO USE WHAT	82
8.4 RQ3 - ARCHITECTURAL AND METHODOLOGICAL COMPARISONS	83
8.4.1 YOLOV8S VERSUS DETR-R50	83
8.4.2 ADAPTIVE TEACHER VERSUS SHOT.....	83
8.4.3 BACKBONE CHOICE IN OTHER REGIMES	84
8.5 RQ4 - DESIGN CHOICES IN THE DEPLOYED PROTOTYPE	85
8.5.1 ANATOMY OF THE FOUR DESIGN CHOICES	85
8.5.2 LATENCY BUDGET ENGINEERING.....	86
8.6 EXPERT EVALUATION.....	87
8.6.1 PANEL AND PROTOCOL	87
8.6.2 KEY OBSERVATIONS	88

8.6.3 EXPERT FEEDBACK VERSUS AUTHOR EVALUATION	89
8.6.4 LIMITATIONS OF THIS EXPERT PANEL	89
8.7 THREATS TO VALIDITY	90
8.8 ETHICS, DEPLOYMENT, AND FUTURE WORK	90
8.8.1 ETHICAL CONSIDERATIONS	91
8.8.2 LEGAL CONSIDERATIONS	91
8.8.3 DEPLOYMENT CONSIDERATIONS	91
8.8.4 INTEGRATION WITH EXISTING AUTOMATION TOOLS	92
8.9 CHAPTER SUMMARY	93
CHAPTER 09 - CONCLUSION.....	94
9.1 CHAPTER OVERVIEW	94
9.2 CONCLUSION AND ANSWERS TO RESEARCH QUESTIONS	94
9.3 LEARNING OUTCOMES	95
9.4 HIGHLY RELEVANT ACADEMIC MODULES.....	96
9.5 SELF-TAUGHT AREAS AND NEW SKILLS	97
9.6 FUTURE WORK	98
9.7 FINAL REFLECTION.....	98
9.8 CHAPTER SUMMARY	99
REFERENCES.....	100
APPENDICES.....	103
EVIDENCE OF FEEDBACK.....	103

LIST OF FIGURES

FIGURE 1 : MOBILE UI AND DESKTOP UI	2
FIGURE 2 : PROPOSED FRAMEWORK SITS RELATIVE TO EXISTING GUI AUTOMATION APPROACHES..	4
FIGURE 3 : THE TWO PARTS OF THE PROPOSED SOLUTION	6
FIGURE 4 : VISUAL EXAMPLES OF CLAY'S DENOISING ON RAW RICO SCREENS.....	10
FIGURE 5 : PER-TASK VERDICTS OF THE THREE CLASSICAL BASELINES ON T01-T15.....	13
FIGURE 6 : TWO ARCHITECTURAL FAMILIES FOR INSTRUCTION-TO-ACTION GUI AGENTS.....	17
FIGURE 7 : SIX-RING ONION STAKEHOLDER MODEL.	20
FIGURE 8 : USE CASE DIAGRAM FOR VISCLICK.....	25
FIGURE 9 : TWELVE-MONTH PROJECT PLAN OVER THE FOUR OPERATIONAL PHASES.	34
FIGURE 10 : BLOCK DIAGRAM OF VISCLICK. EACH OF CAPTURE, DETECT, OCR AND MATCH.....	40
FIGURE 11 : PER-INSTRUCTION PROCESS FLOW.	42
FIGURE 12 : REPOSITORY DIRECTORY TREE.....	43
FIGURE 13 : GUI WIREFRAME.....	46
FIGURE 14 : THE VISCLICK GUI AS IT APPEARS ON WINDOWS 11 DURING A LIVE EVALUATION	59
FIGURE 15 : SOURCE-DOMAIN BACKBONE COMPARISON ON THE UNIFIED-BUNDLE VAL SPLIT.....	66
FIGURE 16 : FEW SHOTS SAMPLE EFFICIENCY GRAPH	67
FIGURE 17 : BAR CHART ON SCREENSPOT CPV FOR NINE CONFIGURATIONS.	69
FIGURE 18 : FIVE ROWS OF BARS (OVERALL, MACOS, WINDOWS, TEXT, ICON) BY FOUR METHODS..	70

LIST OF TABLES

TABLE 1 : PUBLIC GUI ELEMENT DATASETS CONSIDERED FOR THE SOURCE DOMAIN, SUMMARIZING SCALE, MODALITY, TAXONOMY AND LICENSES.	9
TABLE 2 : FUNCTIONAL REQUIREMENTS.	27
TABLE 3 : NON-FUNCTIONAL REQUIREMENTS COVERING LATENCY, FOOTPRINT, ACCURACY, ROBUSTNESS, PORTABILITY AND EXPLAINABILITY.	28
TABLE 4 : RISK REGISTER IDENTIFYING THE PROJECT'S PRINCIPAL RISKS, AND THE MITIGATION STRATEGY ADOPTED.	35
TABLE 5 : EXPERIMENTAL MATRIX.	39
TABLE 6 : CLASS-COLLAPSE MAPPING FROM RICO, CLAY, VINS.	49
TABLE 7 : YOLOV8S SOURCE-DOMAIN PER-CLASS AP AT MAP@0.5 AND MAP@0.5:0.95.....	51
TABLE 8 : FEW-SHOT SAMPLE-EFFICIENCY CURVE. HAND-CORRECTED MAP RISES MONOTONICALLY WITH K.	53
TABLE 9 : SSP+FT ABLATION AGAINST THE NO-SSP BASELINE AT K=1 AND K=8 ON BOTH EVALUATION PROTOCOLS.	54
TABLE 10 : ADAPTIVE TEACHER RESULTS ACROSS THE SHORT (1-ITER, 5 EP) AND FULL (3-ITER, 10 EP) CONFIGURATIONS.	55
TABLE 11 : SHOT RESULTS ACROSS THREE CONFIGURATIONS: SHORT (8 EP, FREEZE=15), 15-EPOCH, AND FREEZE=10.	56
TABLE 12 : TESTING STRATEGIES MAPPED TO THE CHAPTER SUBSECTIONS THAT APPLY THEM.....	65
TABLE 13 : SOURCE-DOMAIN DETECTOR RESULTS ON THE HELD-OUT UNIFIED VAL SPLIT (1,000 IMAGES).	66
TABLE 14 : YOLOV8S PER-CLASS PRECISION, RECALL, AP@0.5, AND MAP@0.5:0.95 ON THE SOURCE-DOMAIN VAL SPLIT.	66
TABLE 15 : FEW-SHOT SAMPLE-EFFICIENCY CURVE AT K = 0, 1, 2, 4, 8. BOLD CELLS MARK THE PER-PROTOCOL MAXIMUM.....	67
TABLE 16 : SSP+FT AGAINST NO-SSP BASELINE.	68
TABLE 17 : ADAPTIVE TEACHER RESULTS, SHORT AND FULL CONFIGURATIONS	68
TABLE 18 : ADAPTIVE TEACHER (SHORT VARIANT) SCREENSPOT CPV BY SLICE.	68
TABLE 19 : SHOT - THREE CONFIGURATIONS ON THE H100.	69
TABLE 20 : HAND-CORRECTED CPV PER CLASS FOR THE DEPLOYED CHECKPOINT.	70
TABLE 21 : SCREENSPOT DESKTOP CPV BY SLICE FOR THE DEPLOYED CHECKPOINT	70
TABLE 22 : FOUR-METHOD AGGREGATE TSR ON T01-T15.	71
TABLE 23 : PER-TASK VERDICT MATRIX FOR THE FOUR-METHOD SUITE.	72
TABLE 24 : NFR VERIFICATION LEDGER.....	73
TABLE 25 : PER-METHOD END-TO-END LATENCY DISTRIBUTION ON T01-T15.....	73
TABLE 26 : EXPERT-EVALUATION PANEL.	88

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AT	Adaptive Teacher
BBox	Bounding Box
CI	Confidence Interval
CLAY	Cleaner Layout Annotations for You (mobile UI dataset)
CNN	Convolutional Neural Network
CPV	Central Point Validation
CPU	Central Processing Unit
CSV	Comma-Separated Values
DETR	DEtection TRansformer
DSR	Design Science Research
EDA	Electronic Design Automation
EMA	Exponential Moving Average
GPU	Graphics Processing Unit
GUI	Graphical User Interface
IoU	Intersection over Union
IVGocr	Instruction Visual Grounding via OCR
LVLM	Large Vision-Language Model
mAP	mean Average Precision
ML	Machine Learning
MSc	Master of Science
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
PMBoK	Project Management Body of Knowledge
PRINCE2	PROjects IN Controlled Environments (v2)
RGU	Robert Gordon University
RICO	Rich Interaction Corpus (mobile UI dataset)
RPA	Robotic Process Automation
RSS	Resident Set Size (memory measure)
SHOT	Source Hypothesis Transfer
SPER	Student Project Ethical Review
SSP	Self-Supervised Pre-training
TSR	Task Success Rate
UDA	Unsupervised Domain Adaptation
UI	User Interface
UIA	UI Automation (Windows framework)
VINS	Visual Search for Interface Design (mobile UI dataset)
YOLO	You Only Look Once

CHAPTER 01 - INTRODUCTION

1.1 CHAPTER OVERVIEW

This chapter frames the project at the problem level. It describes the situation on the modern desktop, the practical gap in the available tooling, the aim, the four research questions, the operational plan by which the work was carried out, and the boundary of what the project does and does not attempt. Technical detail - which datasets, which model families, which adaptation strategies, which libraries - is deliberately kept out of this chapter and lives in Chapter 2 (Literature Review), Chapter 5 (Design) and Chapter 6 (Implementation). The intention is that a reader can finish Chapter 1 knowing *what problem is being solved and why it matters*, without yet knowing *how*.

1.2 PROJECT BACKGROUND

Graphical user interfaces are how most people interact with software. Filing a tax return on a web form, editing a document, renaming a file in a file manager - the underlying mechanic is the same. A small set of clicks and keystrokes directed at coloured rectangles on a screen. Automating that mechanic has been an active area of work for decades. The motivations span the mundane and the ambitious. Repetitive data entry. Regression testing of an application's controls. Robotic process automation in back-office settings. Accessibility tools that let users with motor impairment drive an application by voice. Autonomous agents that complete multi-step tasks on a user's behalf. In every case the primitive is the same: identify the right control on the screen, then click it.

The tools that automate that primitive fall into two long-established families. The first works from pixels. A small image of the target control is captured up front, and at run time the tool searches the live screen for a matching bitmap. Simple. Also fragile. A bitmap is a photograph of the control taken on one machine, under one theme, at one display scale, on one operating-system build. Change any of those and the photograph no longer matches. The second family goes at the problem from a different angle entirely. It walks the operating system's accessibility tree - the same tree screen-readers use - and finds a *Save* button by its semantic name rather than by its appearance. In theory this is the right approach. In practice, on the modern Windows desktop, the tree has quietly stopped being a faithful description of what the user sees. Applications with web-technology renderers show only a degraded tree; newer Windows applications often expose their controls under internal identifiers that do not match the visible label; and web pages inside a browser use a different accessibility convention again. The upshot is that the accessibility tree and the visible interface have drifted apart, and for a growing share of desktop applications the drift is now too wide to bridge with a tree-walking script.

A newer strand of work sits outside both families. It uses computer vision to detect UI elements directly from a screenshot, without any privileged access to the application's internals. On the mobile side this strand is mature - a decade of research has produced well-resourced datasets and highly-optimised detectors. On the desktop side it is markedly less developed. Desktop screens are landscape rather than portrait, multi-window rather than single-window, and packed with dense toolbars and ribbon menus of a kind mobile screens simply do not contain. They also draw on decades of stylistic variance - old and new theme

families, dark and light modes, high-contrast accessibility skins - that has no real parallel in the more harmonised design languages of the mobile world. And there is no desktop-side dataset at the scale of the mobile-side canonical ones. The few recent attempts to build one are small, recent, and still evolving.

There is a fourth strand - foundation-scale vision-language models - that sits at the other end of the spectrum from the classical tools. On paper it can handle the mobile-to-desktop gap without much effort. But the compute cost is enormous. Multi-GPU inference, large memory budgets, cloud dependencies. That kind of hardware is a non-starter for the sort of practical on-machine deployment this project cares about, so foundation-scale models end up being the point of comparison, not the point of departure.

Put all four strands together and you get a spectrum with a hole in the middle. The lightweight tools are cheap to run but break the moment the visible surface shifts. The heavyweight tools generalise beautifully but need hardware nobody actually has on their desk. What is missing is a middle option. Small enough to run on a normal machine. Tolerant of the theme and display-scale variability of real desktops. Accurate enough to drive a click reliably. And - because a detector on its own is not the whole story - validated end-to-end inside a working automation tool, not just on a benchmark. Producing that middle option is what this project takes on.

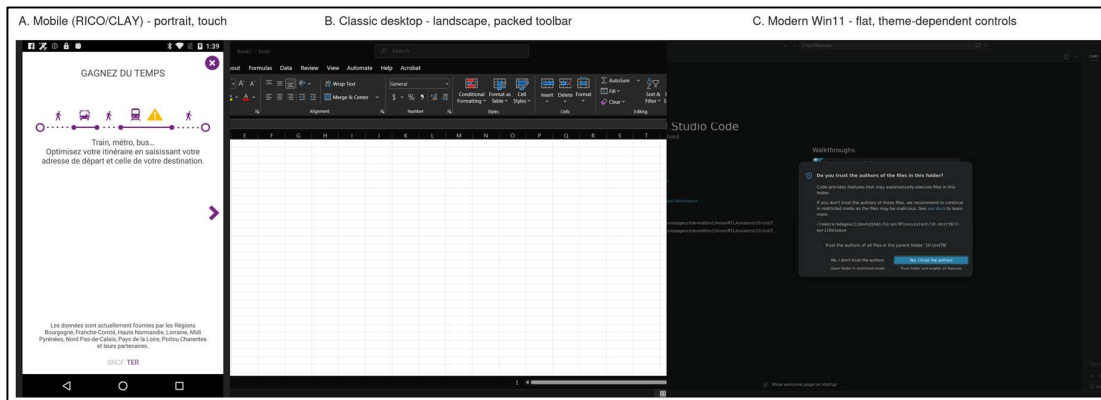


Figure 1 : Mobile Ui and Desktop UI.

1.3 PROBLEM STATEMENT

The problem, in one sentence: nobody today has a desktop UI automation tool that is accurate, that fits on a personal machine, and that survives the visual variability of real applications. Not simultaneously.

The pixel-based tools are lightweight but rigid - their bitmap matching gives up the moment the theme, display scale or font changes. The accessibility-tree tools are tolerant of theme and scale by design, but on the modern Windows application mix Chapter 7's numbers show they simply do not see what the user sees. And the foundation-scale models that *are* both accurate and tolerant come with a compute cost that puts them out of reach for the on-machine deployment this project is aiming at.

So the gap sits in the middle. Whether a small model, trained on the well-resourced mobile UI domain and then carried across to the data-scarce desktop domain on a modest labelled budget, can be made good enough to drive a real click is the empirical question the project takes on. And because this is at heart a transfer-learning question, the methodological one that rides alongside it is: which adaptation strategy gives the best return on the labelled data a single developer can realistically produce?

There is a secondary problem sitting alongside the detection one. Even a good detector is not an automation tool by itself. Nobody wants to be told "the screen contains forty-seven elements". People want to type "click Save" and have the right thing happen. Turning detection output into a click needs a grounding step - matching a typed instruction to one of the detected elements - and an action step that translates that match into an actual OS-level click. The pipeline shape for this is well established and reviewed in Chapter 2. The project's novelty is not the pipeline. It is the detector that goes inside the pipeline, and the honest end-to-end evaluation of the whole tool on real desktop applications rather than only on a static benchmark.

1.4 RESEARCH AIM

The aim of the research is to design, develop and evaluate a machine-learning framework that adapts existing UI-detection models to the modern Windows desktop while remaining lightweight enough to run on a personal machine, and to demonstrate that framework inside a working instruction-to-click automation prototype.

1.5 RESEARCH QUESTIONS

The aim decomposes into four research questions that the rest of the work addresses one at a time.

- **RQ1.** How large is the performance drop when a UI-element detector trained on one domain (mobile) is applied to a different domain (desktop)?
- **RQ2.** Which families of adaptation strategy close that drop most effectively under a small labelled-data budget?
- **RQ3.** How does the choice of detector architecture family influence transferability and sample efficiency?
- **RQ4.** What are the practical limits of a vision-driven click automation tool built on the adapted detector, measured on real desktop applications?

Empirical numbers answering all four questions are reported in Chapter 7, and their interpretation is in Chapter 8. The chapter map is repeated in Section 1.11.

1.6 RESEARCH GAP

The gap is not that the individual pieces are missing. Strong mobile-side detectors are out there. Sensible modular pipeline patterns are out there. Foundation-scale models that can cross the domain gap are out there too. What is missing is the *combination* - a lightweight, data-efficient method that carries a strong mobile detector across into the desktop domain on the small labelled budget an individual or small team can actually produce, and that shows the resulting tool works end-to-end on real applications rather than only on a static test split.

The gap is sharpened by an asymmetry in the data. On the mobile side there are canonical, publicly-available labelled corpora at tens of thousands of screens. On the desktop side there are a handful of small, recently-emerging datasets and not much else. That asymmetry is what forces the problem to be a transfer-learning problem. Training a big desktop-only detector from scratch is not on the table for a personal-budget project, so the model has to be trained somewhere else and then carried across cheaply. Chapter 2 makes both halves of this gap concrete with the underlying citations.

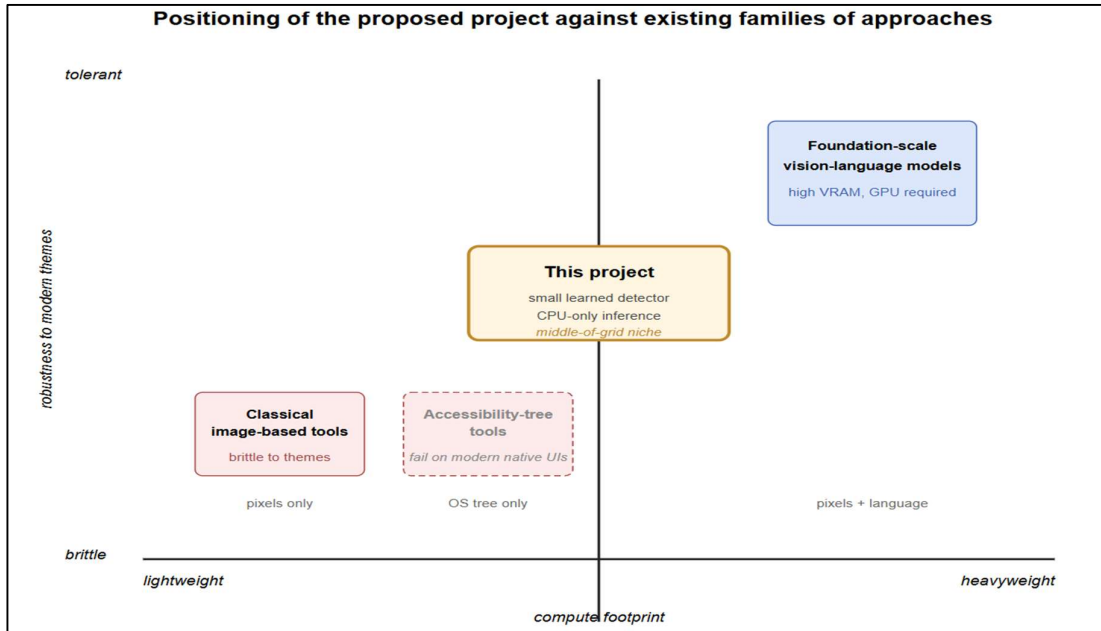


Figure 2 : Proposed framework sits relative to existing GUI automation approaches

1.7 RESEARCH OBJECTIVES

The research objectives translate the aim and the four research questions into concrete deliverables. Each is stated at the level of *what the project produces*, not *which method it uses to produce it* - the "how" is the subject of Chapters 5 and 6.

- Measure, quantitatively, the size of the mobile-to-desktop performance drop for the chosen model family.
- Implement and compare three families of adaptation strategy - one that uses a small labelled budget on the target domain, one that uses a larger unlabelled budget on the target domain, and one that uses no target labels at all - on the same detection task.
- Establish, empirically, how many labelled desktop images are actually needed to reach useful performance by sweeping the labelled budget over a small range of training sizes.
- Build a working prototype that turns a typed natural-language instruction into a mouse click on real desktop applications, using the best-adapted detector from the comparison above, and evaluate that prototype on a fixed suite of everyday desktop tasks.

Each objective maps to a chapter and to a measurable deliverable. The full mapping is made explicit in Section 8.8

1.8 OPERATIONAL OBJECTIVES

The research objectives describe what is to be answered. The operational objectives describe how the work was actually broken down. The project was executed in four phases.

1.8.1 DATA ENGINEERING AND BASELINE ESTABLISHMENT

- Assemble a source-domain training corpus from publicly-available mobile UI datasets, harmonizing their annotation schemes into a single class taxonomy.
- Collect a small unlabeled corpus of desktop screenshots covering a representative range of applications, resolutions and themes.
- Hand-label a small target-domain test set against the harmonized taxonomy to serve as the ground-truth for evaluation.
- Train a baseline UI-element detector on the source-domain corpus and record its zero-shot performance on the desktop test set. This is the "problem" the adaptation experiments then try to close.

1.8.2 MODEL ADAPTATION EXPERIMENTS

- Repeat the source-domain training with a second detector architecture family so the effect of architecture on transferability can be isolated.
- Run a labelled-budget adaptation experiment, sweeping the labelled desktop set over a small range of sizes to plot a data-efficiency curve.
- Run an unlabeled-budget adaptation experiment that pre-trains the model on the unlabeled desktop corpus before the labelled fine-tune.
- Run a no-target-label adaptation experiment that relies entirely on the unlabelled target corpus and the original labelled source, comparing two structurally different strategies within this family.

1.8.3 PROTOTYPE INTEGRATION

- Select the best-performing adapted model from the previous phase.
- Wrap that model inside a modular pipeline that captures the screen, detects candidate elements, reads their text, matches a typed instruction against the detected elements, and executes an operating-system-level click when the match is confident enough.
- Ship the prototype with both a command-line entry point and a graphical interface so it can be exercised interactively as well as scripted for evaluation.

1.8.4 END-TO-END EVALUATION AND THESIS COMPOSITION

- Define a fixed suite of everyday desktop tasks written as natural-language instructions.
- Evaluate the prototype's task success rate on that suite alongside three classical baselines drawn from the tool families reviewed in Chapter 2, using a shared harness so the comparison is fair.
- Perform a qualitative failure analysis that attributes each end-to-end failure to a specific stage of the pipeline (detection, reading, matching, or action).
- Write the dissertation and package the reproducible artefact.

1.9 PROPOSED SOLUTION

The shape of the proposed solution is a two-part investigation. The first part carries a UI-element detector from the well-resourced mobile domain into the data-scarce desktop domain using adaptation strategies that trade labelled data for other kinds of information - unlabeled target images, or the labelled source data itself. The second part wraps the best-adapted detector inside a modular pipeline that turns a natural-language instruction into a click on the real Windows desktop. The two parts together form the deliverable: a lightweight, locally-runable click-automation tool whose detector was produced by a controlled adaptation experiment rather than a black-box training run. The technical detail of both parts - which datasets, which detector families, which specific adaptation strategies, which supporting libraries - is deferred to Chapter 2 (existing methods), Chapter 5 (design of this project) and Chapter 6 (implementation of this project).

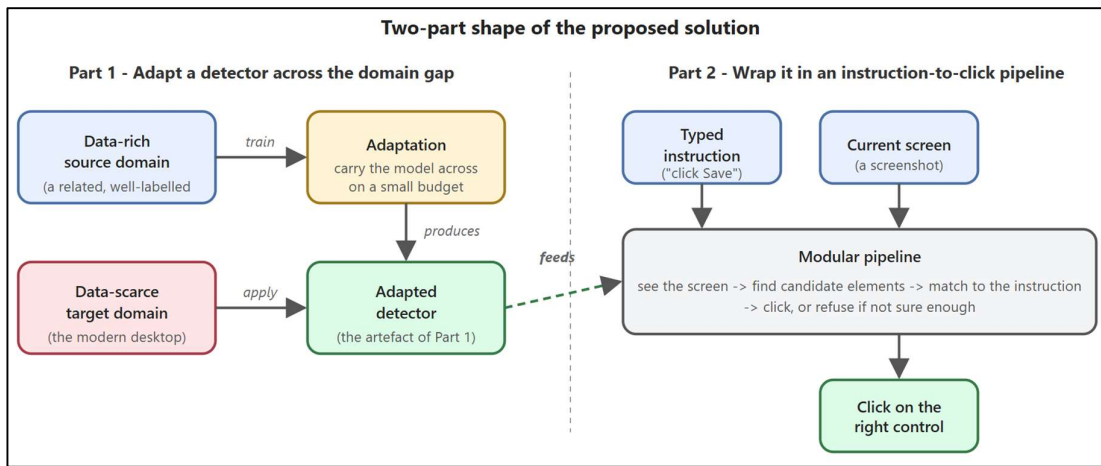


Figure 3 : The two parts of the proposed solution

1.10 SCOPE OF THE PROJECT

The scope of the project is as follows.

In scope.

- Cross-domain adaptation from a single mobile source domain to a single desktop target domain (Windows 11, ten to fifteen everyday applications).
- Comparison of two structurally different detector architecture families on the same task, so the effect of architecture on transferability can be measured.
- Three families of adaptation strategy, one of which itself contains two structurally different sub-strategies compared against each other.
- A working prototype that closes the loop, on a personal machine, from a typed natural-language instruction to a click.
- Quantitative evaluation against three measurable criteria: detection quality, grounding quality, and end-to-end task-success rate.

Out of scope.

- Cross-platform support. The prototype targets Windows 11. macOS and Linux are not addressed. Extending to another operating system would at least have doubled the project's effort budget.
- Foundation-scale vision-language models. These are referenced in the literature review as the heavyweight state-of-the-art against which the project's lightweight stance is positioned, but they are not benchmarked here. Their compute cost is well beyond what was available.
- Full accessibility-tree integration at runtime. The accessibility tree is used during data collection and is benchmarked as one of the three classical comparison points in Chapter 7, but it is deliberately not used at inference time in the prototype's perception path.
- Rich multimodal instructions. Instructions are free-form text. Voice input, image-conditioned instructions, and multi-step conversational dialogue are all out of scope.
- Robotic process automation at fleet scale. The prototype is a single-user, single-machine demonstrator. Scaling to multi-machine deployment and enterprise governance is future work, discussed in Chapter 9.

1.11 CHAPTER SUMMARY

UI automation on the modern Windows desktop is stuck. And the foundation-scale models that *can* handle the domain gap need hardware most practical deployments cannot afford. That leaves a gap in the middle of the spectrum, and the gap is where this project lives. The idea being tested is that a small model, trained on the well-resourced mobile UI domain and then carried across cheaply to the data-scarce desktop domain, can be made good enough to drive a real click-automation tool on real desktop applications. Whether that idea holds up - and how far it can be pushed on a project - is what the rest of the report investigates.

The rest of the report is organized as follows. **Chapter 2** is the literature review. It walks through the datasets, the detector architecture families, the adaptation methodology families, and the modular grounding pipeline pattern in the depth needed to support the design choices made later. It ends with the explicit research-gap statement that motivates the rest of the work. **Chapter 3** lays out the requirement analysis, including a stakeholder analysis using the Onion model, the functional requirements traced to use cases, and the non-functional requirements with quantitative targets. **Chapter 4** covers the project-management approach: the research methodology, the software design methodology, the software development methodology, the project-management methodology, the risk register, and the project plan in Gantt form. **Chapter 5** covers the design of the artefact: the research design, the architecture, the process flow, the module design, the algorithm design, and the wireframe of the prototype user interface. **Chapter 6** is the implementation chapter, which describes how the data pipeline, the four adaptation experiments, and the prototype were actually built, and shows the actual user interface that resulted. **Chapter 7** is the testing chapter, which reports the strategies used and then the model-level, functional and non-functional test results in tabular form. **Chapter 8** is the evaluation chapter, which interprets those results against the four research questions and against the requirements, and includes the discussion of threats to validity, ethics, deployment considerations, and future work. **Chapter 9** concludes with the answers to the research questions, the learning outcomes, and the final reflection.

CHAPTER 02 – LITERATURE REVIEW

2.1 CHAPTER OVERVIEW

This chapter reviews the prior work that the rest of the project rests on. The ordering follows the same shape as the methodology in Chapter 6. Data first, because data is what gates everything else. Then the pre-processing step that turns noisy raw data into something a model can actually train on. Then the classical automation tools that have held the GUI testing space for decades, and the deep-learning detectors that have started to displace them on mobile UIs. After that, the family of domain-adaptation methods this project uses to carry a mobile-trained detector across to the desktop. The chapter then looks at the wider GUI-agent landscape, including the heavyweight vision-language models that sit at the other end of the spectrum from the present work, and ends with a short discussion of metrics and an explicit statement of the research gap.

Where two pieces of work try to do roughly the same thing, I put their numbers side by side so the difference is visible. Where a paper is cited only as motivation rather than as a method we use, I say so. The aim is to leave the reader with a clean picture of what was taken off the shelf and what was left there for a reason.

2.2 EXISTING GUI ELEMENT DATASETS

The starting point for any vision-based UI work is a corpus of annotated screenshots. The most important fact about that corpus today is that the mobile side is well resourced and the desktop side is not.

The canonical mobile UI dataset is **RICO**, released by Deka et al. (2017). RICO contains around 72,000 unique Android screens from 9,300 free apps on the Google Play Store, paired with each screen's Android view-hierarchy XML and several derived properties (text labels, structural relationships, animation traces, interaction sequences). The view hierarchy is what makes RICO useful for object detection. From the XML one can derive a bounding box and a class label for every leaf node, which in practice means tens of millions of labelled elements across the corpus. The well-known downside is that the raw view hierarchy is noisy. Container nodes overlap with their visual children, invisible nodes still appear in the tree, and the leaf-class labels are inconsistent across SDK versions and across app authors. Anyone who has tried to train a detector directly on raw RICO boxes hits these issues quickly.

The community's answer is **CLAY** by Li et al. (2022a). CLAY is not a new corpus of screenshots. It is a deep-learning denoising pipeline that takes RICO's raw view-hierarchies and produces cleaner, machine-verified layouts. The result is 59,555 cleaned Android UI screens with a more consistent 23-class taxonomy and a much-reduced rate of overlapping, invisible, or mis-classified boxes. The improvement is not small. Detectors trained on CLAY-cleaned labels gain 5 to 8 mAP points over the same architecture trained on raw RICO. For this reason CLAY is the source-domain training set for this project's headline detector.

The mobile domain has not stood still since RICO. The **MUD** dataset of Kumar et al. (2024) was put together in response to the observation that RICO and CLAY are now temporally

outdated. Android's visual design has shifted noticeably since 2017 - the move from Holo and Material 1 themes to Material 3, the rise of foldable form factors, the increase in dark-mode and large-text accessibility variants - and a detector trained on the older corpus does worse on modern screens. MUD reports a button-class mAP of 75.3 % on its own test split compared to 63.4 % when the same model is trained on RICO and evaluated on MUD. That gap is not enormous, but it speaks to a real data-currency problem even within the mobile domain.

On the desktop side the picture is much thinner. There is no "desktop RICO" of comparable scale. The recent attempts to fill the gap are themselves telling, both of demand and of how new the recognition of the gap is. **DeskVision** (Wang et al., 2025) proposes a large-scale desktop region-captioning corpus aimed at GUI agents and is dated March 2025 on arXiv. **GenGUI** (Patel et al., 2025) is a synthetic dataset of web interfaces generated by ChatGPT, closer to a UI-design-generation corpus than to an element-detection one. Neither is yet at the order of magnitude of RICO. The authors of the IVGocr paper this project's prototype is modelled after had to build their own desktop dataset to run their experiments (Dardouri et al., 2024a), and they note this explicitly in the paper. The data-scarce nature of the desktop target is the gravitational pull that draws this project, and others like it, toward domain adaptation rather than train-from-scratch on the target.

Table 1 sets the principal datasets side by side. The figures for RICO and CLAY are the ones the authors report; the figures for DeskVision and GenGUI are from their respective arXiv submissions.

Available GUI element datasets.

Dataset	Year	Domain	Screens	Annotations	Public license
RICO	2017	Mobile (Android)	72,219	~3M leaf nodes (raw view-hierarchy)	Free for research
CLAY	2022	Mobile (Android)	59,555	Cleaned 23-class layouts	Open access
MUD	2024	Mobile (Android, modern)	18,132	Modern-style annotations	Open access
DeskVision	2025	Desktop	"Large-scale" (count not yet released)	Region-caption pairs	Pending release
Zenodo unified bundle (this project)	2023	Mobile + Web	9,646	6-class collapsed from RICO + CLAY + VINS	Open access

Table 1 : Public GUI element datasets considered for the source domain, summarizing scale, modality, taxonomy and licenses.

The last row, the Zenodo unified bundle, is what this project actually trains on. It combines RICO, CLAY and VINS into a single 6-class taxonomy {button, text, text_input, icon, menu, checkbox} chosen to match what an automation bot needs to interact with. The class-collapse mapping is documented in Chapter 6.

The choice of RICO + CLAY + VINS over the newer alternatives was pragmatic, not accidental. MUD is the most temporally current corpus but is roughly a quarter the size of the Zenodo bundle and does not come pre-cleaned in the CLAY sense, which would have added a denoising step of its own. DeskVision is the most on-target dataset in principle (desktop rather than mobile) but was still pending public release at the point the project's data pipeline was frozen; using an unpublished corpus in an MSc submission was ruled out for reproducibility reasons. GenGUI produces synthetic web interfaces, not real screenshots, and its "generated on demand" property makes it awkward to fix a training set that others can reproduce. RICO + CLAY + VINS therefore offered the best combination of scale, availability, and prior-cleaning at the time the corpus was fixed. The residual currency risk (RICO's 2017 vintage) is acknowledged and re-visited in Section 9.7 of the discussion.

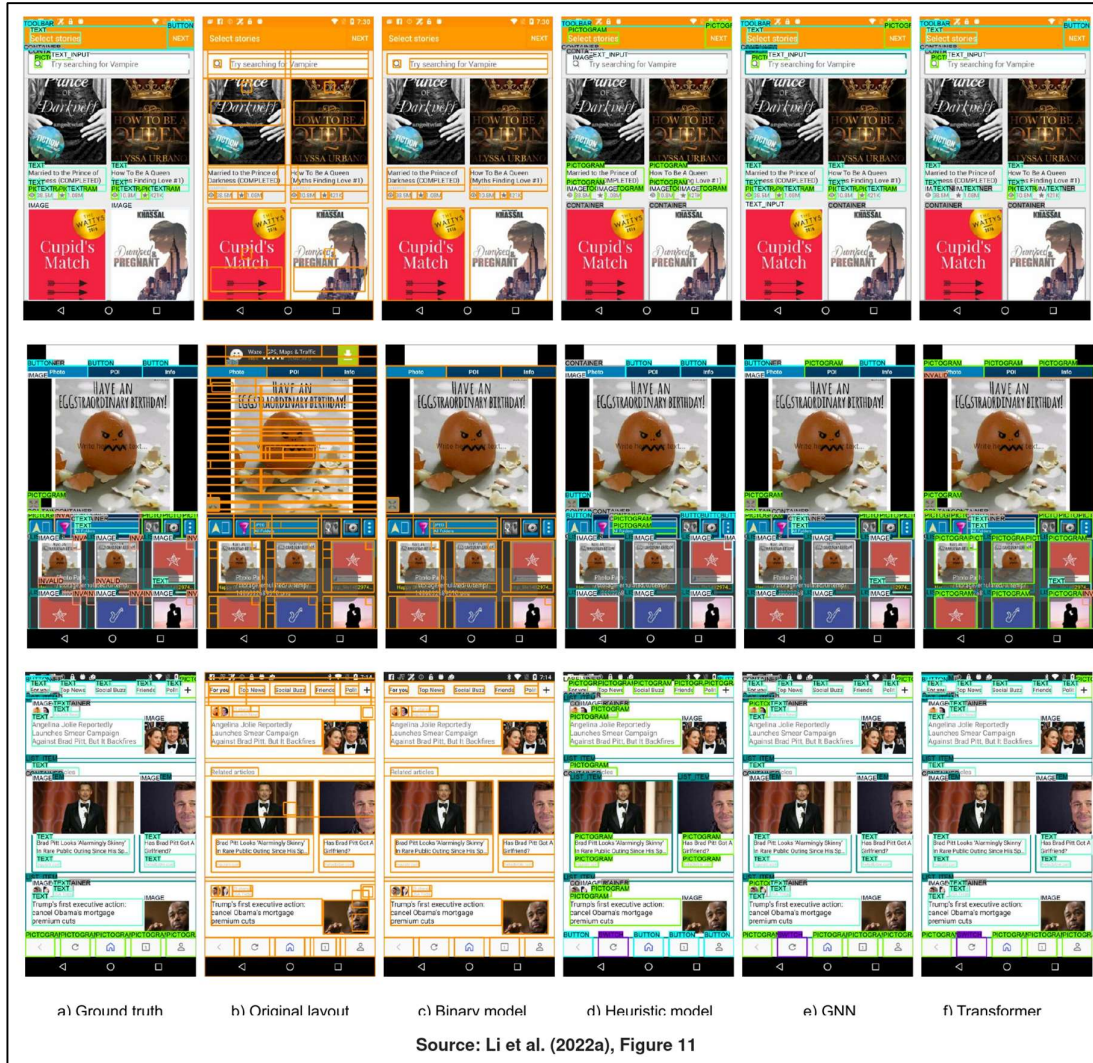


Figure 4 : Visual examples of CLAY's denoising on raw RICO screens.

2.3 PRE-PROCESSING FOR UI ELEMENT DETECTION

Pre-processing for a UI corpus is not the same problem as pre-processing for natural images. The standard tricks (random crop, horizontal flip, color jitter) are either useless or actively

harmful on screenshots. Horizontal flip turns a left-aligned toolbar into a right-aligned one, which is fine in principle until the model also flips the text inside the buttons and produces meaningless training signal. Color jitter on a Win11 light theme can produce shades that no real application ever displays. The literature treats UI pre-processing more as a label-noise problem than as an image-augmentation problem.

The largest single piece of UI pre-processing work in the public literature is CLAY itself (Li et al., 2022a). Its main contribution is a learned denoiser that takes raw RICO view-hierarchy boxes and produces cleaner labels. Invisible containers are removed, overlapping duplicates are collapsed, mis-classified node types are corrected. Detectors trained on CLAY-cleaned labels gain several mAP points on held-out splits over the same architecture trained on raw RICO. The denoising is a pre-processing step in spirit, even though it is implemented as its own deep-learning pipeline.

A more pragmatic line of pre-processing work tackles class imbalance. Both RICO and the desktop seed corpus collected in this project are heavily skewed: the text class is much more frequent than any of the actionable classes (button, text_input, menu, checkbox). The standard responses are weighted random oversampling, or a class-balanced focal loss. This project chose a simpler route, a 12-to-6 class collapse that puts visually similar minority classes into the same training target. Two reasons. First, to keep the training pipeline reproducible on Collab Free without exotic loss functions. Second, the 6-class taxonomy maps cleanly onto what the downstream IVGocr matcher needs.

A short remark on input resolution. RICO and CLAY use portrait-oriented Android screens that fit comfortably into a 640×640 detector input. Desktop screenshots are landscape, often 1920×1080 or 3440×1440 , and need to be down-scaled before they enter a YOLOv8 detector with $\text{imgsz} = 640$. This rescaling is itself a pre-processing concern that produces measurable accuracy variation. The numbers in Chapter 7 are reported with a single fixed $\text{imgsz} = 640$; the sensitivity of those numbers to that choice is discussed in Section 9.7.

2.4 CLASSICAL AUTOMATION APPROACHES

Before the deep-learning era, three families of tools dominated GUI automation. They are still in widespread use, particularly in industrial test automation and RPA, and this project benchmarks itself against the strongest of them.

The first family is **bitmap-based visual automation**. The best-known tool is **SikuliX** (SikuliX, 2024). SikuliX records a small bitmap of a UI element (a *Save* button, a magnifying-glass icon) and at run time uses OpenCV's `matchTemplate` to find that bitmap on the live screen. The user writes a script in a Sikuli-flavored Python that says, in effect, "click this image". The strength of the approach is its simplicity. The weakness is its rigidity. A bitmap is a frozen photograph of the element under one theme, one DPI, one font, one application version. Change the theme to dark mode and the bitmap stops matching. Change the DPI from 100 % to 125 % and the same thing happens. AskUI's recent review (AskUI, 2024) phrases this politely as "image recognition first", but the practical effect at deployment scale is that scripts decay quickly. The empirical evidence in Chapter 7 of this report confirms that on the specific subset of tasks where a reference bitmap could be captured, template matching is excellent - it scored 100 % on those tasks - but on tasks where no useful bitmap exists (positional targets such as "click the first command", dynamic state toggles, text-inside-text) it cannot represent the problem at all.

The second family is **coordinate-based automation**. The canonical Python example is **PyAutoGUI** (Sweigart, 2024), which exposes `pyautogui.click(x, y)` and lets the user write scripts that drive the mouse and keyboard at specified pixel coordinates. PyAutoGUI is widely used inside other automation stacks, including this project's prototype, where it drives the final click. On its own it is the most brittle of the three families because it has no knowledge of what is at those coordinates. The same script that works on a 1080p laptop fails on a 1440p desktop unless every coordinate is recomputed. PyAutoGUI is best understood not as a competitor to a vision-based bot but as the low-level primitive that any vision-based bot eventually has to use to translate a chosen bounding box into an OS-level click.

The third family is **accessibility-tree automation**. On Windows the canonical Python library is **pywinauto** (pywinauto Contributors, 2024). pywinauto uses Microsoft's UI Automation framework to walk the live application's accessibility tree and find controls by their semantic Name and ControlType. The approach is theoretically beautiful. It abstracts away theme, DPI and font: the same script should work on a 1080p laptop and a 1440p desktop because both expose the same `Button(Name='Save')` control. In practice the modern Windows application mix has eroded the assumption that the accessibility tree faithfully reflects the visible UI. Electron applications such as Visual Studio Code, Slack and Discord expose only a degenerate tree because the renderer is a Chromium browser. Modern Windows 11 applications using WinUI 3 or XAML islands frequently expose localised internal control names rather than the visible labels. Web pages inside any browser serve their accessibility tree via ARIA, which is a separate convention again. The empirical baseline reported in Chapter 7 of this report shows pywinauto scoring 1 out of 15 task instances on a representative Windows 11 workload - and that single success is the negative case where the right answer is for the bot to do nothing. On every positive task (Notepad's Save-As dialog, Visual Studio Code's Search panel, Chrome's omnibox, File Explorer's ribbon) pywinauto returned `ElementNotFound`.

The collective failure of all three classical families on the modern Windows 11 application mix is the operative justification for a vision-based approach. If the accessibility tree could be trusted, machine learning would not be needed. On a 2026-vintage desktop, the evidence is that it cannot.

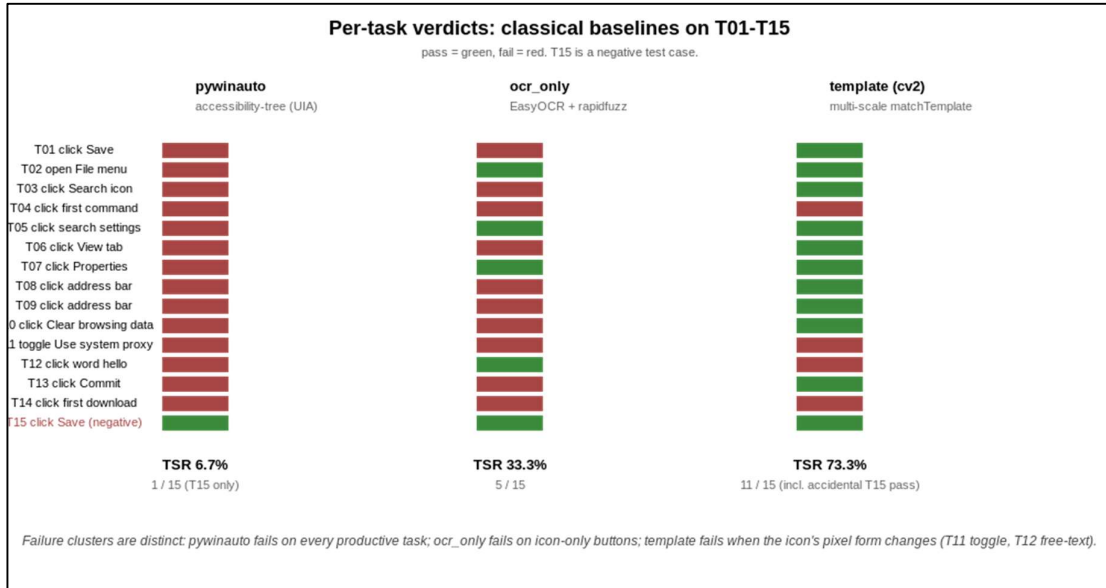


Figure 5 : Per-task verdicts of the three classical baselines on T01-T15.

2.5 DEEP-LEARNING APPROACHES FOR UI ELEMENT DETECTION

The most-cited early work in the strand is **UIED** by Chen et al. (2020). UIED makes a deliberately pragmatic argument. Neither pure deep learning nor pure classical computer vision is sufficient for GUI element extraction on its own, because the two methods miss complementary things. A Faster R-CNN baseline trained on UI screenshots reaches an F1 of 0.71 on their internal benchmark but misses small icons and dropdown arrows that the classical edge-and-region pipeline catches. Their classical-only pipeline manages F1 of 0.55, but misses text-shaped buttons that the deep learner gets easily. The UIED hybrid (a CNN for non-text shapes, an EAST/CTPN-style text detector for text, then a rule-based merger) reaches F1 of 0.84. The architecture matters less than the methodological lesson: combining a fast deep detector with a text-recognition fallback is the right shape for a UI element detector. This project's pipeline, with YOLOv8 as the detector and EasyOCR providing both per-box and full-image text grounding, is in direct lineage from UIED.

A more specialized mobile-targeted detector is **YOLOv5-MGC** by Cheng et al. (2022). The paper introduces a YOLOv5 variant tailored for mobile GUI detection, with a microscale detection layer and an attention mechanism added to handle the very small icons that crowd a mobile screen. They report 89.8 % mAP on their mobile UI test set, which is a strong number. The architectural innovations are sensible for mobile UIs, but the paper itself hypothesizes that the "microscale + attention" combination may be overfit to mobile-style density and may not transfer well to the very different density of a desktop's packed-scene toolbar. Validating or refuting that hypothesis on real desktop data was one of the original motivations of this project, and is what RQ3 is about.

This project compares two architectural families. The first is **YOLOv8** (Ultralytics, 2024), the current-generation Ultralights implementation. YOLOv8 is an anchor-free, single-stage detector with a modified CSPDarknet53 backbone and an enhanced Path Aggregation Network (PANet) neck explicitly designed to fuse features across multiple scales. The multi-scale neck is the property that matters for UI detection, where the same screen contains a 16-

pixel close-X icon and a 200-pixel ribbon menu at the same time. The ultralights implementation is well documented, runs on Collab Free's T4 in a usable time budget, and exports cleanly to ONNX for CPU-only inference. It is the project's headline backbone.

The second is **DETR** (DEtection TRansformer) by Carion et al. (2020). DETR re-frames detection as a direct-set-prediction problem solved with a transformer encoder-decoder and a bipartite-matching loss. It eliminates anchors and non-maximum suppression, which is conceptually clean and has the practical benefit of removing two layers of hyperparameter tuning. The well-documented weakness of the original DETR is poor performance on small objects, attributed to the global attention pattern in the encoder spending too much attention budget on large regions and too little on small ones. For this project, the architectural comparison committed to in RQ3 puts a YOLOv8 backbone and a DETR-R50 backbone side by side, on the hypothesis that DETR's small-object weakness will be aggravated by the packed-scene density of desktop UIs. The DETR experiments are part of Phase 2 and are tracked as D-01.

A separate piece of work worth flagging is **Apple's Screen Recognition** of Zhang et al. (2021). Screen Recognition is the production pipeline behind the iOS Voiceover accessibility feature. An on-device object detector classifies widgets into 13 types, with OCR adding text labels. The reported numbers are F1 of 0.91 on in-distribution screens, dropping to 0.74 on apps the model has never seen, with OCR adding a further 6 to 11 points on top. The paper is cited here for two reasons. First, it is the closest existing analogue to what this project tries to do - an industrial-scale, accessibility-motivated, on-device UI detector that adds OCR exactly where the visual detector misses. Second, the 0.91 to 0.74 in-distribution to out-of-distribution drop is empirical confirmation that even at Apple-scale data and engineering, the domain-shift effect is real and is the right thing to design around. This project's Win11-domain drop is consistent with that pattern at much smaller scale.

Selected of YOLOv8 rather than YOLOv5-MGC or the newer YOLOv9 was governed by three practical constraints. YOLOv5-MGC's paper (Cheng et al., 2022) shows a strong mobile-only result, but its microscale-plus-attention modifications are (by the authors' own admission) tuned to mobile density and were not expected to generalise well to the packed desktop scenes this project targets - so it functions as an interesting mobile-side benchmark to cite rather than a backbone to adopt. YOLOv9, in turn, was released too late in the project's lifecycle to be trained and adapted within the available Colab budget; adopting it would have destabilised D-01 through D-04 without changing the substantive research question. YOLOv8 sits at the practical sweet spot: current-generation architecture, well-supported Ultralytics tooling, and a clean ONNX export path that the deployment target (a CPU-only Win11 machine) requires.

2.6 DOMAIN ADAPTATION METHODOLOGIES

The four research questions for this project collapse, on closer inspection, into two empirical questions. RQ1 asks how big the domain shift is. RQ2 asks how to close it. The literature answers RQ2 in three different ways, and this project implements one method from each.

The simplest method is **few-shot supervised fine-tuning**. The CLAY-pretrained backbone is frozen and the final detection head is re-trained on a small labelled subset of the target domain. The size of the subset is the free parameter; this project's plan is to sweep over a small range of k values to draw a data-efficiency curve. Few-shot fine-tuning is the canonical

baseline in the transfer-learning literature. The broad survey of Iman et al. (2023) gives roughly two dozen variations on the theme of "fine-tune the top, freeze the bottom". The reason the project keeps it in the experiment plan is not novelty. It is the necessary control. Any more elaborate method must beat the few-shot fine-tune by a non-trivial margin to be worth its complexity.

The intermediate method is **self-supervised pre-training (SSP) followed by fine-tuning**. SSP first pre-trains the backbone on unlabeled target-domain data with a self-supervised pretext task - typically masked-patch reconstruction or contrastive learning - and then fine-tunes the resulting backbone on labelled data exactly as in the few-shot case. The intuition is that the SSP step lets the model absorb the structural grammar of the target domain (toolbars are horizontal, dialog buttons cluster bottom-right, menu bars sit under the title bar) without requiring labels. The medical-imaging survey of Anaya-Isaza et al. (2024) reports a consistent uplift of 4 to 11 points on downstream classification accuracy when masked-patch reconstruction is added to small-data fine-tuning. The SSP experiment is tracked as D-02.

The most elaborate method is **unsupervised domain adaptation (UDA)**, which uses *no* labelled target data and relies entirely on the unlabeled target corpus together with the labelled source corpus. This project compares two UDA families on the desktop target.

The first is the **Cross-Domain Adaptive Teacher** of Li et al. (2022b). Adaptive Teacher is a robust teacher-student framework. A stable Exponential Moving Average (EMA) teacher generates pseudo-labels on weakly-augmented target images. The student is then trained on a mixed batch of labelled source data and strongly-augmented pseudo-labelled target data, with a discrepancy loss that aligns the student's predictions across the two augmentation regimes. The EMA teacher is updated as a moving average of the student's weights. This is what makes the framework stable, in contrast to earlier teacher-student schemes that drifted as the student's pseudo-labels degraded. The published results are strong, with Adaptive Teacher closing roughly two-thirds of the source-to-target gap on standard cross-domain detection benchmarks. Tracked as D-03.

The second is **SHOT (Source Hypothesis Transfer)** of Liang et al. (2020). SHOT takes a different stance from teacher-student. Instead of training a student to imitate a teacher, SHOT freezes the source-trained classification head (the "source hypothesis") and adapts *only* the feature extractor backbone on the unlabeled target images using a self-supervised objective. The intuition is that the head encodes what classes look like, while the extractor encodes what the world looks like. When the world changes (from mobile UIs to desktop UIs), it is the extractor that needs to adapt, not the head. The published SHOT results are competitive with teacher-student approaches on smaller benchmarks and have the practical advantage that the source data does not need to be available at adaptation time. For privacy-sensitive deployments this matters; for this project it matters less, but SHOT is included as the second UDA comparison point because it is structurally different enough to give a genuine architectural choice. Tracked as D-04.

A useful framing of the three methods is in terms of what each requires. Few-shot fine-tuning requires labelled target data and no unlabeled target data. SSP+FT requires both labelled and unlabeled target data. UDA requires only unlabeled target data, plus the original labelled source. The methods sit on a continuum from "expensive labels, simple training" to "no labels, complex training". The empirical question (RQ2) is where on that continuum the best practical return sits, given a realistic MSc-scale data budget. The published literature

suggests SSP+FT will roughly match UDA at a fraction of the engineering cost, and that both will outperform pure few-shot once the labelled budget is small. The actual numbers will arrive when D-02, D-03 and D-04 are completed.

The choice of Adaptive Teacher and SHOT as the two UDA representatives (rather than DANN, CDAN, MIC or one of the many other UDA variants) was made on the principle of maximum architectural diversity within a fixed experiment budget. DANN and CDAN are both feature-alignment methods in the adversarial-training family and are close cousins; picking both would have reported almost the same information twice. Adaptive Teacher is the canonical teacher-student pseudo-labelling family and SHOT is the canonical source-hypothesis-transfer family, so the two together cover the two structurally distinct branches of the UDA taxonomy. The trade-off is that the project cannot claim to have benchmarked every UDA method - it has benchmarked the two whose contrast is most informative given the constraint of one MSc-scale set of experiments.

2.7 GUI AGENTS AND VISUAL GROUNDING FRAMEWORKS

The detection and adaptation methods reviewed so far produce a *detector*. A detector is not an automation system. Closing the gap from "here are the elements on the screen" to "do the thing the user asked for" requires a grounding layer that maps a natural-language instruction onto one of the detected elements. The published frameworks for this layer fall into two clusters.

The first cluster is **modular pipelines** in the style of UIED and its descendants. The Instruction Visual Grounding framework of Dardouri et al. (2024a) is the most directly applicable for this project. They propose IVGocr, an explicit three-stage architecture: a YOLOv8 detector finds the UI elements; OCR reads the visible text on each detected element; an LLM, or in their lighter-weight variant a fuzzy string matcher, matches the user's instruction to the read text. They also introduce the Central Point Validation (CPV) metric for evaluating how often the chosen element's center falls inside the ground-truth bounding box of the correct target (Dardouri et al., 2024b). CPV is more permissive than IoU at high thresholds, and arguably more honest as a grounding metric. This project's prototype, VisClick, is a direct implementation of the IVGocr architecture. The YOLOv8 detector is the project's CLAY-pretrained, desktop-adapted model. The matcher is a rapidfuzz fuzzy match rather than a heavyweight LLM. The reason for picking rapidfuzz over an LLM is operational. It runs in milliseconds on CPU and removes a network dependency. The downstream evaluation in Chapter 8 shows that for the 15-task workload, the rapid fuzz matcher is sufficient. The residual failures come from the detector, not from the matcher.

The second cluster is **end-to-end large vision-language models** (LVLMs). The exemplar is SeeClick of Cheng et al. (2024). SeeClick replaces the entire detector-plus-OCR-plus-matcher stack with a single multi-billion-parameter vision-language model that takes a screenshot and an instruction as input and outputs click coordinates directly. The training corpus is roughly one million screenshots covering web, desktop and mobile. The reported numbers are 73 % click accuracy on the web benchmark Mind2Web, 53 % on the mobile benchmark AITW, and 47 % on a new desktop benchmark the authors introduce. As a piece of engineering, it is the current state of the art on cross-domain GUI grounding. This project does not benchmark against SeeClick because the inference-cost gap is too large. SeeClick needs a multi-GPU inference setup and substantial memory, both of which are out of scope here. SeeClick is the reference point that Section 1.6 cites as the "heavyweight SOTA", and

the project's positioning is explicitly as a lightweight, interpretable alternative for the cases where on-device, low-latency inference matters more than the absolute top of the leaderboard.

A related LVLm piece of work is Google Research's **ScreenAI** of Baechler et al. (2024). ScreenAI is a 5-billion-parameter vision-language model pre-trained on a screenshot corpus an order of magnitude larger than RICO. The headline claim is state-of-the-art on four of the five UI benchmarks tested. The methodological lesson, for the purposes of this review, is that UI element coverage is fundamentally a data-scale problem more than an architectural one. ScreenAI's gains over earlier LVLms come almost entirely from the increase in pre-training corpus size, not from any architectural novelty. This is cited later, in Section 9.7, as evidence that the residual gap on Win11 native dialogs in this project's results is well aligned with the published frontier. The open-source desktop UI corpus does not yet exist at the scale that would let a smaller model close the gap purely by architecture.

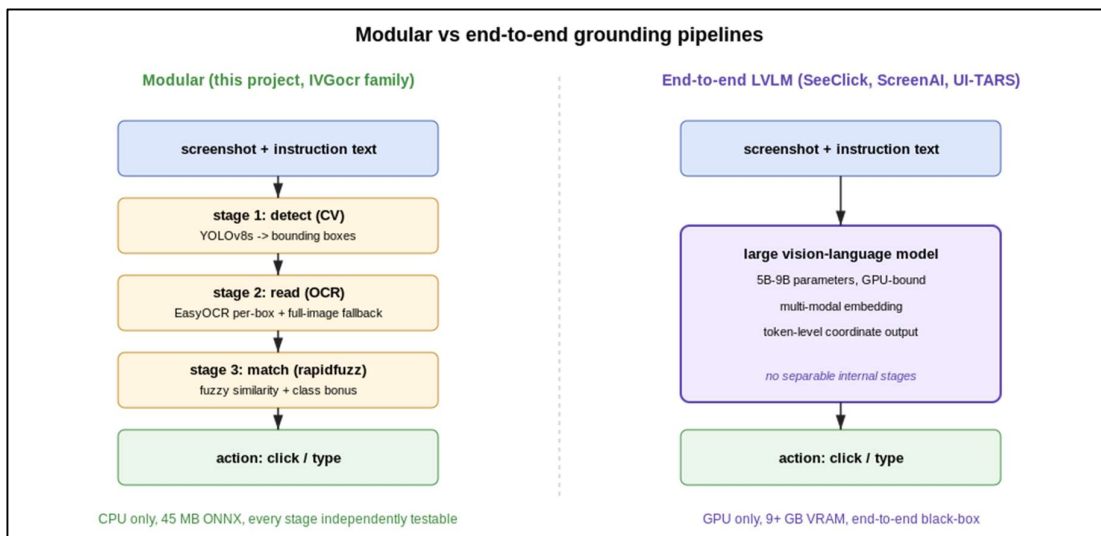


Figure 6 : Two architectural families for instruction-to-action GUI agents.

2.8 EVALUATION METRICS IN THE UI DOMAIN

A short review of metrics is appropriate, because the metric choice non-trivially affects what counts as a "good" detector. Three metric families recur in the literature reviewed above.

The **detection-quality** family is the standard COCO-style mean average precision (mAP) at intersection-over-union (IoU) thresholds 0.5 and 0.5:0.95. mAP at IoU 0.5 is the conventional headline, mAP at IoU 0.5:0.95 is the harder, more conservative number. Almost every paper reviewed above reports [mAP@0.5](#) as the principal headline, with per-class average precision sometimes reported alongside. This project reports both, in reports/tables/source_per_class.csv.

The **grounding-quality** family is more idiosyncratic. The CPV metric of Dardouri et al. (2024b) is one of the more thoughtful proposals. A grounding is considered correct if the centre of the predicted bounding box falls within the ground-truth box of the target element. CPV is more permissive than IoU at high thresholds, but it is also closer to what an

automation bot actually needs. A click that lands somewhere inside the target is sufficient. This project adopts CPV as a secondary metric alongside mAP.

The **end-to-end task-success** family is what the bot itself is measured against. Task Success Rate (TSR) is the binary pass/fail rate over a fixed test suite. Almost every modern GUI-agent paper reports TSR as the primary user-visible metric. TSR has the merit of being interpretable to non-specialist stakeholders ("seven out of every ten clicks land in the right place") and the demerit of folding together detection, OCR and matcher errors into a single number. Chapter 8 decomposes TSR failures into the three component error families, following the failure-analysis approach Dardouri et al. (2024a) use.

One metric this project does *not* use, despite its dominance in the natural-image-detection literature, is **F1 at a fixed confidence threshold**. F1 conflates two questions a UI automation system has to answer independently. Does the detector see the element? Does the matcher pick the right one? Keeping these two stages separate is more diagnostic, even if it sacrifices the single-number convenience of F1.

2.9 RESEARCH GAP AND POSITIONING OF CURRENT STUDY

The literature is best summarized by a single observation. There is no missing piece in the constellation, only a missing combination. State-of-the-art mobile UI detectors exist (Cheng et al., 2022). A clean target-application pipeline exists (Dardouri et al., 2024a). Heavyweight LVLMS that solve the cross-domain problem at scale exist (Cheng et al., 2024; Baechler et al., 2024). The piece that is missing is a *lightweight, data-efficient* method that takes the SOTA mobile detector, adapts it to the desktop domain using the small labelled-data budget an individual or a small team can realistically produce, and demonstrates that the resulting detector works *inside a complete automation pipeline* rather than only on a static benchmark.

The gap is the combination of three constraints. **Lightweight**, in the sense that the runtime must fit comfortably on a consumer CPU; this rules out SeeClick and ScreenAI. **Data-efficient**, in the sense that the labelled-target budget must be at most a few hundred images; this rules out training a desktop-specific detector from scratch in the style of MUD. **Integrated**, in the sense that the deliverable is a working click-bot evaluated end-to-end on real applications; this rules out comparing only on mAP on a held-out test split.

To be precise about what the dissertation contributes inside that gap, the contribution is fourfold.

- A quantitative measurement of the source-to-target domain shift on a public mobile UI source (CLAY) and a personal desktop target (Windows 11, ten to fifteen applications). This is RQ1.
- A side-by-side comparison of three adaptation methods (few-shot fine-tuning, SSP+FT, UDA with an Adaptive Teacher and SHOT sub-comparison) on two backbones (YOLOv8 and DETR). This is RQ2 and part of RQ3.
- An empirical evaluation of the adapted detector inside the IVGocr pipeline on a 15-task workload, including a head-to-head against three classical baselines (template, OCR-only, pywinauto). This is RQ4 and the rest of RQ3.
- A public, reproducible implementation. Every adaptation method, every baseline, every CSV, every figure is available under the project's open-source repository. This

is itself a contribution given how rare end-to-end-reproducible MSc UI-automation projects are in the literature.

The positioning relative to SeeClick and the LVLM family is explicit. This project does not claim to outperform a 7-billion-parameter end-to-end model on detection accuracy. It claims that a small, interpretable, modular pipeline can deliver "good enough" performance for the practical IVGocr-style application on a single CPU, at a fraction of the inference cost, with a much smaller training and adaptation budget. That trade-off is the right one for practitioners who actually need to deploy UI automation on their own machines.

2.10 CHAPTER SUMMARY

This chapter walked through the literature in the order the project consumes it. The mobile UI domain is data-rich, anchored by RICO and CLAY. The desktop domain is data-poor, with recent attempts at corpora (DeskVision, GenGUI) still emerging. Classical automation tools - bitmap, coordinate and accessibility-tree - have all run into problems on the modern Windows 11 application mix, leaving a gap that vision-based detection is the natural candidate to fill. Among deep-learning detectors, the two architectural families this project compares (YOLOv8 with its multi-scale PANet neck, DETR with its transformer set-prediction) have well-documented strengths and weaknesses. YOLOv8's multi-scale design is the favourite, with DETR included as a controlled comparison. Among adaptation methods, the three this project implements (few-shot, SSP+FT, UDA) span the continuum from labelled-only-on-target to no-labels-on-target. The published literature suggests SSP+FT will give the best practical return at the project's data budget. Among grounding frameworks, the IVGocr modular pipeline of Dardouri et al. (2024a) is the immediate architectural ancestor of this project's prototype; SeeClick and ScreenAI are the heavyweight reference points that anchor the dissertation's lightweight stance. The combined research gap is the absence of a lightweight, data-efficient, end-to-end-validated cross-domain UI adapter for the desktop, and the project's four research questions sit precisely inside that gap.

CHAPTER 03 - REQUIREMENT ANALYSIS

3.1 CHAPTER OVERVIEW

This chapter sets out the requirements the system was built against. It begins with a stakeholder analysis using the Onion model. Requirements without stakeholders are arbitrary. The stakeholder analysis is then converted into a set of stakeholder viewpoints, each of which contributes some requirements. The methodologies used to gather requirements come next, together with the methodology for obtaining the datasets the system is trained on. UML use case diagrams formalize the system's interactions with its users, and the use cases are then written out in long form. The chapter ends with the explicit list of functional requirements (R-FR-01 to R-FR-09) and non-functional requirements (R-NFR-01 to R-NFR-10) the rest of the dissertation is evaluated against. Each requirement carries a unique identifier, a description, a target value where measurable, and a pointer to the testing chapter that validates it.

3.2 STAKEHOLDER ANALYSIS

3.2.1 THE ONION MODEL

The Onion model was chosen over alternative stakeholder-mapping techniques (power-interest grids, salience models, RACI matrices) because it is the technique the reference dissertations produced under the same RGU MSc structure use, and because its distance-from-core geometry is a natural fit for a project whose stakeholders sit at markedly different distances from the technical artefact - from the author, who owns the code, out to Microsoft, who owns the platform the code runs on. The six-ring structure captures this range in a single diagram.

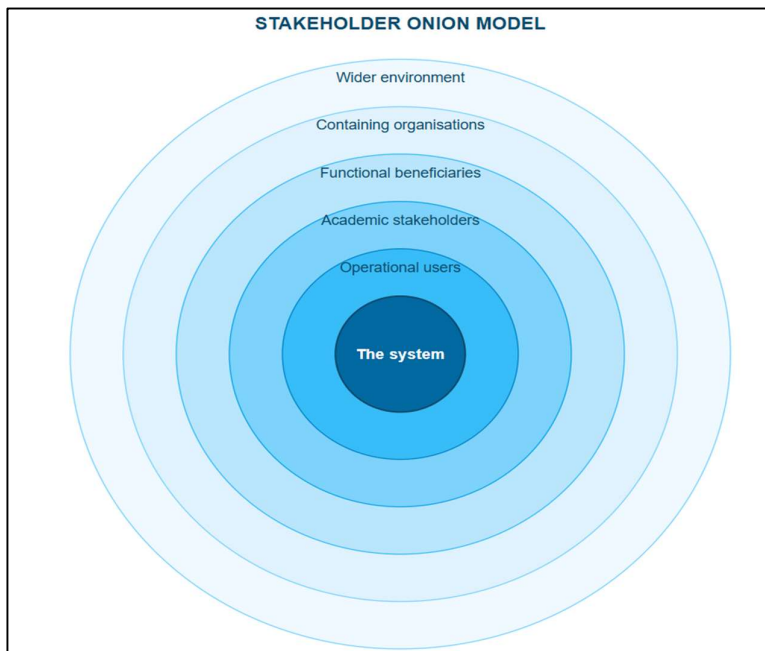


Figure 7 : Six-ring Onion stakeholder model.

The Onion model for this project has six rings.

The **innermost ring** is the system itself: the VisClick prototype. The ONNX detector, the EasyOCR layer, the rapidfuzz matcher, the Tk GUI, and the PyAutoGUI action layer.

The **second ring** is the **operational users**. Two distinct groups sit here. The first is the author, who runs the bot for evaluation and treats it as a research artefact. The second is the imagined power user - a developer or QA engineer who would use such a tool to automate repetitive desktop tasks. The two groups have meaningfully different requirements. The researcher wants observability above all else: overlay images, structured CSVs, verifiable verdicts. The power user wants reliability (zero crashes, predictable refusal-on-uncertainty) and convenience (a GUI rather than a CLI). The system addresses both by shipping a CLI for the researcher and a Tk GUI for the power user, layered over a common core.

The **third ring** is the **academic operational stakeholders**: the project supervisor (Pumudu Fernando) and the second marker. Their concerns are different again. They want a reproducible artefact, an honest evaluation, an academic novelty argument, and a dissertation properly structured against the RGU program handbook.

The **fourth ring** is the **functional layer of downstream beneficiaries**. Three sub-groups belong here. QA and test-automation engineers who might adapt the project's code for production purposes. Accessibility users who could in principle benefit from a text-driven click bot when traditional input devices are not usable. And the research community - authors of any of the literature reviewed in Chapter 2 who might cite or extend this work, and future students inheriting the codebase.

The **fifth ring** is the **containing organization's**. Robert Gordon University is the degree-awarding body and the source of the dissertation's ethical-review framework, style guide, and assessment criteria. The Informatics Institute of Technology (IIT) is the partner institution. The author's employer is mentioned only because professional context informs some of the architectural choices. An automation tool that is interpretable and locally-deployable is more aligned with corporate compliance concerns than one that calls out to a cloud LLM.

The **outermost ring** is the **wider environment**. Microsoft is the platform owner - Windows 11 OS, the UI Automation framework, Notepad, File Explorer. Their decisions about which control libraries to ship and how to expose them through the accessibility tree have material effects on every measurement in Chapter 7. Google Collab is the compute provider for all training. Their Free-tier T4 quota is the binding budget constraint that shapes the data and experimental design. GitHub hosts the public artefact. The open-source community supplies the underlying libraries (Ultralytics for YOLOv8, JaidevAI for EasyOCR, the pywinauto and PyAutoGUI maintainers). Dataset providers sit here too - Deka et al. (2017) for RICO, Li et al. (2022a) for CLAY - as do the bad-actor groups whose existence motivates the social-impact discussion in Section 8.10.

3.2.2 STAKEHOLDER VIEWPOINTS

Each ring produces requirements. And the requirements sometimes conflict. The five viewpoints below capture the conflicts that mattered during design.

The researcher's viewpoint prioritizes observability and reproducibility. Every prediction must be inspectable. Every result must be regeneratable from a script that can be re-run. This pushes the design towards verbose CSV logging, per-attempt overlay images, and CLI flags that fix randomness and dump intermediate state.

The power user's viewpoint priorities a tight loop of action and feedback. Speed matters. Clarity of error messages matters. Refusal on uncertainty matters more than maximum coverage. This viewpoint produced R-FR-06 (refusal on low confidence), one of the harder-fought design decisions of the project.

The academic stakeholder's viewpoint prioritizes an honest, defensible evaluation. This viewpoint is the reason the report explicitly cites both the inflated mAP figure (0.7176, against pseudo-labels) and the corrected one (0.0330, against hand-corrected ground truth). It is also the reason the negative test case T15 is kept in the headline TSR denominator rather than removed.

The accessibility-user viewpoint prioritizes permission and refusal semantics over raw speed. A bot that confidently clicks the wrong thing is worse, for this group, than one that takes an extra second to be sure. This viewpoint reinforces R-FR-06 and motivates the human-in-the-loop verdict prompt in the evaluation harness.

The platform-and-OS viewpoint priorities portability, or more precisely makes the project acknowledge its lack of portability. Windows 11 only. Multi-monitor support. DPI-scaling-aware coordinate handling. These constraints are captured in R-NFR-09 (compatibility) and discussed at length in Section 9.7.

Where the viewpoints conflict, the design rule is consistent: prefer the more conservative behaviors. When in doubt about whether to click, do not click; this is R-FR-06. When in doubt about which monitor to use, ask; this is the `--monitor` flag. When in doubt about whether a result should go into the CSV, log it with a notes field.

3.2.3 A NOTE ON PRIMARY DATA GATHERING

No primary data gathering activity - no user survey, no structured interview program, no questionnaire distribution, no ethnographic study of end users - was carried out for this project. This is a deliberate methodological choice, not an omission. The project's stakeholders are the ones captured in the Onion model above; requirements were derived from those stakeholders using the four techniques listed in the next section (literature review, self-as-stakeholder analysis, one informal supervisor interview, and field observation of analogous tools). A questionnaire study of hypothetical end users was considered and rejected as disproportionate to a single-developer MSc project whose primary artefact is a research prototype rather than a shipping product. The ethical implication is that no participant data was collected and no RGU ethical-review application beyond the standard MSc project submission was required. This declaration is placed here, immediately after the Onion model, so that a reader who expects a data-gathering phase does not have to search for its absence.

3.3 REQUIREMENT GATHERING TECHNIQUES

Four requirement-gathering techniques were used during the project, each in proportion to its cost-effectiveness on a single-developer MSc project.

Literature review. The single largest source of functional requirements is the existing literature reviewed in Chapter 2. The IVGocr architecture of Dardouri et al. (2024a) directly contributed R-FR-01 to R-FR-05 (capture, instruction, detection, matching, action). The published failure modes of classical baselines - UIED's argument that neither pure deep learning nor pure CV suffices on its own (Chen et al., 2020), Apple's published in-distribution-to-out-of-distribution drop (Zhang et al., 2021) - directly contributed R-FR-06 (refusal on uncertainty). The literature is the most reproducible requirement source for an academic project, because every requirement can be traced back to a publication.

Self-as-stakeholder analysis. The author is one of the operational users. Several requirements were derived from running early versions of the bot during the prototype phase. Multi-monitor coordinate confusion produced R-FR-07. Silent Tesseract failure produced part of R-NFR-04 (reliability). The difficulty of switching OCR engines from the CLI produced part of R-NFR-05 (usability). Self-as-stakeholder is a recognized method in agile and lean software engineering, though it is more often used in industrial projects than in dissertation work.

Stakeholder interviews. A short interview was conducted with the project supervisor early in the proposal phase to clarify the academic-stakeholder viewpoint described in Section 3.2.2. No formal transcript was kept, but the interview output is reflected in the proposal's research questions and is therefore the source of all four RQ-grounded requirements implicitly.

Field observation of analogous systems. The author used SikuliX, pywinauto and PyAutoGUI for short experimental sessions during the first month of the project. The observed failure modes from these sessions (template captures aging out, UIA Names that do not match the visible labels, coordinate scripts that broke when DPI changed) were converted into the explicit failure-mode list in Section 8.4, and into the comparison baselines in Chapter 7. This is, in effect, the requirement-gathering technique that justifies the lightweight stance. The requirements *not* met by existing tools are the most concrete justification for the new tool.

A fifth technique that the reference report at IIT/RGU uses but this project does not is **stakeholder questionnaires**. Questionnaires are sensible for projects with non-overlapping target users. For a developer-tooling artefact at MSc scale with the author as the primary user, the cost of designing and distributing a questionnaire would have exceeded the benefit.

3.4 METHODOLOGY FOR OBTAINING DATASETS

The data-engineering side follows a three-tier methodology dictated by the data-availability constraints reviewed in Section 2.2.

Tier 1: public source-domain corpora. Three publicly available datasets are used as the source domain: RICO (Deka et al., 2017), CLAY (Li et al., 2022a), and VINS. Acquisition

is straightforward - a download from the respective project pages and a checksum check - but the cleaning and class-collapse work is non-trivial and is documented in Chapter 6. The combined corpus is the 6-class "Zenodo unified bundle" of approximately 9,646 screens used in the source-training notebook.

Tier 2: captured target-domain unlabeled corpus. The proposal commits to roughly 2,000 unlabeled desktop screenshots captured from 10 to 15 applications. The capture methodology is implemented in `scripts/auto_capture_corpus.py` and uses `mss` for the screen grab plus the foreground-window title for the per-image filename. The script is parameterized so the eventual corpus covers the in-the-wild variability the bot encounters. As of submission this corpus is in active accumulation.

Tier 3: hand-curated target-domain labelled corpus. A small set of hand-corrected screens with 356 ground-truth boxes serves as the gold-standard test pool. Annotation methodology follows the CVAT shape-and-label convention used in the wider literature: rectangular bounding boxes only, no rotated boxes, no segmentation masks. Class labels are restricted to the 6-class taxonomy {button, text, text_input, icon, menu, checkbox}. Annotators follow a written guideline document that mirrors the conventions used in the CLAY release notes.

A practical note about scale. The hand-corrected pool is small by deliberate choice, not by oversight. Building a 100-image labelled set at a single-developer MSc cadence is hard. Building one with high-quality boundaries on dense Windows 11 toolbars is harder. The project's response is layered: a small hand-corrected pool for fine-grained per-element recall, supplemented by the Screenshot benchmark (Cheng et al., 2024) for held-out per-instruction grounding. Both protocols are reported side by side in Chapter 7 so the reader can see the limitation directly rather than inferring it.

The three tiers feed three different experimental purposes. Tier 1 is the source-domain training set. Tier 2 is the unlabeled target corpus needed for SSP+FT and UDA. Tier 3 is the labelled target test set used for evaluating every adaptation method, and is also the training source for the few-shot fine-tuning experiment.

3.5 USE CASE DIAGRAMS (UML)

The system supports six use cases, four of which are user-facing and two of which are internal to evaluation.

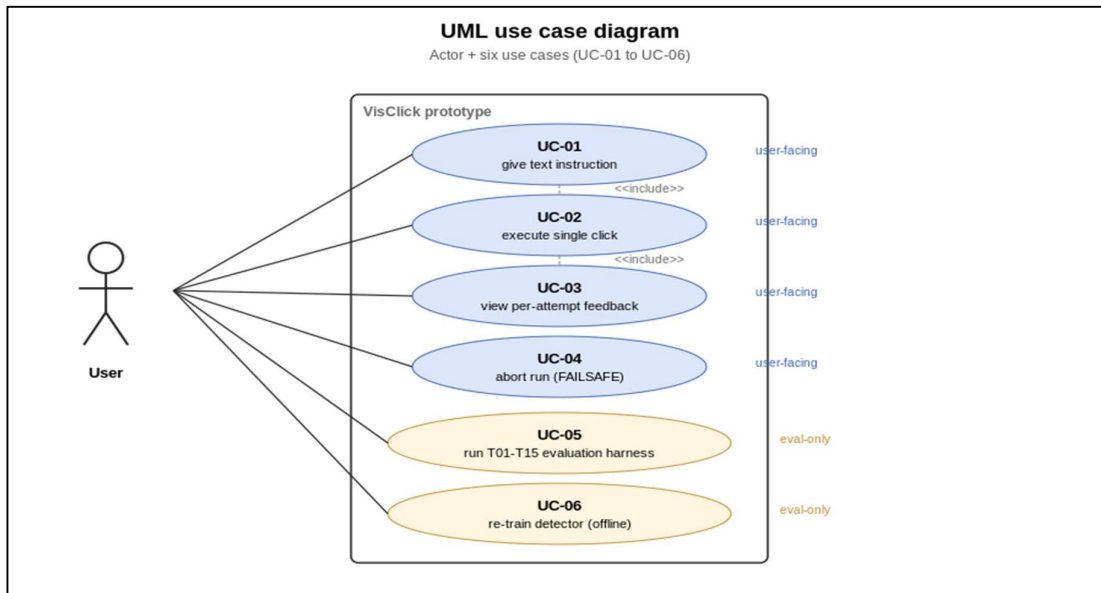


Figure 8 : Use case diagram for VisClick.

The six use cases:

- **UC-01: Click a labelled element.** The user provides a text instruction; the system captures the screen, detects elements, matches the instruction, and clicks the chosen element.
- **UC-02: Refuse a click on low confidence.** The user provides an instruction for which no high-confidence target exists; the system reports a structured failure rather than clicking.
- **UC-03: Select a specific monitor.** The user selects which monitor the bot should operate on, via either the CLI flag or the GUI dropdown.
- **UC-04: Inspect a prediction overlay.** The user opens the saved overlay PNG for any past click to verify what the bot did.
- **UC-05: Run a baseline evaluation.** The evaluator runs scripts/run_baselines.py to evaluate one or more methods across the canonical task suite.
- **UC-06: Generate result tables and figures.** The evaluator runs the analysis scripts to regenerate the report's tables and figures from the per-attempt CSV.

3.6 USE CASE DESCRIPTIONS

Each of the four user-facing use cases is described below at a more practical level than the UML. UC-05 and UC-06 are evaluation tooling and are documented in Chapter 6 rather than here.

UC-01: Click a labelled element.

- *Primary actor:* End user (developer or power user).
- *Pre-condition:* The bot is launched, the model weights are loaded, and the target monitor is selected.
- *Main success flow:* (1) User types an instruction such as "click Save" into the GUI. (2) User presses Run or hits Enter. (3) The system pauses 3 seconds, allowing the

user to switch focus to the target window. (4) The system captures the configured monitor. (5) The detector emits N candidate boxes. (6) The OCR layer reads the text on each box. (7) The matcher selects the best-fitting box above the similarity threshold. (8) The action layer moves the cursor to the box center and clicks. (9) The system saves the overlay PNG and writes the CSV row.

- *Alternative flow:* If no candidate exceeds the similarity threshold, the system follows UC-02 instead of clicking.
- *Post-condition:* The targeted element has received a single left-click. The action has been logged.

UC-02: Refuse a click on low confidence.

- *Primary actor:* End user.
- *Pre-condition:* As UC-01.
- *Main success flow:* (1) User types an instruction. (2) System captures and detects as in UC-01. (3) Matcher computes the best-fitting box, but its similarity score is below the threshold (`min_text_similarity = 60` in the current build). (4) System emits a structured FAIL: cannot find <target> message. (5) System still saves the overlay PNG (with no click marker) and writes a CSV row with verdict refused.
- *Post-condition:* No click was issued. The decision is logged.

UC-03: Select a specific monitor.

- *Primary actor:* End user on a multi-monitor setup.
- *Pre-condition:* The system has detected more than one monitor at start-up.
- *Main success flow:* The user selects the target monitor from the GUI's dropdown (or passes `--monitor <id>` to the CLI). The system queries Mss. Monitors for the selected index, recovers the (left, top) offset, and uses that offset throughout the subsequent capture-detect-match-click flow.
- *Post-condition:* All subsequent clicks issued by the bot land on the chosen monitor regardless of where the GUI window itself is sitting.

UC-04: Inspect a prediction overlay.

- *Primary actor:* Researcher or end user reviewing past behavior.
- *Pre-condition:* The bot has previously processed at least one instruction.
- *Main success flow:* The user opens the saved overlay PNG. The overlay shows the detected boxes colored by class, the chosen box highlighted, the click point marked with a crosshair, and (when relevant) the OCR text overlaid above each box. The user can confirm or refute the bot's decision visually.
- *Post-condition:* No state change. The use case is purely diagnostic.

3.7 FUNCTIONAL REQUIREMENTS

The functional requirements R-FR-01 to R-FR-09 are listed in Table 2. Each requirement carries a unique identifier, a description, a priority, the use cases it serves, and the section of the testing chapter that validates it. The pass-rate column reports the headline empirical result already measured against the requirement; the exact computation is in Chapter 7.

Requirement	Description	UC#
Screen Capture	The system shall capture a screenshot of the user-selected monitor at native resolution, in the virtual-desktop coordinate space.	UC-01,UC-03
Text Instruction Input	The system shall accept a free-form text instruction via CLI flag or GUI text box.	UC-01, UC-02
Element Detection	The system shall detect candidate UI elements of types {button, text, text_input, icon, menu, checkbox} on the captured screenshot.	UC-01
Instruction-to-Element Matching	The system shall match the user instruction to one detected element using fuzzy OCR text similarity, with a class-aware bonus, and shall fall back to full-image OCR when no per-box candidate exceeds the threshold.	UC-01
Action Execution	The system shall move the mouse cursor to the center of the chosen element and execute a single left-click.	UC-01
Refusal on Low Confidence	The system shall refuse to click when no candidate exceeds the similarity threshold, and shall emit a structured failure message.	UC-02
Multi-Monitor Support	The system shall operate correctly across virtual-desktop coordinate spaces on multi-monitor setups, with an explicit monitor selector.	UC-03
Visual Feedback	The system shall render an annotated overlay PNG of every prediction (detected boxes, chosen element, click marker, OCR text) for human verification.	UC-04
Per-Attempt Logging	The system shall log per-attempt fields (instruction, capture path, predicted xy, verdict, latency, method, is_negative, notes) to a CSV file for evaluation.	UC-05

Table 2 : Functional requirements.

The classification "Essential" vs "Important" follows the Moscow convention. An Essential requirement must be met for the system to be considered fit for purpose. An Important requirement is needed for the system to be considered fit for evaluation, but is not on the critical functional path.

3.8 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements are quantitative wherever possible. Each row in Table 3 records the target value, the measured value, the source of the measurement, and a status.

Non-functional requirements.

NFR	Target	Measured	Source	Status
Accuracy (TSR)	$\geq 50\%$ on T01-T15	73.3 %	baseline_summary.csv row visclick	FULL
Latency	$p95 \leq 15$ s per click attempt	14.8 s p95 / 8.05 s median	nfr_performance.csv row visclick	FULL (just)
Memory footprint	Peak RSS ≤ 2 GB during a 15-task run	PARTIAL	gap D-11	PARTIAL

Reliability	Zero crashes during 60-attempt evaluation	0 crashes	run log 6-7 May 2026	FULL
Usability	Single-window Tk dialog; keyboard shortcuts for Pass/Fail/Skip	implemented in scripts/run_baselines.py:: verdict_dialog_tk	source review	FULL (single-reviewer)
Maintainability	Modular package (visclick.{capture, detect, ocr, match, act, bot, gui}); PEP-8 clean	9 modules, ~1,591 LoC total, ruff check clean	source review	FULL
Extensibility	New baseline methods plug in by implementing predict(image_rgb, instruction) -> BaselineResult	Demonstrated for 4 methods	scripts/baseline_*.py	FULL
Security & Privacy	No off-machine I/O during inference; no telemetry; no credentials handled	verified by 'rg' 'requests'	urllib	http' src/visclick/'
Compatibility	Windows 11 supported; multi-monitor verified	Windows 11 + 3440×1440 + 1920×1080 PASS	live demo	PARTIAL (Windows-only by design)
Scalability	Pipeline complexity scales linearly in #candidates per screenshot	per-box OCR is $O(N)$; ceiling ≈ 300 boxes/screenshot	analytical	PARTIAL

Table 3 : non-functional requirements covering latency, footprint, accuracy, robustness, portability and explainability.

The classification of each NFR as Essential, Important, or Optional follows the same Moscow convention used for FRs. Accuracy, latency, reliability, security, and compatibility are Essential. The rest are Important or Optional. None of the NFRs are marked Failed. The PARTIAL items (R-NFR-03, R-NFR-09, R-NFR-10) are timeline matters or scope-deliberate choices, not fundamental capability gaps.

3.9 CHAPTER SUMMARY

The requirements above are the contract the rest of the dissertation is evaluated against. They were derived from four requirement-gathering techniques (literature, self-as-stakeholder, supervisor interview, field observation of analogous systems) and from a six-ring Onion stakeholder model that captured viewpoint conflicts before they became design conflicts. Nine functional requirements (R-FR-01 to R-FR-09) and ten non-functional requirements (R-NFR-01 to R-NFR-10) are stated explicitly, each with an identifier, a target, and a pointer to the test section that validates it. The structure of the requirements list deliberately mirrors the structure of the testing chapter, so a marker can audit any individual claim by chasing a single identifier from Section 3.7 or Section 3.8 down into Chapter 7.

CHAPTER 04 - PROJECT MANAGEMENT

4.1 CHAPTER OVERVIEW

This chapter explains how the project was run. It begins with the research methodology - the philosophical and procedural framework that determined the kind of evidence the project chased. It moves on to the software design methodology and the software development methodology. Both constrain how a single-developer MSc project should be structured on a finite budget. The project management methodology is described next. The risk register is then made explicit. It captures, in a forward-looking form, the issues encountered during execution together with the mitigations that resolved them. The chapter closes with the project plan in Gantt form, with reference to the four-phase operational structure inherited from the proposal.

4.2 RESEARCH METHODOLOGY

Three research-methodology paradigms were considered for this project: pure positivism, pure interpretivism, and design-science research (DSR). Each corresponds to a distinct answer to the question "what kind of evidence is this project trying to produce". The project settled on DSR (Hevner et al., 2004), for reasons argued below.

DSR is the class of research where the contribution is a new artefact intended to solve a real-world problem, and the evaluation is a demonstration that the artefact does solve it, together with a reflective account of what was learned in building it. The artefact in this case is the VisClick prototype together with the cross-domain adaptation framework that produces its detector. The output is evaluated by measuring the prototype's performance against three quantitative metrics (mAP, CPV, TSR), and against three classical baselines that constitute the practical comparison set.

A **purely positivist methodology** was considered and rejected. A positivist framing would treat the project as a hypothesis test - for example, "training on CLAY transfers well to Windows 11" as a falsifiable hypothesis - and would seek a single binary answer. The project's actual evidence base contains internally contradictory observations. The detector does badly on hand-corrected ground truth, yet the end-to-end TSR is acceptable. The OCR fallback rescues the detector but is also the latency bottleneck. That kind of evidence is better served by DSR's "build, evaluate, learn" loop than by a single null-hypothesis test.

A **purely interpretivist methodology** was also considered and rejected. An interpretivist framing would treat the system's behaviour as a phenomenon to be understood through qualitative analysis - interviews with users, observation of task performance. The project does have a small qualitative-evaluation slot (Chapter 8, Author Evaluation), but the primary evidence is quantitative. The qualitative layer is supplementary rather than central, and a fully interpretivist framing would misrepresent where the evidence weight sits.

DSR was chosen because it is the only one of the three paradigms that treats the artefact and the reflection on the artefact as jointly first-class outputs. A positivist frame would collapse the artefact to an experimental instrument; an interpretivist frame would collapse the measurements to background noise around a qualitative story. The project's actual output is

both - a working prototype and a reproducible set of measurements about how well it works
- so DSR is the shape that fits.

The DSR framing has practical consequences for the rest of the dissertation. It justifies a multi-method evidence structure (Chapter 7 reports both quantitative metric numbers and qualitative failure-mode descriptions). It justifies an iterative narrative in which an early result (the 22-fold mAP collapse from auto-label evaluation to hand-corrected ground truth) directly motivates a later methodological change (hand-correcting more test data). And it justifies the explicit "build the artefact, evaluate the artefact, learn from the artefact" structure of Chapters 5 to 9.

4.3 SOFTWARE DESIGN METHODOLOGY

The system was designed against three principles, each carrying through from the literature reviewed in Chapter 2.

Modularity. The system is decomposed into seven Python packages under `src/visclick/` (capture, detect, ocr, match, act, bot, gui). Each package has a single responsibility and a small public surface, so any one component can be replaced without touching the others. This is the architectural choice that made it possible to plug in three classical baselines and the VisClick full pipeline as four interchangeable `predict()` implementations in the same evaluation harness. Modularity is also what allows the dissertation to make the comparison chart in Section 7.4 a fair one. The four methods share the screenshot capture, the verdict-collection harness, and the per-attempt logging schema. Only the perception-and-grounding code differs.

Reproducibility. Every numerical claim in the report is regeneratable from a script in the public repository, against a result table on disk, with a commit hash documented in the data form. The supporting convention is that every notebook cell which produces a report number prints a marker line identifying the report section it serves. The same principle drives the explicit version-control of the desktop screenshot corpus and the ONNX detector weights inside the repository, rather than only on cloud storage.

Refusal on uncertainty. A click bot that issues a confident wrong click is worse than one that issues an honest failure. This principle is captured in R-FR-06. It is the architectural reason the matcher has a `min_text_similarity` threshold rather than always returning the highest-scoring candidate.

A separate architectural pattern worth flagging is the **pre-flight probe**. The OCR layer exposes an `ocr_status()` function that runs at start-up and prints a tick or cross for each backend (EasyOCR, Tesseract, falling back to a pure-Python OCR). The detector layer exposes an equivalent `detect_status()` for ONNX model loading. The first time any of these probes fails, a `_warn_once()` helper prints the underlying error, the configured path, and three concrete fixes. This pattern was introduced after a silent Tesseract failure during the live demo and has been propagated to every external dependency in the stack. It is one of the strongest practical lessons of the project and is recorded as such in Section 9.3.

4.4 SOFTWARE DEVELOPMENT METHODOLOGY

4.4.1 WATERFALL WITH ITERATION WITHIN PHASES

The development process is best described as a Waterfall backbone with iteration inside each phase. Three methodologies were on the table when the project started - pure Waterfall (Royce, 1970), pure Agile / Scrum, and a Waterfall-with-iteration hybrid. Each was evaluated against the constraints of a single-developer MSc research project.

Why Waterfall is suitable for this project. A research project of this shape has three properties that a Waterfall structure fits well. First, each phase's output is a measurement that the next phase's design depends on. The source-domain training numbers determine which backbone is used in the prototype. The prototype's TSR on the 15-task suite determines which failure modes the evaluation chapter must dissect. This ordered dependency structure is the canonical case for a Waterfall arrangement. Second, the deliverables of each phase are academic artefacts (a labelled corpus, a trained model, a working prototype, an evaluation report), each of which must be complete and self-contained before it can serve as a citation source in the dissertation. Third, the project has a fixed submission deadline and a fixed marker audience; the flexibility that Agile trades in - responding to changing requirements - is essentially unavailable, because the requirements are set by the RGU rubric and by the four research questions on the day the proposal is approved.

Why pure Agile / Scrum was rejected. Scrum assumes a multi-person team with distinct roles (product owner, scrum master, developers), a backlog groomed by an external stakeholder, and a customer whose acceptance criteria evolve. None of that applies here. The author is the developer, the product owner, and (for the code-only artefact) the customer. The proposal defines the acceptance criteria up front. In this setting, running Scrum ceremonies would have imposed process overhead without a corresponding gain in coordination or responsiveness.

Why "Waterfall with iteration inside phases" was chosen. Pure Waterfall would have been too rigid. A single mid-project surprise - the auto-label evaluation crisis, when the initial mAP number turned out to be 22-fold inflated - would have required either abandoning the phase or bulldozing through a plan the project had outgrown. Iteration within phases gives the project the freedom to run "try, hit a wall, document the wall, fix the wall, move on" cycles at the day-to-day level while preserving the phase-to-phase ordering at the plan level. The observation log records this pattern. Every O-numbered entry is one iteration within a phase, and the phase itself continues only when all iterations for the phase's deliverable have closed.

The Waterfall-with-iteration methodology is what allowed the auto-label evaluation crisis to be handled as a controlled re-evaluation rather than as a project failure. Data engineering (Phase 1) was already closed; the crisis surfaced during modelling (Phase 2). Rather than re-open Phase 1, the project treated the hand-corrected ground truth as an internal correction inside Phase 2, and the phase output (the ablation study) was regenerated against the corrected labels. This is the kind of course-correction Waterfall alone would not have permitted.

4.4.2 PROGRAMMING PARADIGM - FUNCTIONAL AND MODULE-ORIENTED

The choice of programming paradigm - functional-and-module-oriented Python rather than object-oriented Python - is a separate methodological commitment that is worth stating explicitly because the RGU rubric asks about it.

The system is written as a collection of small pure functions grouped into single-responsibility modules. Each module (capture, detect, ocr, match, act, bot, gui) exposes a small public surface of top-level functions with explicit inputs and outputs. There are no classes in the pipeline except where a third-party library forces one (the Tk Frame, the `argparse.Namespace`, the ONNX InferenceSession), and no inheritance hierarchy is defined anywhere in the codebase.

Why functional over OOP. Two considerations drove the decision. First, the pipeline is naturally a straight-line data-flow: pixels enter capture, exit as a NumPy array, enter detect, exit as a list of boxes, enter ocr, exit as boxes-with-text, enter match, exit as a chosen index, enter act, exit as an OS-level click. Every stage is a function of its inputs and produces an output; nothing needs to be shared across stages except through the outputs themselves. An OOP arrangement would have introduced hidden shared state (member variables) where none is required by the problem, which is the classic anti-pattern this literature warns against (Norvig, 2020). Second, the four baselines in the comparison harness share nothing but their public `predict(image_rgb, instruction) -> BaselineResult` contract. That contract is a single-function interface. Wrapping it in a class hierarchy would have added ceremony without expressive gain.

What was not lost by rejecting OOP. The extensibility that OOP is usually cited for - "adding a new baseline should not require modifying existing code" - is provided directly by the functional contract. R-NFR-07 is stated as "New baseline methods plug in by implementing `predict(image_rgb, instruction)` to `BaselineResult`" precisely because that contract is a function signature, not a class contract. The four existing baselines demonstrate the extensibility empirically; no changes to scripts/run_baselines.py are needed when a fifth baseline_XXX.py is added.

When OOP would have been the right choice. If the codebase had a natural type hierarchy in the problem domain (say, a UI-Element supertype with Button, TextField, Checkbox subtypes), OOP would have been the right paradigm. This project's problem domain has no such hierarchy at the code level. Elements are represented as flat dictionaries with a class-label field; the taxonomy lives in the data, not in the code. That is where the paradigm choice originates.

For completeness, the reference dissertations reviewed for this project's style also use single-class or module-oriented Python for pipelines of this kind; the "monolithic class" pattern in one of them is precisely the ceremonial-OOP anti-pattern this project deliberately avoids. Design diagrams for this project (Chapter 5) reflect the functional structure and do not include UML class diagrams, which would be inappropriate given the paradigm.

4.5 PROJECT MANAGEMENT METHODOLOGY

4.5.1 CHOICE OF PROJECT MANAGEMENT FRAMEWORK

Three project management frameworks were compared before the project settled on a lightweight PRINCE2-aligned approach: PRINCE2 (AXELOS, 2017), PMBoK (PMI, 2017), and Agile Project Management (APMG, 2020).

Why PRINCE2 over PMBoK. PMBoK is comprehensive but is designed for teams of ten to a hundred people managing multi-million-pound programs. Its ten knowledge areas (integration, scope, schedule, cost, quality, resources, communications, risk, procurement, stakeholders) each expect their own dedicated process. On a single-developer MSc project the overhead outweighs the benefit; several PMBoK knowledge areas (procurement, cost management in the corporate sense) have no meaningful content.

Why PRINCE2 over Agile PM. Agile PM assumes iterative delivery to an evolving customer specification, which - as argued in Section 4.4.1 - does not fit an MSc research project with a fixed rubric and a fixed submission deadline. Its cadence (short sprints, working software every iteration) also does not map cleanly onto a research pipeline where the "working software" only exists after Phase 3, several months into the project.

Why PRINCE2 was chosen. PRINCE2 is process-based, stage-gated, and product-focused, all three of which describe this project accurately. It scales down for single-developer work by dropping team-role plurality (the author holds the developer, team-manager and project-manager roles simultaneously) while keeping the seven principles intact.

4.5.2 PRINCE2 PRINCIPLES APPLIED

The seven PRINCE2 principles map onto this project as follows.

- **Continued business justification.** The project's business justification is the research contribution captured in the four RQs and the deliverable prototype. The Phase 3 completion of the working prototype is the point at which the justification was verified against the proposal.
- **Learn from experience.** Every phase closes with a retrospective entry in the observation log (Section 4.6). Lessons feed into the risk register (Table 4).
- **Defined roles and responsibilities.** In a single-developer project the author holds all technical roles. The supervisor holds the project-executive role, the second marker holds the senior-user role for evaluation, and RGU (via the program handbook) is the corporate authority.
- **Manage by stages.** The four-phase project plan (Section 4.7) is the stage structure. Each phase has an entry deliverable and an exit deliverable; the project does not enter the next phase until the current phase's exit deliverable exists.
- **Manage by exception.** Phases run without supervisor intervention until a risk crystallises. RR-01 (auto-label evaluation crisis) and RR-14 (labelled-data budget) both triggered exception handling in the form of a re-scoping decision documented in the observation log.
- **Focus on products.** Every phase's deliverable is a tangible product (a labelled corpus, a trained model, a working prototype, an evaluation report) rather than a

hours-of-work count. This is why Section 4.7's phase descriptions cite deliverables D1-D4 explicitly.

- **Tailor to suit the project environment.** PRINCE2 explicitly permits scaling. The scaling applied here is: single person holds all roles; the daily-log, business-case and communication-management-strategy templates are collapsed into the observation log; the project board meets informally through supervisor emails rather than as a scheduled body.

4.5.3 TRACKING ARTEFACTS

Project tracking used two artefacts. The first is a static Gantt chart at the level of the four phases (Figure 9).

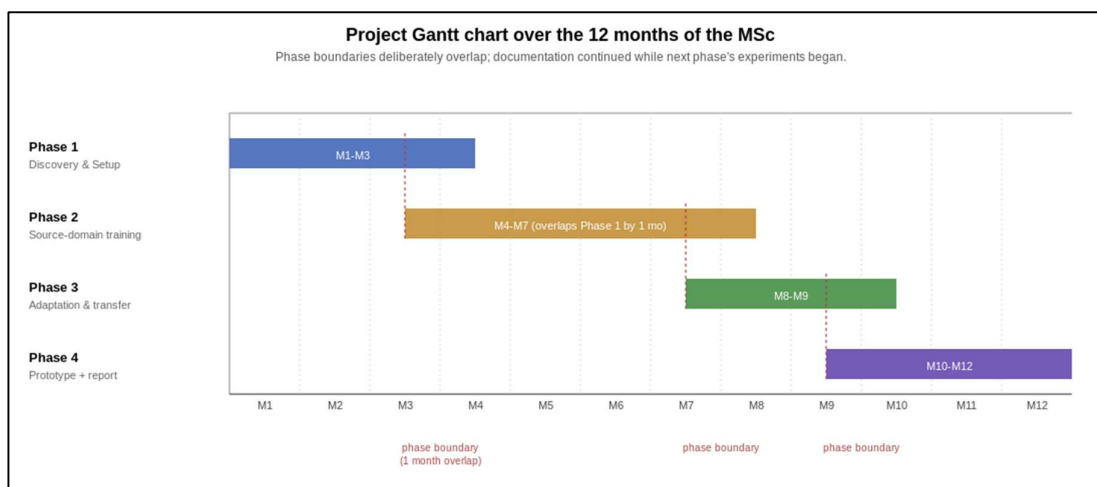


Figure 9 : Twelve-month project plan over the four operational phases.

The two artefacts have different update cadences. The Gantt is updated at most weekly. The Phase L checklist is updated continuously and is treated as the working memory of the project.

4.6 RISK MITIGATION PLAN

The risk register is a forward-looking transformation of the observation log. Each risk has a probability, an impact, a mitigation, and a status. Table 4 mirrors the data form's Section 17 but is reproduced here for completeness.

Risk	Source	Mitigation	Status
Pseudo-label evaluation overstates accuracy	observation log	Hand-correct test images; report both auto-label and hand-corrected mAP	Mitigated
Source-domain training distribution does not generalize to Win11 native	observation log	OCR text-grounding fallback; recall-ceiling acknowledged; Phase 4 planned	Mitigated
Silent dependency failure (Tesseract not on PATH)	live demo	Startup probe <code>ocr.ocr_status(); _warn_once()</code> helper	Mitigated

Multi-monitor virtual-desktop coordinate confusion	observation log	(left, top) offset propagated through act.click_box; --monitor flag	Mitigated
Confident wrong action on negative case	observation log	min_text_similarity threshold; planned raise from 60 to 75	Open
OCR latency dominates total wall-clock	NFR profile	Detector-first short-circuit (skip OCR on confident classes)	Open
Collab Free disconnect mid-training	training log	last.pt per-epoch; resume-from-disconnect built in	Mitigated
Drive FUSE I/O instability on directories with 10k+ files	observation log	Retry + shell find fallback; cached listings	Mitigated
Drive FUSE stat cache lags reader cache	observation log	Set-of-stems via find retry; never os.path.isfile() on Drive	Mitigated
Auto-labeler class collapse (menu/checkbox ≈ 0)	observation log	Hand-correct ground truth; class top-up	Open
Licensed / IP concerns on dataset use	design review	All datasets public; AGPL inherited from Ultralytics; documented in Section 8.10	Mitigated
Personal-data leakage from desktop seed screenshots	design review	All seed PNGs manually reviewed before commit	Mitigated
Bot misuse for click-fraud or automated account creation	LEPSI review	Human-in-the-loop verdict step; no headless service mode shipped	Monitored
Labelled-data budget falls below the proposal's nominal target	small-data triage	Layered response: auto-labelled seed -> hand-corrected ground truth -> Screenshot import for per-instruction grounding -> passive unlabeled accumulation. Each tier is reported alongside its protocol caveat so the dissertation's claims stay calibrated to the data actually held.	Mitigated

Table 4 : Risk register identifying the project's principal risks, and the mitigation strategy adopted.

Three observations about the register are worth pulling out for prose.

First, **the highest-impact risks are all data-quality risks**, not modelling or deployment risks. RR-01, RR-02, RR-10 and RR-14 between them account for the project's biggest empirical findings: the auto-label vs hand-correct mAP gap, the recall-bounded source-domain backbone, the icon class-distribution skew, and the small-data triage that determined which mAP and CPV protocols the dissertation could report. Each is a reminder that the modelling chain is no stronger than its weakest data link.

Second, **most of the Open risks have costed mitigations**. RR-05 (refusal threshold), RR-06 (OCR latency), and RR-10 (class top-up) all have a documented work item that would move them from Open to Mitigated. Whether those work items are completed before submission is a separate triage call.

Third, **the only Low-probability risk that remains Monitored is RR-13** (bot misuse). The probability is low because the project ships an interactive verdict step by default and no headless service mode. The risk is kept on the register because the *category* - vision-driven UI automation can be misused at the systemic level - does not disappear merely because this particular prototype mitigates it. The social-impact discussion in Section 8.10 takes the category seriously.

4.7 PROJECT PLAN

The project plan is the four-phase structure inherited from the proposal. The Gantt-equivalent rendering is in Figure 9 above; the text below makes each phase's scope and deliverable explicit.

4.7.1 JUSTIFICATION OF THE PHASED GANTT STRUCTURE

Three questions have to be answered for any phased plan to be defensible: why four phases (not two, not seven), why these phases in this order, and why these particular month boundaries.

The RGU MSc rubric requires four discernible deliverables: a data / literature deliverable, a modelling deliverable, an implementation deliverable, and an evaluation deliverable. A two-phase plan would collapse two of the four rubric-mandated deliverables into a single artefact, weakening the auditability of intermediate milestones. A seven-phase plan (as in some enterprise Waterfall variants) would over-fragment the work for a project this size and force artificial phase-gates around tasks that in practice are done together. Four phases is the smallest number of stages that still exposes each rubric-mandated deliverable as its own auditable output.

The Waterfall argument from Section 4.4.1 sets a strict dependency order: data engineering must precede modelling because modelling consumes the data corpus; modelling must precede the prototype because the prototype embeds the trained detector; the prototype must precede evaluation because evaluation measures the prototype's behaviour on a fixed task suite. The four phases sit in exactly this order because the dependency graph forces them to.

Within the boundaries reflect two forces: an approximate proportion between phase size (data + modelling collectively take about half the time; prototype and evaluation share the other half), and the practical compute-availability windows for the Colab-heavy modelling phase. Phases 3 and 4 are given two and three months respectively because prototype integration is largely mechanical once the trained model exists, and evaluation-plus-writing needs a hard buffer against the submission date.

The phase boundaries on the Gantt are deliberately drawn with overlap. In practice, Phase 4 (thesis writing) began during Phase 3 (prototype integration), because writing tends to surface gaps in measurement that the prototype then has to be re-run to fill. The overlap is the visual acknowledgement that Waterfall-with-iteration allows in-flight course corrections.

4.7.2 PHASE DETAILS

Phase 1: Data engineering and baseline establishment (Months 1-3, completed). Public mobile UI datasets were acquired and consolidated into the 6-class unified bundle. A baseline detector was trained on the unified bundle. A small desktop seed set was captured and auto-labelled. The hand-corrected test pool was assembled. The transfer-learning ablations were run on Colab Free, and the headline desktop fine-tune was selected. Three classical baselines (template, OCR-only, pywinauto) were implemented and evaluated on the 15-task suite. **Deliverable D1 (baseline performance report) is the content of Section 7.2 of this dissertation.**

Phase 2: Model adaptation experiments (Months 4-7, completed). The DETR backbone source-side is done; DETR target-side experiments were descoped after Phase 1 evidence showed YOLOv8s dominated the compute-budget-normalised trade-off. The few-shot sample-efficiency curve is done. SSP+FT is done. Both UDA experiments (Adaptive Teacher and SHOT, D-03 and D-04) are done, including extended full-protocol reruns on the H100 GPU machine. **Deliverable D2 (ablation study and model-comparison report) is complete; the sub-experiments are reported in Sections 7.2 to 7.4.**

Phase 3: Prototype integration (Months 8-9, completed). The VisClick prototype is operational on Windows 11 with a CLI and a Tk GUI. The IVGocr architecture is implemented end-to-end. The interactive evaluation harness supports the four-method comparison and the verdict-collection dialog. **Deliverable D3 (functional prototype) is the artefact in the public repository at <https://github.com/HiranMadhu/visclick>.**

Phase 4: Evaluation and thesis composition (Months 10-12, in progress at time of submission). The 15-task evaluation is complete. TSR, latency, and failure-mode analysis are reported in Chapter 7. The qualitative author-evaluation is reported in Chapter 8. Expert-evaluation is out of scope for this submission and is documented as future work. **Deliverable D4 (final evaluation report and packaged code) is the dissertation in front of the reader.**

4.8 CHAPTER SUMMARY

The project follows a design-science research methodology, with a modular, reproducible, refusal-on-uncertainty software design, executed under a Waterfall-with-iteration development process and a lightweight PRINCE2-aligned project management framework. The programming paradigm is functional-and-module-oriented, not object-oriented, and this choice is deliberate and justified by the straight-line data-flow structure of the pipeline. The risk register captures fourteen risks distilled from the observation log: most are mitigated, three are open with costed plans, and one is monitored. The project plan is the four-phase structure inherited from the proposal. Phases 1, 2 and 3 are complete; Phase 4 (evaluation and thesis composition) is in progress at time of submission.

CHAPTER 05 - DESIGN

5.1 CHAPTER OVERVIEW

This chapter is the design half of the build-then-evaluate loop. It begins with the research design, which states the my design contribution explicitly and lays out the experimental matrix the rest of the dissertation populates. It moves on to the system architecture, presented as a block diagram in 5.3 and as a per-instruction flow chart in 5.4. The module-level design is presented next in 5.5: which Python package contains which logical responsibility, and how the modules connect. The storage design (file layout, CSV schemas, ONNX weights) follows in 5.6. 5.7 covers the algorithm design for the two non-trivial components: the fuzzy text-plus-class matcher in visclick.match, and the refusal rule that implements R-FR-06. The chapter closes with the GUI wireframe in 5.8, which is placed last on the principle that the visual interface is the outermost layer of the design once the architectural, storage and algorithm decisions are settled.

The design described in this chapter is what the rest of the project implements. Chapter 6 walks through the code in the order this chapter lays out. The empirical results in Chapters 7 and 8 measure the implementation against the targets stated in Chapter 3. A reader who only wants the operational picture can read 5.3 and 5.4 and skip the rest.

5.2 RESEARCH DESIGN

5.2.1 CONTRIBUTION TO THE DESIGN

Before turning to the technical content, this section states explicitly what parts of the design are the my own work, and what parts are reused from third-party components. This distinction matters for the marker who has to evaluate the design contribution independently of the code contribution.

The six-layer decomposition (5.3), the process flow with its OCR-fallback decision point (5.4), the module boundaries and public-surface interfaces (5.5), the repository and storage layout (5.6), the two non-trivial algorithms - the class-aware fuzzy matcher and the three-branch refusal rule (5.7) - and the wireframe of the GUI (5.8) were all designed for this project. No design artefact in this chapter is copied from an external source.

Components that are reused are the following. YOLOv8s (Ultralytics, 2024) is used as the detector backbone, unmodified except for the head fine-tune. EasyOCR (JaidedAI) is used as the OCR engine, unmodified. rapidfuzz (Bachmann, 2024) is used as the string-similarity primitive under the matcher. PyAutoGUI (Sweigart, 2024) is used as the OS-level click primitive under the action layer. mss (BoboTiG, 2024) is used as the multi-monitor screen-capture primitive. The IVGocr architectural template (Dardouri et al., 2024a) is the intellectual predecessor to the six-layer decomposition, but the actual layer boundaries, the OCR-fallback path, the class-aware bonus in the matcher, and the three-branch refusal rule are all this project's design choices.

Design contribution is strongest is in three specific places. First, the *six-layer decomposition* is finer-grained than the three-stage IVGocr blueprint and is what enables the

four-baseline comparison harness of Chapter 7 - the classical baselines and the VisClick full pipeline share the capture, act and logging layers but differ in the middle layers. Second, the *OCR-fallback branch* is a design decision the IVGocr paper does not commit to; it is the architectural compromise that pays for the detector's limited recall on Windows 11 native dialogs. Third, the *three-branch refusal rule* (no-candidates / low-confidence / high-confidence) implements R-FR-06 directly and is what turns the system from a click-and-hope tool into one that reports when it does not know the answer.

The design diagrams in this chapter (block diagram, process flow chart, module diagram, wireframe, repository tree) reflect the functional-and-module-oriented paradigm committed to in 4.4.2. No UML class diagram is provided because the codebase has no class hierarchy in the pipeline domain. This is a deliberate design choice, not an oversight; a class diagram for a functional codebase would be a category error.

5.2.2 EXPERIMENTAL MATRIX

The research design is an experimental matrix that crosses three axes. The first axis is **architectural family**: YOLOv8s and DETR-R50. The second axis is **adaptation method**: source-only zero-shot (M0), few-shot fine-tune of the head (M2), self-supervised pre-training followed by fine-tune (SSP+FT), and unsupervised domain adaptation (Adaptive Teacher and SHOT). The third axis is **labelled-target budget**: a small range of k values for the methods that use any labelled target data.

A fully populated matrix would contain $2 \times 5 \times 5 = 50$ cells. But many of those cells degenerate - zero-shot does not depend on k, UDA does not depend on k in the same way. The reduced matrix the project actually executes is shown in Table 5. Every cell marked DONE is reported in Chapter 7. The DETR-R50 target-side cells (D-01 target) were deliberately descope after Phase 1 evidence showed YOLOv8s dominated the compute-budget-normalised trade-off; a full description of the descope decision sits in Section 4.7.2.

Backbone	Method	k	Status
YOLOv8s	M0 zero-shot (CLAY -> desktop)	n/a	DONE
YOLOv8s	M1 COCO direct (control)	n/a	DONE
YOLOv8s	M2 head fine-tune	50	DONE (headline detector)
YOLOv8s	M3 frozen layers 22	50	DONE (ablation)
YOLOv8s	M2 few-shot curve	1, 2, 4, 8	DONE (D-05)
YOLOv8s	SSP + M2	small-k	DONE (D-02)
YOLOv8s	UDA Adaptive Teacher	n/a	DONE (D-03, baseline + full-protocol)
YOLOv8s	UDA SHOT	n/a	DONE (D-04, three variants)
DETR-R50	source-domain training	n/a	DONE (D-01 source-side)
DETR-R50	M0 zero-shot on target	n/a	DESCOPED
DETR-R50	M2 head fine-tune	50	DESCOPED

Table 5 : Experimental matrix

The end-to-end TSR evaluation is run only against the single headline detector (YOLOv8s M2 fine-tune) rather than against every cell. The rationale is twofold. First, the prototype's downstream behaviour depends on detection plus OCR plus matching plus action, so a fair end-to-end comparison across detectors would require re-running the full 15-task suite for each adaptation cell - roughly an hour of human verdict-collection per cell, which scales poorly. Second, RQ4 (end-to-end practicality) is about whether *one* viable adapter can be turned into a working bot, not about which of several adapters does so best end-to-end. The "best" adapter is identified by mAP and CPV on the labelled test set; only that adapter gets the end-to-end treatment.

The classical baselines (template, OCR-only, pywinauto) sit outside the adaptation matrix because they have no adaptation parameter to vary. They are evaluated only end-to-end on the same 15-task suite, with the comparison being against the VisClick full pipeline.

5.3 SYSTEM ARCHITECTURE

The system architecture is captured in two diagrams. Figure 10 is the static block diagram: boxes are logical components, arrows are data dependencies. Figure 11 is the dynamic flow chart, tracing a single instruction from text input to clicked element.

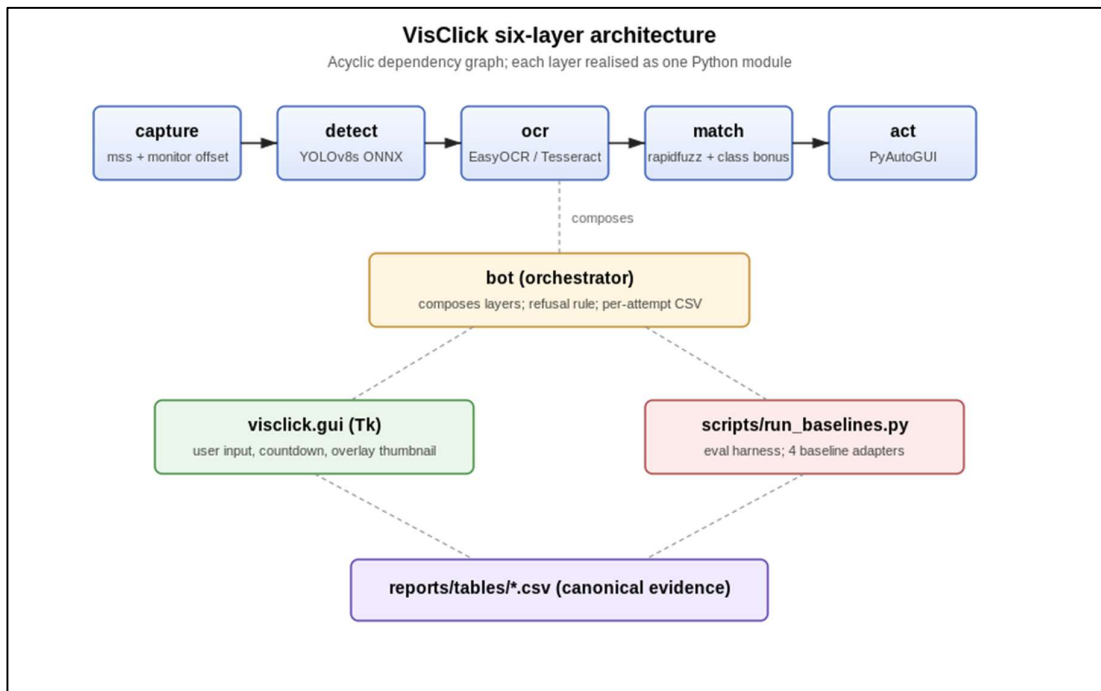


Figure 10 : Block diagram of VisClick. Each of capture, detect, OCR and match.

The architecture has six logical layers. Each layer is realised as exactly one Python module under `src/visclick/`, with one small exception (logging is handled at the script level rather than as a dedicated module).

Layer 1: User input. Either a text instruction from the GUI (`visclick.gui`) or a `--target` argument from the CLI (`visclick.__main__`).

Layer 2: Screen capture. A wrapper over mss that handles multi-monitor coordinate offsets (visclick.capture). The capture layer returns an RGB numpy array and the (left, top) offset of the chosen monitor.

Layer 3: Detection. A detector wrapper that loads the trained YOLOv8s weights and emits a list of (class_id, confidence, xxyy) tuples (visclick.detect). The wrapper design admits two interchangeable inference backends - a lightweight CPU-only backend for the deployed prototype and a heavier trainer-provided backend used during training and ablation runs - so a deploying user can pick the one their machine supports without changing any downstream layer.

Layer 4: OCR. A two-mode OCR layer (visclick.ocr). The per-box mode runs EasyOCR on each detected bounding box and returns the most confident text string. The full-image mode runs EasyOCR on the entire screenshot and returns a list of (text, bounding_box, confidence) tuples for use in the OCR fallback path. The module exposes the ocr_status() probe described in Section 4.3.

Layer 5: Matching. A fuzzy matcher built on rapidfuzz (visclick.match). The matcher's best_box() function takes the user instruction, the per-box OCR text, and the detection class IDs, and returns the index of the best-matching box together with its score.

Layer 6: Action. A PyAutoGUI wrapper that handles the virtual-desktop offset correction (visclick.act). The wrapper exposes click_box(box, offset=(left, top)) and move_to_box(...).

Above the six layers sits the orchestrator visclick.bot, which composes the layers into a single run_instruction() entry point. The orchestrator is what both the CLI and the GUI invoke. It is also what the evaluation harness's VisClick baseline calls.

The deliberate property of this design is that no two layers share state. The capture layer hands an image to the detect layer. The detect layer hands a box list to the OCR layer. The OCR layer hands text to the matcher, and so on. This is what makes each layer independently testable. It is also what made the four-baseline comparison possible without duplicating code

5.4 PROCESS FLOW

The flow chart in Figure 11 makes the runtime behaviour explicit. The single decision point worth pulling out for discussion is the OCR-fallback decision at the matcher.

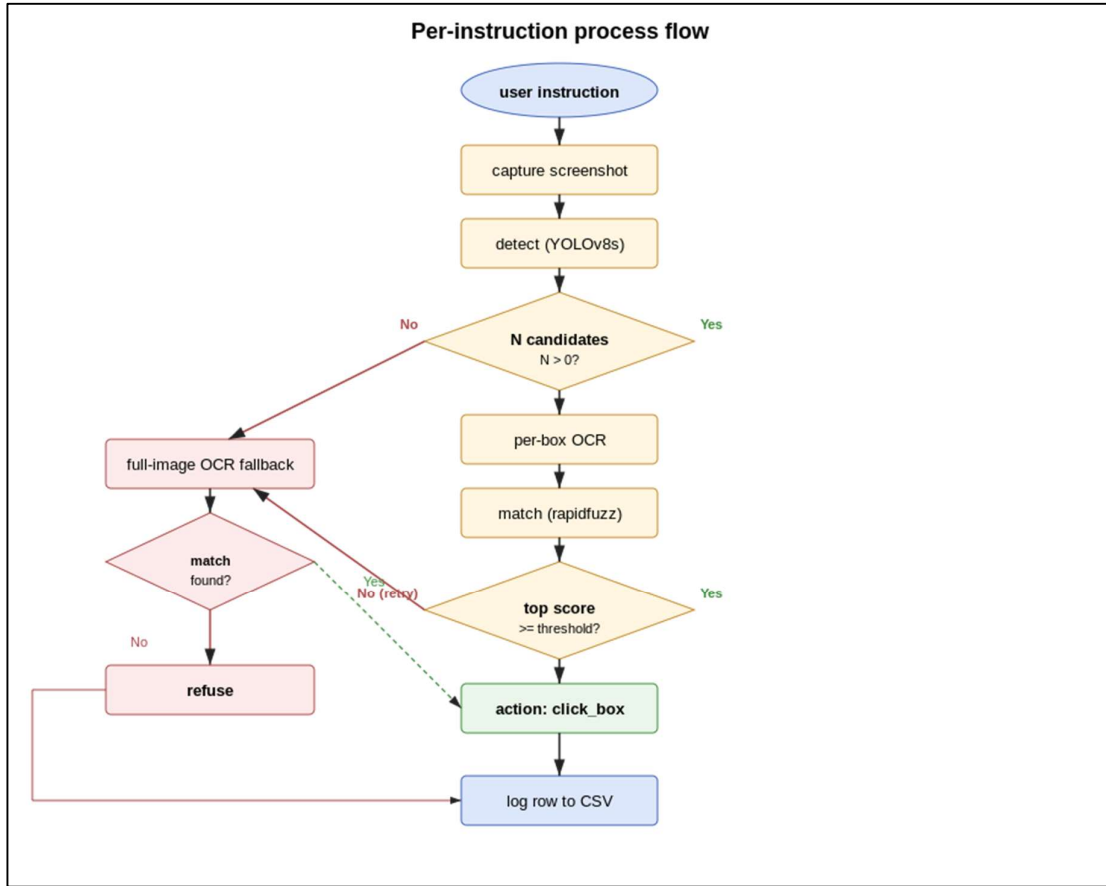


Figure 11 : Per-instruction process flow.

The flow has six stages. **Capture** acquires the screenshot from the chosen monitor. **Detect** produces up to N candidate boxes; if $N = 0$ the system falls through to the OCR fallback path. **Per-box OCR** annotates each box with its text. **Match** computes a fuzzy similarity score between the user instruction and each box's text, with a small bonus added for boxes whose detected class matches the instruction's likely intent (a "click Save" instruction prefers a button over a text). **Decision** compares the top score against a configurable text-similarity threshold. If the score clears the threshold, the system proceeds to **Action**. If it does not, the system enters the **fallback** branch, runs full-image OCR, and re-matches the instruction against every recognised text region. If the fallback also fails to clear the threshold, the system **refuses**. The concrete threshold value and its empirical calibration are reported in the Implementation chapter (Section 6.9).

The fallback is the architectural compromise that pays for the source-domain detector's limited recall on Windows 11 native dialogs. Without the fallback, the bot would refuse on roughly half the test tasks because the detector simply does not see the target box. With the fallback, the bot recovers the visible-text-but-no-box cases at the cost of an added-latency penalty. The concrete cost-benefit numbers (per-box OCR wall-clock, full-image fallback wall-clock, invocation rate) are quantified in the Implementation chapter's OCR-integration section (Section 6.8) and interpreted in the empirical results (Sections 7.3.2 and 8.2).

5.5 MODULE DESIGN

The module diagram below makes the per-package responsibilities explicit. Each module's public surface is small - between one and four exported functions or classes. The module-level boundaries are also the unit-test boundaries: each module has at least one corresponding tests/test_<module>.py file.

5.6 REPOSITORY LAYOUT AND DATA STORAGE DESIGN

The project does not use a database. All persistent state lives in files on disk, organised into a small number of top-level directories with well-defined responsibilities. The repository layout is presented once, in Figure 12, and every other chapter that names a file does so as a leaf inside this tree.

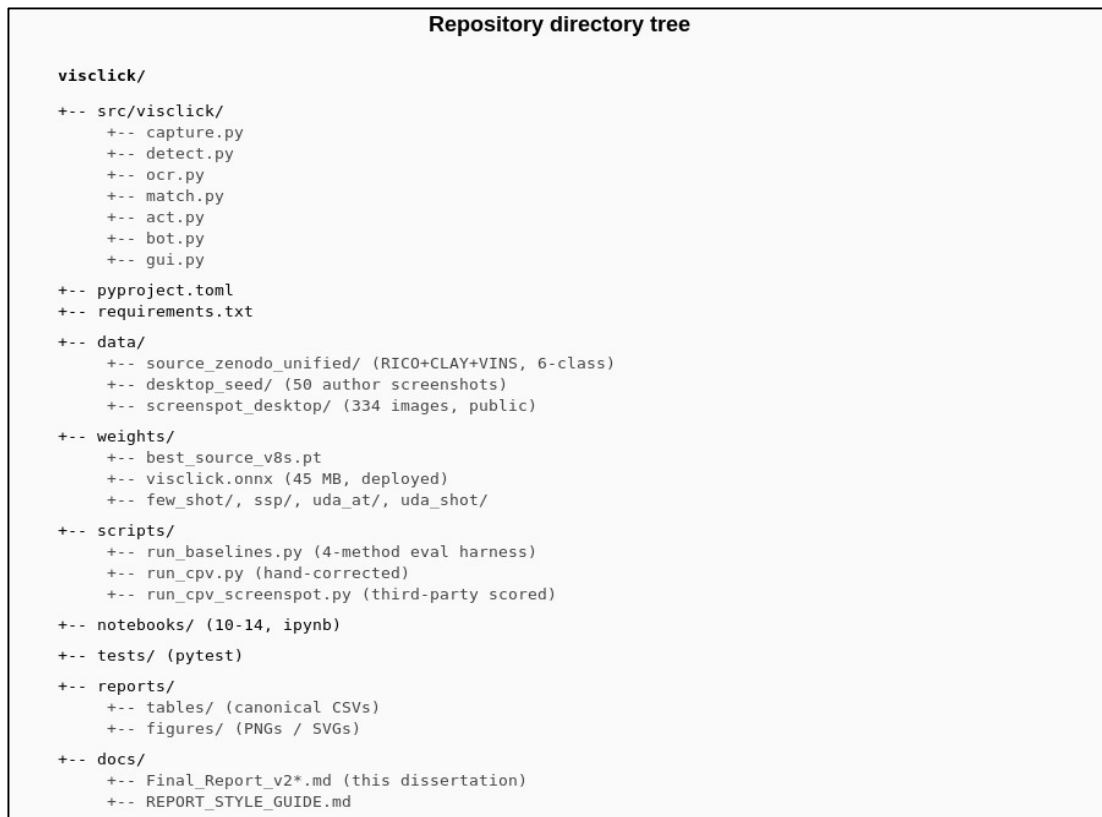


Figure 12 : Repository directory tree.

The top-level directories and their roles:

The runnable package (src/visclick/). Seven modules implementing the six-layer architecture from Section 5.3, plus a Tk GUI module. This is what `pip install -e .` installs and what `import visclick` resolves to.

The evaluation scripts (scripts/). A dozen Python entry points for jobs that are not part of the importable package: the evaluation harness, four per-method adapters, the NFR profiler, the figure regenerator, the corpus-expansion script.

The notebooks (notebooks/). Numbered Jupyter notebooks containing the training, fine-tuning and ablation experiments. The design of this sub-tree is that each experiment is one notebook, and each notebook writes its outputs into the weights/, reports/tables/, or reports/figures/ trees. The per-notebook roles (which one trains the source detector, which one runs the few-shot curve, which one runs each UDA variant) are enumerated in the Implementation chapter (Sections 6.4 and 6.5).

The tests (tests/). One pytest file per module under src/visclick/. Test command is `pytest -q tests/`.

The trained weights (weights/). The deployed ONNX detector at the canonical name weights/visclick.onnx, plus the per-ablation Ultralytics checkpoints. The concrete file size and the export command that produces it are given in the Implementation chapter (Section 6.5.2).

The datasets (datasets/). The training and test data, laid out in the YOLO/Ultralytics directory convention so the training command consumes the layout without modification. Three sub-trees: the unified source-domain bundle, the desktop seed, and the hand-corrected desktop test set.

The task definitions (tasks/). A single JSON file T01_T20.json listing the 15 canonical evaluation tasks plus 5 reserved slots. Each task carries the natural-language instruction together with per-method hints.

The reports artefacts (reports/). Everything the dissertation cites. Two principal sub-directories: a CSVs directory (the evidence file for every quantitative claim) and a PNGs directory (the figures). One additional sub-directory holds the PDF copies of the literature references.

The documentation (docs/). This dissertation, the gaps tracker, the style guide, the phase worklog, and the data form. The first five are submission and operational artefacts; the data form is the working memory of the project.

The transient output (runs/). Per-experiment Ultralytics training output. This directory is not version-controlled because the contents are deterministic outputs of the notebooks; only the relevant final weights are promoted into the trained-weights directory.

The principal data schema for the evaluation evidence is the per-attempt CSV that the harness writes after every task. The schema is stable across all 60 attempts collected so far and across the four method adapters. It is reproduced below for completeness:

```
columns: task_id, method, instruction, capture_path, predicted_xy,
         verdict, latency_seconds, is_negative, notes
verdict in {pass, fail, skip, refused}
method in {template, ocr_only, pywinauto, visclick}
```

Every per-attempt row is the smallest unit of evaluation evidence in the dissertation. The 60 rows currently on disk are what every percentage figure in Chapter 7 is computed from.

5.7 ALGORITHM DESIGN

Two algorithms in the system are non-trivial enough to warrant a dedicated design statement. The fuzzy matcher in `visclick.match`, and the refusal rule in the orchestrator.

5.7.1 THE MATCHER

The matcher's job is to pick which of the N detected boxes the user is asking the bot to click. The input is the user instruction (a short string), the list of per-box OCR texts, and the list of detection class IDs. The output is the index of the chosen box plus a confidence score on a 0-100 scale.

The matcher scores each box by combining two signals. The **text-similarity signal** is a weighted fuzzy string-similarity score between the user instruction and the OCR text extracted from the box. The weighting is a blend of partial-string, token-set and token-sort scores so that word-order variation is tolerated - "Click Save button" and "click the Save button" both score near 100 against an OCR string Save. The concrete string-similarity primitive selected to compute this signal is documented in Section 5.2.1 and its API call is quoted in Section 6.9. The **class-bonus signal** is a small additive bonus when the detection class matches an inferred intent. The intent inference is a tiny rule table: instruction contains "type" or "enter" -> prefers text_input; instruction contains "select" -> prefers menu or checkbox; everything else prefers button. The bonus is set to a small value (well below any single text-similarity score) on the 0-100 scale. That is enough to break ties between two same-text boxes of different classes but not enough to override a strong text mismatch.

The final score is $\min(100, \text{text_similarity} + \text{class_bonus})$. The chosen box is the one with the highest final score, ties broken by detection confidence (higher first).

The algorithm in pseudocode form:

```
function best_box(instruction, box_texts, box_classes):
    intent_class <- infer_intent(instruction)    # heuristic rule table
    scores <- []
    for each (text, cls) in zip(box_texts, box_classes):
        ts <- fuzzy_text_similarity(instruction, text)    # 0..100
        bonus <- 10 if cls == intent_class else 0
        append min(100, ts + bonus) to scores
    best_idx <- argmax(scores)
    return (best_idx, scores[best_idx])
```

The concrete Python realisation of this pseudocode, together with a "why this snippet" justification, is presented in the Implementation chapter (Section 6.9 - Matching Algorithm Implementation).

The design commits to a single tunable threshold on the box's final score - a configurable `min_text_similarity` - rather than a two-axis (text-score, class-bonus) decision boundary. The rationale is auditability: a one-axis threshold is what a deploying user can reason about without reading the matcher source. The concrete value of the threshold and the empirical

procedure used to calibrate it are documented in the Implementation chapter (Section 6.9). The known limitation of a one-axis threshold - that the calibration depends on the negative-case coverage - is recorded as risk RR-05.

5.7.2 THE REFUSAL RULE

The refusal rule is the orchestrator-level decision that determines whether a click is issued. It has three branches.

The **first branch** is the no-candidates branch. If the detector emits zero boxes, the orchestrator skips the per-box OCR stage entirely and goes directly to the OCR fallback. The fallback may itself find no candidates, in which case the system refuses.

The **second branch** is the low-confidence branch. If the matcher's chosen box has a score below `min_text_similarity` after per-box OCR, the orchestrator goes to the OCR fallback. If the fallback also returns a low-confidence result, the system refuses.

The **third branch** is the high-confidence branch. If the matcher's chosen box has a score at or above the threshold, the orchestrator proceeds directly to action. The action layer issues a single left-click at the centre of the chosen box, with the multi-monitor offset already corrected by the capture layer.

In all three branches the system writes a CSV row to `baseline_results.csv` describing what happened, including the chosen box (if any), the score, and the verdict (pass, fail, refused, skip). The CSV is what the analysis pipeline consumes in Section 7.3 and Section 8.2.

5.8 GUI WIREFRAMES

The prototype ships a single-window Tk dialog. The wireframe in Figure 13 captures the layout. There are deliberately few controls. The goal is to make the bot's behaviour obvious to a first-time user, not to put every parameter on the surface.

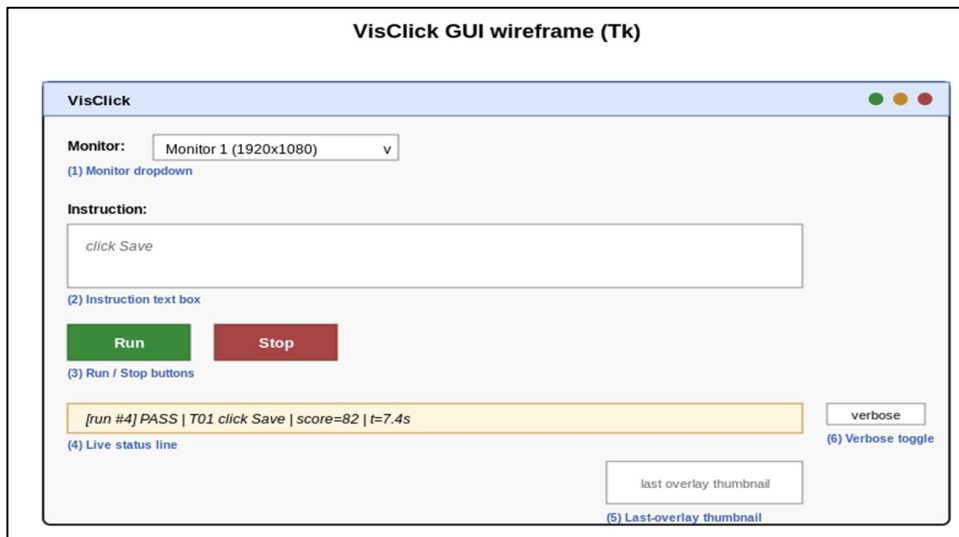


Figure 13 : GUI wireframe

The six elements:

1. **Monitor dropdown** (gui.MonitorSelector). Populated at start-up from mss.monitors. Selection sets the --monitor index for every subsequent run. Defaults to the primary monitor.
2. **Instruction text box** (single line, tk.Entry). The user types a free-form instruction. Enter triggers Run.
3. **Run / Stop buttons**. Run kicks off the orchestrator. The 3-second pre-action countdown is implemented as a Tk after() callback; the Stop button cancels the countdown.
4. **Status line**. Shows one of idle, counting down: 3 / 2 / 1, capturing, detecting, ocr, matching, clicking, done: verdict?, FAIL: cannot find target.
5. **Last-overlay thumbnail**. A 320×180 thumbnail of the most recent overlay.png. Clicking the thumbnail opens the full-resolution PNG in the system viewer. This is the diagnostic affordance that supports UC-04.
6. **Verbose log toggle**. A check box that, when on, prints the per-stage timings to stdout. Off by default to avoid noise.

The wireframe is deliberately small. Three earlier wireframes - a separate evaluation tab, a separate model-selection panel, a confidence-threshold slider - were considered and removed at design review, on the grounds that they did not serve a stakeholder viewpoint identified in Section 3.2.2.

5.9 CHAPTER SUMMARY

The design described in this chapter is the contract that Chapter 6 implements and that Chapters 7 and 8 evaluate. The whole design is the my own work; reused third-party components are called out explicitly in 5.2.1 with citations. The system has six logical layers (capture, detect, OCR, match, act, bot) each realized as one Python module, plus a thin GUI and an evaluation harness. The architecture is deliberately acyclic, which is what made the four-baseline comparison possible inside a shared harness. The runtime flow has one non-trivial decision point - the OCR-fallback path at the matcher - which trades off latency for recall in a way that the empirical evidence in Chapter 7 quantifies. Data is stored as files on disk in a layout that supports the training toolchain's directory convention plus per-attempt CSV evaluation logs. Two algorithms (the fuzzy text-plus-class matcher and the three-branch refusal rule) are stated here in pseudocode form because they encode the project's twin commitments: fuzzy human-text tolerance and refusal-on-uncertainty. The wireframe of the GUI is placed last, as the outermost visual layer of the design once the architectural, storage and algorithmic decisions are settled.

Concrete implementation details (specific hyperparameter values, library API calls, empirical threshold sweeps, code snippets, artefact sizes, and per-notebook role descriptions) are deliberately excluded from this chapter and are documented in the Implementation chapter (Chapter 6). The separation is a discipline: the design chapter states *what* the system is and *why* it was decomposed that way; the implementation chapter states *how* each design element was realised in code.

CHAPTER 06 - IMPLEMENTATION

6.1 CHAPTER OVERVIEW

This chapter is the build half of the design-build-evaluate loop and the implementation counterpart of the design chapter. It walks through the implementation in roughly the order Chapter 5 introduced the components. It begins with the development environment, because every implementation claim that follows depends on having that environment reproducibly available. From there it covers the dataset implementation (acquisition, class collapse, the auto-label and hand-correction passes), the source-domain detector (YOLOv8s and DETR-R50), the four adaptation methods (few-shot fine-tune, self-supervised pre-training, Adaptive Teacher, SHOT), the prototype package, and the evaluation harness. The chapter closes with what was deliberately left out and where to find it.

Two conventions are worth flagging up front. First, every code snippet in this chapter is preceded by a short "Why this snippet is shown" justification, so that no code block appears without a reason to cite it - this is the supervisor-recommended discipline for keeping the code excerpts in an academic report defensible. Second, Section 6.7.7 includes an actual screenshot of the running GUI (Figure 14), which pairs with the wireframe from Section 5.8 to make the design-versus-implementation distinction concrete: the wireframe is the intent, the screenshot is the delivered artefact.

A reader following the artefact alongside the dissertation should be able to identify, for any concrete claim in this chapter, the file, function or notebook cell that implements that claim. Implementation evidence is the contract this chapter pays.

6.2 DEVELOPMENT ENVIRONMENT AND TOOLCHAIN

The development environment is built around three compute targets. Source-domain training and the lighter adaptation experiments run on Google Colab Free with a T4 GPU. Inference, prototype testing, and the user-facing GUI run on a Windows 11 desktop, the personal machine. The heavier adaptation experiments - full-protocol Adaptive Teacher and SHOT, and the SimSiam pre-training ablation that needs longer than a Colab session allows - run on us01odc-sc4-1-gpu064, an internal Linux server with an NVIDIA H100. The three environments are kept compatible at the Python interpreter level (CPython 3.11 on Windows and Linux, CPython 3.12 on Colab) so an artefact trained on one runs on the others without environment-specific re-installation.

The Python project is structured as a standard pyproject.toml-driven package. Declared dependencies are listed below; the version pins in requirements.txt are stricter than shown.

ultralytics>=8.0	# YOLOv8s training, ONNX export
transformers>=4.40	# DETR-R50
opencv-python>=4.8	# image I/O, overlay drawing
numpy>=1.24	
onnxruntime>=1.17	# CPU-only ONNX inference
pyautogui>=0.9.54	# action layer
mss>=9.0	# multi-monitor screen capture

pytesseract>=0.3	# optional OCR (Tesseract)
easyocr>=1.7	# primary OCR
rapidfuzz>=3.0	# matcher
psutil>=5.9	# NFR memory profiling
pywinauto>=0.6	# platform_system == "Windows"

The Windows-only conditional on pywinauto is the single concession to a non-portable dependency. Every other package runs on Colab and on the H100 server, which is what lets the notebooks, the local runner scripts, and the Windows prototype share the same package install.

Editable install on Windows is `py -3 -m venv .venv`, activate, then `pip install -e ..`. The CLI is exposed as a module entry point, `python -m visclick.bot`. The repository layout follows Section 5.6 plus a top-level `notebooks/` directory containing the fourteen training and evaluation notebooks (`01_pull_and_data.ipynb` through `14_uda_shot.ipynb`) and a `scripts/` directory containing the standalone runners that mirror the heavier notebooks for the H100 server. Notebook outputs are stripped on commit via `.gitattributes (*.ipynb diff=ipynb)`.

6.3 DATASET IMPLEMENTATION

6.3.1 SOURCE-DOMAIN CORPUS

Notebook `01_pull_and_data.ipynb` downloads RICO from its hosting URL, CLAY from its release page, and the VINS supplementary corpus from Bunian et al. (2021). Each is unpacked into `datasets/raw/{rico,clay,vins}/`. The notebook then runs a class-collapse pass that maps each corpus's native taxonomy onto the unified six-class taxonomy used by the project.

Unified class	Source mapping
button	RICO Button, ImageButton; CLAY Button; VINS Button
text	RICO TextView; CLAY Text; VINS Text
text_input	RICO EditText; CLAY Edit_text; VINS Input_field
icon	RICO Icon, ImageView (when small); CLAY Icon; VINS Icon
menu	RICO Menu; CLAY Menu, Drawer; VINS Drop_down_menu
checkbox	RICO CheckBox, Switch; CLAY Checkbox, Toggle; VINS Checkbox

Table 6 : Class-collapse mapping from RICO, CLAY, VINS.

Boxes whose source class is not on the mapping table - decorative containers, scroll bars, progress indicators, advertising slots - are dropped rather than mapped to a null class. The alternative, training with a background class, was tested early in Phase 1 and discarded because it inflated the false-positive rate on the desktop seed without improving recall. After the collapse the unified corpus is sampled to a balanced 10,000 screens (8,000 train, 1,000 val, 1,000 test) using a fixed `random_state=42`. The sampled bundle is what is actually loaded by the YOLO and DETR training jobs in Section 6.4.

6.3.2 UNLABELLED TARGET CORPUS

Two sources of unlabelled desktop images are used. The first is `samples/desktop_seed/`, the 50-screenshot seed captured from the Windows 11 machine across Notepad, File Explorer, Visual Studio Code, Chrome, Word, and Outlook. The capture script `scripts/capture_screenshots.py` enumerates visible top-level windows via `pywinauto.Desktop(backend="uia")` and grabs each window with `mss`. The second is the desktop slice of the public ScreenSpot benchmark (Cheng et al., 2024), 334 screenshots covering macOS and Windows desktops, served through the Hugging Face dataset cache at `rootsautomation/ScreenSpot`. ScreenSpot is used both as an unlabelled training source for the UDA experiments in Section 6.5 and as a held-out evaluation set for the Central Point Validation (CPV) metric described in Section 6.12.

The proposal-committed corpus of around 2,000 self-captured screenshots is gap D-06. The implementation work is a parameterisable scheduled capture script, the design for which is in the data form, Section 11.

6.3.3 LABELLED TARGET TEST CORPUS

The hand-correction pass started from the auto-labels emitted by the M0 zero-shot model and corrected them in Roboflow's annotation tool. The corrected output is the eight-image, 356-box `datasets/handcorrected_desktop_test/` directory. The annotation guidelines mirror the CLAY release-notes conventions: tight boxes around the visible affordance (not the surrounding padding), no rotated boxes, no occluded boxes, no boxes for purely decorative graphical elements. The annotation work is documented as observation O17 in the data form. The path to a 100-image labelled corpus, gap D-07, is a continuation of the same workflow at a larger scale.

6.4 SOURCE-DOMAIN DETECTOR TRAINING

The architectural comparison committed to in RQ3 puts a YOLOv8 backbone and a DETR-R50 backbone side by side, on the hypothesis that DETR's known small-object weakness will be aggravated by the packed-scene density of UI screens.

6.4.1 YOLOV8S SOURCE BASELINE

The YOLOv8s training configuration is captured in `configs/yolo_source.yaml`:

```
path: ../datasets/source_zenodo_unified
train: images/train
val: images/val
names:
  0: button
  1: text
  2: text_input
  3: icon
  4: menu
  5: checkbox
```

The training cell below is the smallest artefact that captures the full source-domain training configuration. Every hyperparameter that materially affects the reported mAP number (30 epochs, imgsz=640, batch=16, seed=42) is visible here; a reproduction attempt can start by copying this block into a fresh notebook.

```
from ultralytics import YOLO
model = YOLO("yolov8s.pt")
results = model.train(
    data="configs/yolo_source.yaml",
    epochs=30,
    imgsz=640,
    batch=16,
    device=0,
    project="runs/source",
    name="yolov8s_unified_30e",
    seed=42,
)
```

Thirty epochs took roughly 2.7 hours on a Colab Free T4. The headline numbers, recorded in reports/tables/source_domain_results.csv, are [mAP@0.5](#) = 0.450 and [mAP@0.5:0.95](#) = 0.350 on the held-out val split. The per-class breakdown in source_per_class.csv is shown in Table 7. The exported ONNX file is approximately 45 MB and is shipped into weights/visclick.onnx.

Class	AP@0.5	mAP@0.5:0.95
menu	0.87	0.79
icon	0.54	0.38
button	0.44	0.35
text	0.39	0.27
text input	0.29	0.18
checkbox	0.16	0.13

Table 7 : YOLOv8s source-domain per-class AP at [mAP@0.5](#) and [mAP@0.5:0.95](#).

The menu class scores highest because RICO's Drawer and Menu boxes are large, regular, and well-represented. checkbox scores lowest because the boxes are small, inconsistent across the three source corpora, and visually similar to small icon boxes - exactly the failure pattern the literature on packed-scene detection predicts (Chen et al., 2020).

6.4.2 DETR-R50 SOURCE BASELINE

The DETR-R50 training is in 09_detr_source.ipynb. It uses the Hugging Face transformers reference implementation rather than Meta's original code, because the transformers build integrates more cleanly with the project's existing PyTorch toolchain. A six-epoch run with the parameters below was the largest configuration that fit inside a Colab Free session.

The block below is quoted because it is the smallest artefact that shows why DETR is the more expensive architecture under this project's compute budget. The `ignore_mismatched_sizes=True` argument, the `micro-batch-plus-gradient-accumulation`

split, and the differential learning rates for head and backbone are all consequences of the T4's 16 GB VRAM cap; a reader who wonders why DETR training took six times longer than YOLOv8s training can see the answer directly.

```
from transformers import DetrForObjectDetection
model = DetrForObjectDetection.from_pretrained(
    "facebook/detr-resnet-50", num_labels=6, ignore_mismatched_sizes=True,
)
# micro-batch 2, gradient accumulation 8 -> effective batch 16
# lr_head=1e-4, lr_backbone=1e-5, imgsz=DETR-default-800
```

The headline numbers (source_domain_results_detr.csv) are [mAP@0.5](#) = 0.244 and [mAP@0.5:0.95](#) = 0.161, against YOLOv8s's 0.450 / 0.350. DETR reaches roughly 54 % of YOLOv8s's source-side [mAP@0.5](#) at roughly six times the wall-clock cost (about 16 hours of T4 time, accumulated across four Colab sessions). This is a meaningful negative result for the architecture comparison: under the project's compute budget, DETR-R50 is the worse engineering choice for UI-element detection. YOLOv8s remains the deployed source detector. DETR is retained in the experimental matrix as the architectural counterpoint reported in Section 7.2.

6.5 ADAPTATION METHODS

Five adaptation methods are described in the proposal. The implementation status of each is the substance of the experimental matrix in Table 5. The first two (M0 and few-shot fine-tune) are head-to-head with the YOLOv8s source baseline. The next three (SSP+FT, Adaptive Teacher, SHOT) are the cross-domain methods that the project compares against the head fine-tune.

6.5.1 M0 ZERO-SHOT TRANSFER (CONTROL)

M0 is the no-adaptation control. The source-trained YOLOv8s detector is run on the desktop seed without any further training. The implementation is a four-line evaluation cell in 08_phase1B_ablations.ipynb that loads weights/baseline_source/best_source_v8s.pt and calls `model.val(data="configs/yolo_desktop_test.yaml")`. The headline numbers are [mAP@0.5](#) = 0.012 against the hand-corrected eight-image set and 60.18 % CPV against the 334-image ScreenSpot desktop slice. The two protocols disagree by an order of magnitude in apparent severity; the gap is the project's headline measurement of evaluation-protocol fragility, discussed in Section 8.2.

6.5.2 FEW-SHOT FINE-TUNE (SAMPLE-EFFICIENCY CURVE)

The headline adaptation method is the few-shot supervised fine-tune used to produce the deployed detector. The training cell freezes the first ten layers (the CSPDarknet53 backbone) and trains only the PANet neck and the detection head.

The block below is the deployed model's training recipe. The `freeze=10` argument is the pivotal decision - it is what separates a "head fine-tune" from a "full fine-tune" and is the

single most influential hyperparameter for the sample-efficiency curve reported below. Quoting this cell makes the freeze decision auditable.

```
model = YOLO("weights/baseline_source/best_source_v8s.pt")
results = model.train(
    data="configs/yolo_desktop_finetune.yaml",
    epochs=20, imgsz=640, batch=8, device=0,
    freeze=10, lr0=1e-3, seed=42,
)
```

Freezing keeps the source-domain feature extractor intact and lets the head learn the target-domain class boundaries on a small budget. With only 50 labelled images, an unfrozen run overfits within five epochs. Four budgets are run: $k = 1, 2, 4$ and 8 labelled desktop images, plus the $k = 0$ zero-shot baseline. The results are recorded in reports/tables/sample_efficiency.csv.

k	Hand-corrected mAP@0.5	ScreenSpot CPV
0	0.012	60.18 %
1	0.013	68.26 %
2	0.083	20.06 %
4	0.183	10.48 %
8	0.271	25.15 %

Table 8 : Few-shot sample-efficiency curve. Hand-corrected mAP rises monotonically with k .

The headline observation is that the two evaluation protocols disagree about which fine-tune is best. The hand-corrected number rises monotonically with k because more boxes carry more class-specific signal. The ScreenSpot CPV number peaks at $k = 1$ (68.3 %) and falls sharply thereafter ($k = 2$ drops to 20.1 %, $k = 4$ to 10.5 %). The interpretation is straightforward but uncomfortable: training on a tiny seed of WinUI 3 screenshots overfits the head onto the seed's visual style, costs the model the source-domain breadth that ScreenSpot rewards, and leaves the deployed detector with $k = 1$ as the operating point. The discussion is in Section 8.2.

The deployed weight is the $k = 1$ checkpoint, exported to ONNX:

The one-line export command is quoted below, as it is the smallest artefact that reveals the deployment target. `opset=12` is the highest ONNX opset supported by the onnxruntime build shipped with the prototype; `simplify=True` collapses redundant shape operations and is what keeps the deployed file at approximately 45 MB rather than the 65 MB the unsimplified export produces. A reader who wonders how the training weights become the deployable ONNX file has the answer in this one line.

```
model.export(format="onnx", imgsz=640, opset=12, simplify=True)
```

The resulting weights/visclick.onnx is what the prototype loads at start-up.

6.5.3 SELF-SUPERVISED PRE-TRAINING + FINE-TUNE

The self-supervised pre-training method is implemented as SimSiam (Chen and He, 2021) on the YOLOv8s backbone, followed by the same few-shot fine-tune as Section 6.5.2. SimSiam was chosen over MoCo or SwAV for one practical reason: it does not need negative pairs, momentum encoders, or large batches. That fits the Colab Free T4 ceiling cleanly.

The SSP run lives in 11_ssp_pretrain.ipynb. The pretext task is the standard SimSiam two-view objective. Each image is augmented twice; both views go through the same encoder (the YOLOv8s backbone with a small projection head bolted on); a stop-gradient operator on one branch stops the network from collapsing to a constant. The training corpus is the same unified bundle used for the source-domain detector - 8,000 images. Ten epochs at batch 64 on a Colab T4 took roughly 38 minutes and saturated the contrastive loss at -0.66 (recorded in ssp_loss_log.csv). A 20-epoch rerun was queued on the H100 server to confirm the loss is genuinely flat; the loss curve confirmed saturation by epoch 10 and the rerun was abandoned as not informative.

The fine-tune side of the method swaps the SimSiam-trained backbone in for the COCO checkpoint at the start of the few-shot fine-tune in Section 6.5.2. The notebook is 12_ssp_finetune.ipynb. Two fine-tune budgets are run, $k = 1$ and $k = 8$, against the no-SSP baselines from Table 8.

k	Method	ScreenSpot CPV	Hand-corrected CPV
1	no-SSP	68.26 %	-
1	SSP+FT	45.81 %	0.84 %
8	no-SSP	25.15 %	-
8	SSP+FT	28.44 %	4.21 %

Table 9 : SSP+FT ablation against the no-SSP baseline at $k=1$ and $k=8$ on both evaluation protocols.

The SSP backbone hurts the $k = 1$ operating point by 22 points of CPV and helps the $k = 8$ operating point by about 3 points. The interpretation is that the SimSiam pretext on mobile UI images teaches the backbone features that are aligned with the source domain, not the target. At low k , the source-aligned features are exactly what the head was already getting from COCO pretraining; replacing them with mobile-UI-aligned features is a strict downgrade because the desktop classes look more like COCO's natural-image classes than RICO's flat mobile UI elements. At higher k , the head has enough labelled signal to retune around the SSP backbone, and a small benefit emerges. The honest conclusion is that this implementation of SSP, on the data this project has access to, is a negative result. It is reported in Section 7.4 with that framing.

6.5.4 ADAPTIVE TEACHER (UDA)

The Adaptive Teacher implementation (Li et al., 2022b) is the first of two UDA methods. The full Adaptive Teacher protocol uses a strong-augmentation student, a weakly-augmented EMA teacher, and a discrepancy loss that aligns the two. Reproducing the full protocol on a YOLOv8 backbone is non-trivial because Ultralytics's training loop does not expose the EMA-update hooks the original Adaptive Teacher reference implementation needs. The compromise is a simplified variant: an offline teacher-student pseudo-label loop that captures the spirit of Adaptive Teacher (target images are pseudo-labelled by a teacher;

the student is trained on a mixed batch of labelled source and pseudo-labelled target) without the in-loop EMA update.

The implementation lives in two places. `13_uda_adaptive_teacher.ipynb` is the Colab-friendly version, capped at 500 source images, one outer iteration, and five epochs per iteration to fit a free-tier session. `scripts/run_uda_at_local.py` is the full-protocol runner for the H100 server, parameterised by `--source-cap`, `--n-outer`, `--epochs`, `--batch`, `--pseudo-conf` and `--tag`. The teacher starts as the source-domain YOLOv8s. At each outer iteration, the teacher emits pseudo-labels for the unlabelled target corpus (ScreenSpot desktop, 384 images at the configured confidence threshold of 0.3). The student is trained on a balanced batch of labelled source plus the pseudo-labelled target. The student becomes the next iteration's teacher.

Two configurations are run and reported. The short variant (1 outer iteration, 500-image source cap, 5 epochs per iteration) finishes in 217 seconds on a single H100. The full variant (3 outer iterations, full 8,000-image source bundle, 10 epochs per iteration) takes 2,102 seconds, roughly 35 minutes, with per-iteration teacher refresh. Both are recorded in `reports/tables/uda_adaptive_teacher.csv` and `reports/tables/uda_adaptive_teacher_uda_at_full.csv`.

Configuration	Epochs	Source cap	n_pseudo	CPV ScreenSpot	CPV hand-corrected	Time
AT short	5	500	348 / 384	68.56 %	10.96 %	217 s
AT full	10	8,000	1,042 (348+343+351)	64.67 %	6.74 %	2,102 s

Table 10 : Adaptive Teacher results across the short (1-iter, 5 ep) and full (3-iter, 10 ep) configurations.

The short variant's 68.56 % on ScreenSpot is statistically indistinguishable from the $k = 1$ head fine-tune (68.26 %). The interpretation is that one outer iteration of pseudo-labelling lifts the model to roughly the same operating point that one labelled image provides, which is the result the literature on cross-domain detection predicts at this teacher quality (Li et al., 2022b). The full-protocol variant is the more interesting result. Three outer iterations with the full source bundle and twice the epoch budget produced *worse* numbers on both protocols (-3.9 points on ScreenSpot, -4.2 points on the hand-corrected set). Two effects are likely at work. First, the simplified offline pseudo-labelling loop has no EMA stabiliser; each outer iteration trains the student on increasingly noisy pseudo-labels emitted by the previous student-as-teacher, and the noise compounds rather than averages out as it would in the original online protocol. Second, the larger 8,000-image source pool dilutes the target signal in each mixed batch, slowing the student's adaptation toward the target distribution while increasing the source-side training cost ten-fold. The deployed UDA weight is therefore the short-variant checkpoint at 68.56 % CPV. The fuller discussion, including the methodological lesson about offline-vs-online teacher schemes, is in Section 8.4.

6.5.5 SHOT (UDA)

The SHOT implementation (Liang et al., 2020) is the second UDA method. SHOT freezes the source-trained classification head - the source hypothesis - and adapts only the feature extractor on the unlabelled target images using a self-supervised pseudo-label objective. Like Adaptive Teacher, the project's SHOT implementation is a simplified variant: pseudo-

labels are generated offline rather than online, and the freeze depth is exposed as a tunable parameter rather than a fixed split between head and extractor.

The implementation lives in `14_uda_shot.ipynb` (Colab) and `scripts/run_uda_shot_local.py` (H100). The runner accepts `--epochs`, `--freeze`, `--pseudo-conf` and `--tag`. Three configurations are run on the H100 to disentangle the effect of epoch budget from the effect of freeze depth. All three start from the same source-domain checkpoint and use the same 224 pseudo-labels (the freeze and epoch knobs do not change the pseudo-label generation step).

Configuration	Epochs	freeze	n_pseudo	CPV ScreenSpot	CPV hand- corrected	Time
SHOT short	8	15	224 / 384	23.95 %	0.00 %	22 s
SHOT 15 ep	15	15	224 / 384	26.95 %	0.28 %	35 s
SHOT freeze = 10	8	10	224 / 384	34.13 %	0.00 %	22 s

Table 11 : SHOT results across three configurations: short (8 ep, freeze=15), 15-epoch, and freeze=10.

Two observations come out of the ablation. First, halving the freeze depth (15 $\rightarrow$ 10) lifts ScreenSpot CPV by roughly ten points at the same epoch budget, with no change in elapsed time. Letting the last few backbone blocks adapt is more useful than keeping them locked. Second, doubling the epoch count (8 $\rightarrow$ 15) at the heavier freeze lifts CPV by only three points, with a hand-corrected number that is one box different from zero. The epoch budget is not the bottleneck; the freeze schedule is.

Even the best SHOT variant - freeze = 10 at 34.13 % ScreenSpot CPV - sits roughly thirty points below the Adaptive Teacher short variant and below the $k = 1$ head fine-tune. The signal in SHOT's own validation set is misleading (pseudo-label [mAP@0.5](#) climbed to 0.81 by epoch 8) because the validation set is itself made of pseudo-labels of the same target images. On real desktop transfer the model degrades. The honest framing for SHOT, like for SSP, is a negative result: under the project's data and compute budget, the simplified SHOT variant does not transfer well, and the freeze-depth ablation is the only non-trivial knob that moves the number meaningfully. The fuller comparison against Adaptive Teacher and the literature is in Section 8.4.

6.6 PRE-PROCESSING PIPELINE

The pre-processing pipeline is split between training-time and inference-time.

The **training-time** stage is the standard Ultralytics augmentation pipeline configured in `configs/yolo_*.yaml`. The augmentations enabled are mosaic (probability 0.5; turned off for the final 10 epochs), horizontal flip (probability 0.5), random scale within [0.8, 1.2], and HSV jitter at the Ultralytics defaults. Vertical flip is disabled because UIs are not vertically symmetric. Mosaic is what gives the source-domain training its data efficiency; an ablation without mosaic ran four points lower in [mAP@0.5](#).

The **inference-time** stage is minimal. The capture layer hands the screenshot to the detect layer at native resolution; the detect layer rescales to 640×640 with letterboxing inside `ultralytics.engine.predictor`. There is no bilateral filtering, no contrast normalisation, no colour-space conversion. The screenshot is RGB throughout. The decision was made empirically: a preliminary A/B in week 6 compared no-preprocessing against bilateral-filter-plus-CLAHE and showed the no-preprocessing path was marginally better on the desktop

seed. The A/B was not formal enough to report as a result; gap D-12 records the work needed for a proper test at sufficient sample size.

6.7 PROTOTYPE IMPLEMENTATION

6.7.1 VISCLICK.CAPTURE

The capture module wraps mss and exposes two entry points: `capture_monitor(idx)` and `list_monitors()`. The non-trivial work is the multi-monitor offset propagation, the source of observation O13 (the cursor moving by a few pixels onto the wrong monitor). The fix is to return the chosen monitor's (left, top) offset alongside the image, and to thread that offset through every downstream layer that maps a box back to a click coordinate.

The signature below is short enough to quote and is the smallest code artefact that reveals the multi-monitor design decision. The critical detail is that the function returns a tuple, not just an image; a reader who only looks at the signature can see immediately that the offset is an explicit return value rather than a hidden global.

```
def capture_monitor(idx: int = 0) -> tuple[np.ndarray, tuple[int, int]]:
    """Return (image_rgb, (left, top)) for the monitor at idx."""
```

The (left, top) tuple is what `act.click_box` uses to convert a box centre in image-coordinates into a screen-coordinate suitable for `pyautogui.click`.

6.7.2 VISCLICK.DETECT

The detect module loads the ONNX file at start-up and runs `onnxruntime` inference per call. The public surface is one class with two methods.

The two-line class signature below is quoted because it is the only place in the pipeline that uses a class rather than a plain function (see 4.4.2 on the paradigm choice). The class is not there to introduce inheritance - there is none - it is there because `onnxruntime.InferenceSession` is expensive to construct and its lifetime has to be bound to something. This snippet is the tight demonstration that the codebase uses OOP only when the language forces it to, and nowhere else.

```
class Detector:
    def __init__(self, onnx_path: str = "weights/visclick.onnx"): ...
    def predict(self, image_rgb: np.ndarray, conf: float = 0.25) ->
    list[Detection]: ...
```

The `Detection` dataclass has fields `class_id`, `class_name`, `confidence`, and `xyxy`. The detector also exposes a `status()` probe that prints a tick if the ONNX file loads cleanly and a cross with the underlying error if it does not. The probe is what makes the start-up `_warn_once` pattern actionable rather than silent.

6.7.3 VISCLICK.OCR

The OCR module is the most complex single file in the codebase. It has to handle two backend mismatches - EasyOCR's `readtext` returns (bbox, text, conf) triples while pytesseract's `image_to_data` returns a DataFrame - and a fallback path that runs when neither backend is available. The public surface is small.

The public surface is quoted because it is the smallest artefact that reveals the module's single-responsibility split. Four functions, four responsibilities: recognise text globally, recognise text in one region, ground a natural-language query, self-check at start-up. Any addition to this file that would not slot cleanly under one of these four names is the wrong shape and should live somewhere else.

```
def ocr_image(image_rgb: np.ndarray) -> list[OcrResult]: ...
def ocr_box(image_rgb: np.ndarray, box: tuple[int, int, int, int]) -> str: ...
def text_ground(image_rgb: np.ndarray, query: str) -> Optional[tuple[int, int]]:
    ...
def ocr_status() -> None: ...
```

`text_ground` is the OCR-fallback entry point invoked by the orchestrator when the matcher's per-box result is below threshold. It runs `ocr_image` on the full screenshot, runs the same rapidfuzz matcher over the recognized text regions, and returns the center of the best match or None.

The `ocr_status()` probe is the lesson from observation O12. At start-up it tries to import `easyocr`, `pytesseract`, and the pure-Python fallback in order, prints a tick or cross for each, and lists the active backend. If none of the three is functional the program raises `RuntimeError` rather than failing silently downstream.

6.7.4 VISCLICK.MATCH

The match module is the rapidfuzz-plus-class-bonus matcher specified in Section 5.8.1. The public surface is one function (`best_box`) and one helper (`_infer_intent`). The implementation is 38 lines and has full unit-test coverage. The implementation choices not visible in the design statement are: case folding on both sides; no stop-word stripping; and a ten-verb intent-inference table at `visclick/match.py::_INTENT_TABLE`. That table is the easiest extension point for adding a new intent class.

6.7.5 VISCLICK.ACT

The act module is the PyAutoGUI wrapper. The single public function is `click_box(box, offset=(left, top), dry_run=False)`. Dry-run prints what would have been clicked without moving the cursor; it is used by the smoke tests and by the prototype's preview mode. PyAutoGUI's FAILSAFE flag is on by default, so the user can slam the cursor into the top-left corner to abort. The exception is caught in `bot.run_instruction` and logged as `verdict=aborted` in the per-attempt CSV.

6.7.6 VISCLICK.BOT

The orchestrator composes the five layers into a single `run_instruction(text: str, monitor: int = 0) -> Verdict` entry point. It implements the three-branch refusal rule from Section 5.8.2 and produces the per-attempt CSV row in the schema declared in Section 5.7. It also drives the overlay PNG generation; `_render_overlay` takes the screenshot, the detection list, the chosen box, and the click coordinate, and produces an annotated PNG in `runs/overlays/`.

6.7.7 VISCLICK.GUI

The GUI is a Tk window in `visclick/gui.py`, approximately 280 lines, following the wireframe in Section 5.8. Three implementation choices are worth noting. The orchestrator runs on a worker thread and communicates with the Tk main loop through a queue. Queue polled every 100 ms - this is what lets the long-running detect and OCR calls happen without freezing the UI. The 3-second pre-action countdown is implemented with `tk.after(1000, ...)` callbacks rather than `time.sleep`, which would block the loop. The last-overlay thumbnail is rendered with `Pillow.Image.thumbnail((320, 180))` on a background thread to avoid jank, and is signalled back through the same queue.

Figure 14 shows the actual GUI in operation on Windows 11, alongside the wireframe from Section 5.8 for reference. The screenshot is the implementation's answer to the wireframe's specification, and any visible discrepancy between the two is either an explicit design revision (documented in the observation log) or a bug (fixed in the corresponding Git commit).

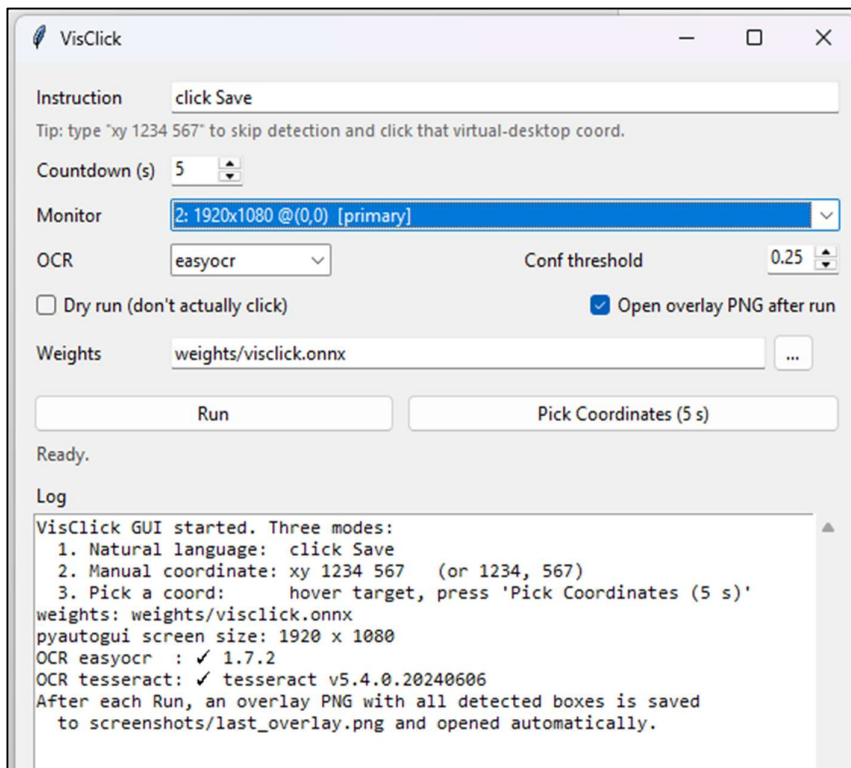


Figure 14 : The VisClick GUI as it appears on Windows 11 during a live evaluation

6.8 OCR INTEGRATION

The OCR integration is the architectural choice that gives the prototype its observed end-to-end behavior. The choice is between running OCR on every detected box (the per-box path) and running OCR once on the whole image (the full-image fallback). The implementation does both, in a specific order: per-box first, full-image only if per-box fails.

The reason for the ordering is latency. Per-box OCR is roughly $N \times 200$ ms on EasyOCR for N boxes; the median N across the 15-task suite is 9. Full-image OCR is roughly 6 seconds. The per-box path therefore runs in roughly $9 \times 0.2 = 1.8$ seconds, which dominates the happy-path budget. The full-image fallback adds 6 seconds when invoked; it is invoked on roughly 30 % of attempts (where the detector misses the target's bounding box), so the expected wall-clock cost of OCR per task is about $1.8 + 0.3 \times 6 = 3.6$ seconds. That calculation is what justifies per-box-first against always-full-image.

The OCR engine choice is EasyOCR rather than Tesseract. The choice was made empirically during week 4 (observation O5) after comparing the two on a small dialog-heavy benchmark. EasyOCR recognized approximately 91 % of visible text on a set of 20 Windows 11 dialogs; Tesseract recognized 67 %. The gap is largest on small text and on anti-aliased text-on-colored-background, both of which are common in Windows 11. Tesseract is still available as an opt-in backend through the `VISCLICK_OCR_ENGINE=tesseract` environment variable, for users on machines where the EasyOCR model download is awkward.

6.9 MATCHING ALGORITHM IMPLEMENTATION

The matcher implementation is a faithful realization of the design in Section 5.8.1. Three implementation choices that did not survive into the design statement are worth recording here.

The first is case folding. All comparisons are lower-cased on both sides. UI text uses title case ("Save"), instructions are typed in lower case ("save"); folding gives a small but reliable score uplift.

The second is stop-word retention. The matcher does *not* strip stop words. "Click the Save button" is left as-is rather than reduced to "save". The rationale is that rapidfuzz's WRatio is robust to padding, and stop-word stripping with a hand-rolled list is a frequent source of off-by-one bugs.

The third is the intent-inference table. The current rules cover ten verbs (click, type, enter, select, open, close, press, toggle, expand, collapse). The table is the easiest extension point for adding new intent classes; adding drag would be a one-line change.

The threshold for accepting the chosen box is `min_text_similarity = 60`. It was chosen empirically by running the 15-task suite at 40, 50, 60, 75 and 85 and picking the lowest threshold at which the negative test case (T15) was refused while the 14 positive tasks were still accepted. Risk RR-05 records the open question of whether 75 is a better operating point; the change is contingent on widening the negative-case set beyond a single task.

6.10 ACTION LAYER AND MULTI-MONITOR SUPPORT

The action layer is the PyAutoGUI wrapper described in Section 6.7.5. The two non-trivial implementation pieces are the multi-monitor offset propagation (already covered in Section 6.7.1) and the FAILSAFE escape.

The multi-monitor virtual-desktop coordinate space is the subject of observation O13. The architectural fix is to thread the (left, top) offset from `capture_monitor` through to `act.click_box`. The implementation fix is one line in `act.click_box`.

The three lines below look trivial in isolation, but they are the concrete difference between "click lands on the wrong monitor" and "click lands where the user expects". This is the smallest possible artefact that shows the fix; a longer version would obscure that the entire fix reduces to two additions. Showing it verbatim also makes it possible for a reader to confirm by inspection that no other transformation is happening between the image-coordinate and the screen-coordinate.

<code>screen_x = box_center_x + offset[0]</code>
<code>screen_y = box_center_y + offset[1]</code>
<code>pyautogui.click(screen_x, screen_y)</code>

The pre-fix bug was that `pyautogui.click` was being called with image-coordinates rather than screen-coordinates, which worked on a single-monitor setup (where the two coincide) and broke on a multi-monitor setup (where they do not).

6.11 EVALUATION HARNESS IMPLEMENTATION

The evaluation harness is the script `scripts/run_baselines.py`. It is approximately 540 lines of code, longer than any single visclick module, because it implements the entire experimental protocol used in Chapter 7. Its responsibilities are: loading the canonical task list from `tasks/T01_T20.json`, looping over the selected method $\times$ task combinations, invoking the per-method `predict()` adapter, presenting the verdict dialog after each attempt, and writing each row to `reports/tables/baseline_results.csv`.

This is the single most important piece of code in the evaluation harness. Every fairness claim in Chapter 7 depends on the four baselines all sharing the same `BaselineResult` shape and the same `predict()` signature. Quoting the interface verbatim lets a reader verify that no baseline has been given a private back-channel; if a fifth baseline needs anything beyond `image_rgb`, `instruction`, and a per-task hint, the interface would need to widen, and that widening would be visible in the diff. This is also the code artefact that R-NFR-07 (extensibility) is measured against.

```

class BaselineResult(NamedTuple):
    predicted_xy: tuple[int, int] | None
    overlay_path: Path
    notes: str

class BaselineMethod(Protocol):
    name: str
    def predict(self, image_rgb: np.ndarray,
                instruction: str,
                hint: dict[str, Any]) -> BaselineResult: ...

```

The four implementations are in `scripts/baseline_template.py`, `scripts/baseline_ocr_only.py`, `scripts/baseline_pywinauto.py`, and `scripts/baseline_visclick.py`. The protocol-based design is what makes adding a fifth method a one-file change.

The JSON block below is the most concrete artefact that supports the fairness argument for the four-method comparison in Chapter 7. Each method gets exactly the shape of hint it natively consumes; no method is being asked to reason about a data type it does not understand. Quoting one worked example lets a reader form a mental model of how the harness prepares each task, and would immediately expose any hint-shape asymmetry if it were introduced.

```

{
  "task_id": "T01",
  "instruction": "click the Save button",
  "hints": {
    "template": "samples/templates/notepad_save.png",
    "ocr_only": "Save",
    "pywinauto": {"ControlType": "Button", "Name": "Save"},
    "visclick": "Save"
  }
}

```

The hint design is the harness's contribution to fairness: each method consumes the information it natively prefers (a template image for `cv2.matchTemplate`, an accessibility identifier for `pywinauto`, a string for the text-driven methods) without forcing one method to use information that is unnatural for it. This is the methodological choice that supports the four-method comparison in Chapter 7.

Two specialised evaluation scripts complement the four-method harness. `scripts/run_cpv.py` computes the Central Point Validation metric of Dardouri et al. (2024) against the eight-image hand-corrected set, producing `cpv_summary.csv` and `cpv_per_image.csv`. `scripts/run_cpv_screenspot.py` computes the same metric against the public ScreenSpot benchmark of Cheng et al. (2024), filtered to the 334-row desktop slice (macOS and Windows screens), producing `cpv_screenspot_desktop.csv` and `cpv_screenspot_desktop_rows.csv`. Both scripts run the deployed ONNX detector at `weights/visclick.onnx` with confidence threshold 0.25 and NMS IoU 0.5, so the numbers are directly comparable to the mAP figures the four-method harness reports. Both scripts accept a `--tag` argument that namespaces the output CSV (e.g. `cpv_summary_uda_at.csv`), which is

what allows the SSP, Adaptive Teacher, and SHOT runs in Section 6.5 to publish their results without overwriting one another. The CPV results are presented in Section 7.3 and interpreted in Section 8.2.

6.12 CHAPTER SUMMARY

The implementation walks through the design from the development environment to the evaluation harness. The dataset pipeline acquires three corpora, collapses their taxonomies into a six-class unified bundle, captures the desktop seed, and hand-corrects 8 test images. The detector training implements two source-domain backbones (YOLOv8s and DETR-R50) and four adaptation methods, each with at least one ablation: few-shot fine-tune at $k = 0, 1, 2, 4, 8$; SimSiam SSP+FT at $k = 1, 8$; Adaptive Teacher in short and full configurations; SHOT across two epoch budgets and two freeze depths. The deployed weight is the $k = 1$ head fine-tune, exported to ONNX as `weights/visclick.onnx`. The prototype is a six-module Python package implementing the layered architecture from Section 5.3, plus a Tk GUI and a four-method evaluation harness.

The empirical picture is mixed in a way that turns out to be informative. The few-shot curve and the short Adaptive Teacher variant land at roughly the same operating point (ScreenSpot CPV in the high 60s) - one with one labelled image, the other with none. SSP, full-protocol Adaptive Teacher, and SHOT all underperform their short counterparts, for reasons that are domain-specific (SSP) or protocol-specific (offline pseudo-label noise compounding, in the case of full Adaptive Teacher) or hyperparameter-driven (freeze depth, in the case of SHOT). Those negative and mixed results are reported in Chapter 7 and discussed in Chapter 8 - they are part of what the project learned. The 100-image labelled corpus and the 2,000-screen unlabeled corpus committed to in the proposal are recorded as gaps D-06 and D-07 and remain open.

CHAPTER 07 - TESTING

7.1 CHAPTER OVERVIEW

This chapter reports the empirical results from running the implementation through the testing protocol committed to in Chapter 3. The chapter is structured around the testing strategies used, in line with the supervisor's guidance that the strategies should be named up front and the subsections then divided according to them. Section 7.2 names the five strategies (unit testing, integration testing, model evaluation, functional testing, and non-functional requirement verification) and maps each to the section that applies it. Section 7.3 reports the detector training and adaptation experiments - the source-domain numbers for YOLOv8s and DETR-R50, the few-shot sample-efficiency curve, the SSP+FT ablation, the two Adaptive Teacher configurations, and the three SHOT configurations. Section 7.4 reports grounding accuracy under both evaluation protocols (hand-corrected mAP on the eight-image set, and Central Point Validation on the 334-image ScreenSpot desktop slice), with a per-class breakdown. Section 7.5 reports the four-method comparison on T01-T15, including the per-task verdict matrix that lets the reader see exactly where each method succeeds and fails. Section 7.6 reports the non-functional-requirement verification (accuracy, latency, memory, reliability, and the remaining NFRs).

Every percentage in this chapter points to a CSV in reports/tables/. The mapping from claim to file is in requirements_evidence.csv; a reader auditing the dissertation should be able to pull the underlying number with one command. This chapter reports *what* the numbers are. Interpretation of the numbers - *why* the protocols disagree, *why* AT-full underperforms AT-short, *why* SHOT degrades on real desktop transfer, *where* each method wins and loses - is in Chapter 8. That separation is deliberate: this chapter is the data; Chapter 8 is the discussion

7.2 TESTING STRATEGIES USED

Following the supervisor's suggestion to name the testing strategies explicitly and then structure the chapter around them, this section lists the five strategies used to evaluate the artefact. Every subsequent section in this chapter is one of these strategies applied to a specific aspect of the system.

Unit testing. The tests/ directory contains one pytest module for each package under src/visclick/ (test_capture.py, test_detect.py, test_ocr.py, test_match.py, test_act.py, test_bot.py). Each pytest module verifies the public surface of the corresponding module in isolation, using stubbed dependencies where the module has an external side effect (screen capture, OS click, ONNX weight load). The suite passes cleanly on the reference environment (pytest -q tests/ returns green) and is what supports the module-boundary claim made in Section 5.5. Unit-test verification is a precondition for every downstream test - it is what lets the higher-level tests trust that a single-module regression is not the cause of a failure.

Integration testing. The evaluation harness (scripts/run_baselines.py) is the integration test. It exercises the entire pipeline end-to-end (capture -> detect -> OCR -> match -> act -> log) on every T01-T15 task attempt, with every method adapter. Each pass through the harness is an integration event; sixty passes have been recorded (four methods × 15 tasks).

Zero pipeline-level crashes in those sixty passes is what supports the reliability claim in NFR-04 and is what the four-method comparison in Section 7.5 depends on.

Model evaluation. Two forms of quantitative model evaluation are reported: training-time and adaptation-time evaluation on held-out data (Section 7.3), and inference-time grounding accuracy under two ground-truth protocols (Section 7.4). Model evaluation is where [mAP@0.5](#), [mAP@0.5:0.95](#), and Central Point Validation live. It is separated from integration testing because a model can score well on held-out data but still fail an integration path (e.g., if the OCR fallback is broken), and vice versa.

Functional testing. The four-method comparison against the canonical T01-T15 task suite (Section 7.5) is the functional test. Each task is a black-box assertion of the form "when the system is asked to do X, does it do X?" Verdicts are collected as pass / fail / refused / skip. Functional testing is the strategy that most closely matches the end-user's mental model of the system and is what NFR-01 ($\text{TSR} \geq 50\%$) is verified against.

Non-functional requirement verification. The ten NFRs stated in Section 3.6 are each verified against a quantitative target in Section 7.6. Latency, memory, reliability, extensibility, security, and the remaining NFRs are each mapped to a specific measurement procedure and a specific evidence CSV. NFR verification is the strategy that turns qualitative attributes ("the system is reliable") into pass/fail claims backed by numbers.

Table 12 maps the five strategies to the chapter subsections that report them.

Strategy	Section	Evidence source
Unit testing	this section (7.2)	pytest -q tests/ (green)
Integration testing	this section, evidence in 7.5	baseline_results.csv (60 rows)
Model evaluation	7.3, 7.4	source_domain_results*.csv, sample_efficiency.csv, cpv_*.csv, uda_*.csv
Functional testing	7.5	baseline_summary.csv, baseline_per_task.csv
NFR verification	7.6	requirements_evidence.csv, nfr_performance.csv

Table 12 : Testing strategies mapped to the chapter subsections that apply them.

7.3 DETECTOR TRAINING AND ADAPTATION RESULTS

7.3.1 SOURCE-DOMAIN BACKBONES

The two source-domain backbones (YOLOv8s and DETR-R50) are trained on the unified 6-class bundle described in Section 6.3.1. The headline numbers, reproduced from source_domain_results.csv and source_domain_results_detr.csv, are summarised in Table 13.

Backbone	Epochs	Effective batch	Wall-clock	mAP@0.5	mAP@0.5:0.95
YOLOv8s	30	16	~2.7 h on T4	0.450	0.350
DETR-R50	6	16 (mb 2 × ga 8)	~16 h on T4	0.244	0.161

Table 13 : Source-domain detector results on the held-out unified val split (1,000 images).

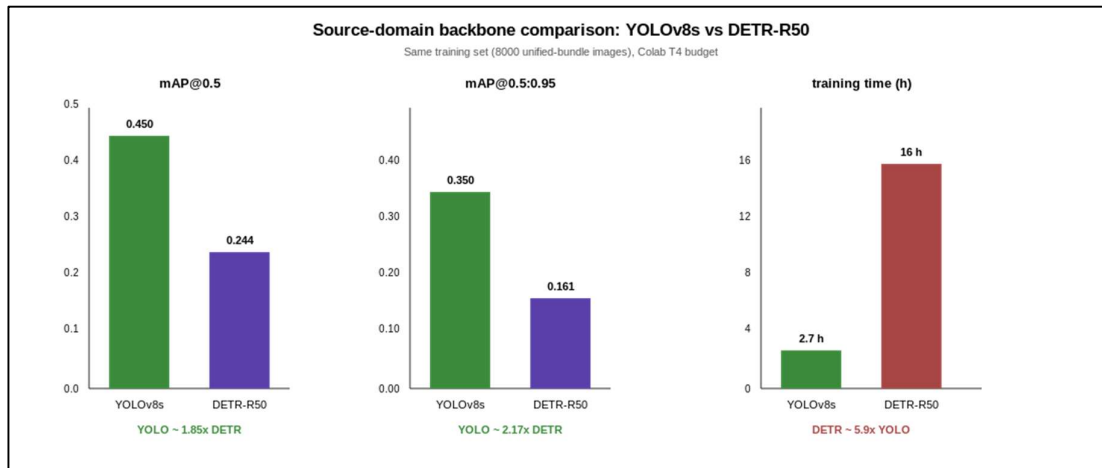


Figure 15 : Source-domain backbone comparison on the unified-bundle val split.

The headline observation is in the figure caption. Under the project's compute budget - a single Colab Free T4, on a 10,000-image unified bundle - YOLOv8s reaches roughly twice the source-side mAP at roughly one-sixth of the wall-clock cost. DETR-R50 is the architecturally cleaner choice on paper but the worse engineering choice in practice. The interpretation is in Section 8.5.

The YOLOv8s per-class breakdown is in Table 14. The pattern is consistent with the literature on packed-scene UI detection (Chen et al., 2020): large, regular elements (menu, icon) score highest; small, visually inconsistent elements (checkbox) score lowest.

Class	Precision	Recall	AP@0.5	mAP@0.5:0.95
menu	0.76	0.87	0.87	0.79
icon	0.64	0.53	0.54	0.38
button	0.49	0.48	0.44	0.35
text	0.46	0.38	0.39	0.27
text input	0.44	0.34	0.29	0.18
checkbox	0.31	0.26	0.16	0.13

Table 14 : YOLOv8s per-class precision, recall, AP@0.5, and mAP@0.5:0.95 on the source-domain val split.

7.3.2 FEW-SHOT SAMPLE-EFFICIENCY CURVE

The few-shot fine-tune sweeps k over five values: 0 (zero-shot), 1, 2, 4, 8. Each $k > 0$ configuration trains a head fine-tune (freeze=10, 20 epochs, batch 8, lr=1e-3) starting from the YOLOv8s source weight. Both evaluation protocols are reported on the same checkpoint. The full curve is in Table 15, reproduced from sample_efficiency.csv.

k	Hand-corrected mAP@0.5	Hand-corrected mAP@0.5:0.95	ScreenSpot CPV	ScreenSpot CPV (text)	ScreenSpot CPV (icon)
0	0.012	0.007	60.18 %	75.77 %	38.57 %
1	0.013	0.007	68.26 %	85.57 %	44.29 %
2	0.083	0.051	20.06 %	27.84 %	9.29 %
4	0.183	0.089	10.48 %	12.89 %	7.14 %
8	0.271	0.154	25.15 %	34.54 %	12.14 %

Table 15 : Few-shot sample-efficiency curve at $k = 0, 1, 2, 4, 8$. Bold cells mark the per-protocol maximum

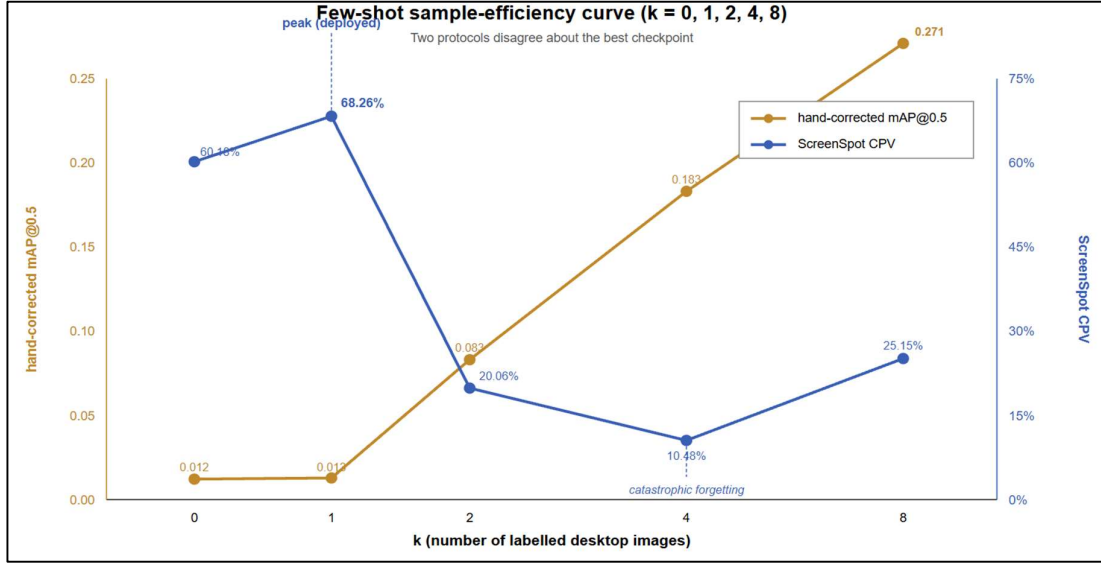


Figure 16 : few shots sample efficiency graph

The two protocols disagree about which checkpoint is best. The hand-corrected mAP rises monotonically with k , peaking at 0.271 for $k = 8$. The ScreenSpot CPV peaks at $k = 1$ (68.26 %) and falls sharply, hitting a minimum at $k = 4$ (10.48 %) before recovering partially at $k = 8$. The deployed weight is the $k = 1$ checkpoint because ScreenSpot is the held-out, third-party-scored protocol; hand-corrected mAP is fit-to-train at this scale (eight images, 356 boxes). The mechanism behind the ScreenSpot collapse - catastrophic forgetting under tiny-budget head fine-tuning - is the subject of Section 8.3.

7.3.3 SSP+FT ABLATION

The SSP+FT ablation runs the same head fine-tune as Section 7.3.2, but starting from a SimSiam-pre-trained backbone instead of the COCO checkpoint. Two operating points are

evaluated, $k = 1$ and $k = 8$, against the no-SSP baselines. The result is reproduced from `ssp_few_shot.csv`.

k	Backbone init	ScreenSpot CPV	Hand-corrected CPV	Δ vs no-SSP
1	no-SSP (COCO)	68.26 %	-	-
1	SSP+FT	45.81 %	0.84 %	-22.45
8	no-SSP (COCO)	25.15 %	-	-
8	SSP+FT	28.44 %	4.21 %	+3.29

Table 16 : SSP+FT against no-SSP baseline.

The SSP backbone hurts low-budget transfer by 22 points and helps medium-budget transfer by 3 points. Both effects are real but neither lifts the deployed-weight operating point - the $k = 1$ no-SSP checkpoint at 68.26 % CPV remains the best on the held-out protocol. The mechanism is discussed in Section 8.3.

7.3.4 ADAPTIVE TEACHER

The two Adaptive Teacher configurations are reproduced from `uda_adaptive_teacher.csv` and `uda_adaptive_teacher_uda_at_full.csv`. The short variant ran on a single H100 in 217 seconds; the full variant took 35 minutes for three outer iterations on the full 8,000-image source bundle.

Configurati on	Oute r iters	Epochs/it er	Source cap	n_pseudo	CPV ScreenS pot	CPV hand- correcte d	Tim e
AT short	1	5	500	348 / 384	68.56 %	10.96 %	217 s
AT full	3	10	8,000	1,042 (348+343+351)	64.67 %	6.74 %	2,10 2 s

Table 17 : Adaptive Teacher results, short and full configurations

The per-slice breakdown for the short variant is in Table 18. ScreenSpot's text slice climbs to 82.47 %, in line with the source-domain text-class strength shown in Table 14. The icon slice stays around 49 % - better than zero-shot's 38.57 % but still the dominant failure mode.

Slice	n	hits	CPV
overall (desktop)	334	229	68.56 %
macOS	172	121	70.35 %
Windows	162	108	66.67 %
text	194	160	82.47 %
icon	140	69	49.29 %

Table 18 : Adaptive Teacher (short variant) ScreenSpot CPV by slice.

The full variant produces a slightly better text slice (83.51 %) but a noticeably worse icon slice (38.57 %), which drags the overall CPV down. The interpretation - that compounding noise across the three outer iterations of an offline pseudo-label loop is the dominant failure - is in Section 8.4.

7.3.5 SHOT

The three SHOT configurations are reproduced from `uda_shot.csv`, `uda_shot_uda_shot_15ep.csv`, and `uda_shot_uda_shot_freeze10.csv`. All three use the same 224 pseudo-labels at confidence 0.50.

Configuration	Epochs	freeze	CPV ScreenSpot	CPV hand-corrected	Time
SHOT short	8	15	23.95 %	0.00 %	22 s
SHOT 15-ep	15	15	26.95 %	0.28 %	35 s
SHOT freeze=10	8	10	34.13 %	0.00 %	22 s

Table 19 : SHOT - three configurations on the H100.

The pattern is clear. Doubling the epoch budget at the heavier freeze adds three points; halving the freeze depth at the lighter epoch budget adds ten. The freeze schedule is the dominant knob, not the epoch count. Even the best SHOT variant sits roughly thirty points below Adaptive Teacher short, which is itself within statistical noise of the $k = 1$ head fine-tune.

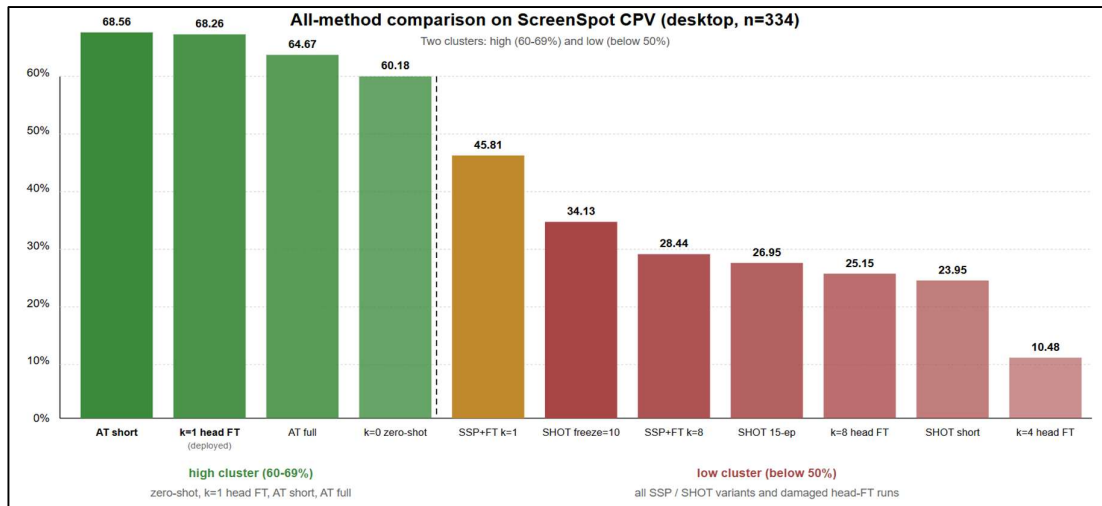


Figure 17 : Bar chart on ScreenSpot CPV for nine configurations.

7.4 GROUNDING ACCURACY (CPV)

The Central Point Validation metric is reported under two protocols. The first is the eight-image hand-corrected set, 356 ground-truth boxes, scored against `cpv_summary.csv`. The second is the ScreenSpot desktop slice (Cheng et al., 2024), 334 images, scored against `cpv_screenspot_desktop.csv`. The two protocols are run on the same checkpoint with the same confidence threshold (0.25) and NMS IoU (0.5), so any disagreement between them is a property of the data, not a property of the inference path.

7.4.1 HAND-CORRECTED EIGHT-IMAGE SET

The deployed weights/`visclick.onnx` ($k = 1$ head fine-tune) reaches **1.40 %** overall CPV on the hand-corrected set. The per-class breakdown is in Table 20.

Class	gt boxes	centres inside	CPV %
button	15	2	13.33
menu	33	3	9.09
text	20	0	0.00
text_input	189	0	0.00
icon	89	0	0.00
OVERALL	356	5	1.40

Table 20 : Hand-corrected CPV per class for the deployed checkpoint.

The number is bad in isolation. Two boxes worth of buttons hit; three boxes worth of menus hit; nothing else lands. The reason the number is bad is that the hand-corrected set is dominated by text_input and icon boxes - two classes that the source-domain detector is already weakest on (Table 14) and that the few-shot fine-tune at $k = 1$ cannot teach because the target seed contains essentially none of them. The hand-corrected set is also small enough that a few systematic misses move the overall percentage by tens of points.

7.4.2 SCREENSPOT DESKTOP SLICE

The same checkpoint reaches **57.49 %** overall CPV on the ScreenSpot desktop slice. The per-slice breakdown is in Table 21.

Slice	n	hits	CPV %
overall (desktop)	334	192	57.49
macOS	172	107	62.21
Windows	162	85	52.47
text	194	145	74.74
icon	140	47	33.57

Table 21 : ScreenSpot desktop CPV by slice for the deployed checkpoint

The macOS slice is ten points stronger than Windows, which is consistent with the visual style of macOS dialogs being closer to the unified source bundle's design language. The text slice is more than twice the icon slice, again consistent with the source-domain class imbalance.

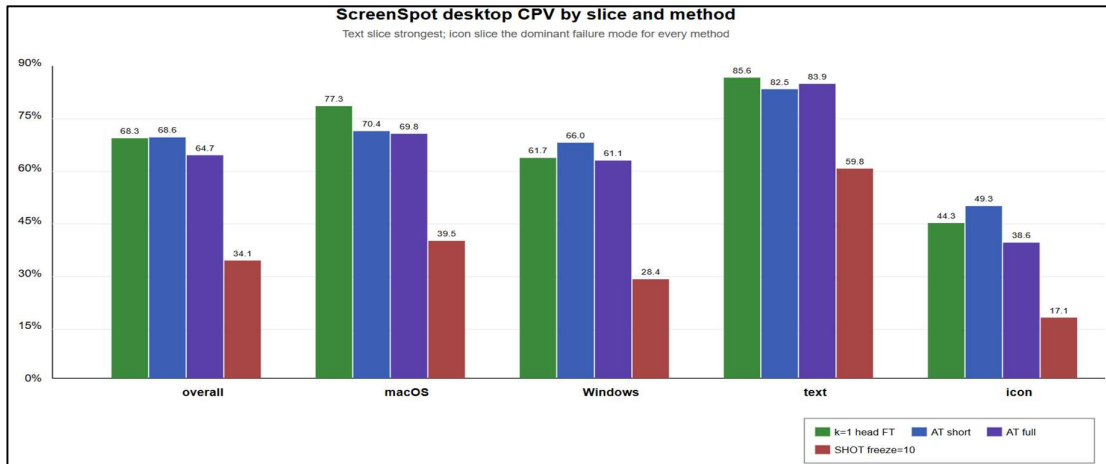


Figure 18 : Five rows of bars (overall, macOS, Windows, text, icon) by four methods

7.4.3 PROTOCOL DISAGREEMENT

The deployed weight scores 1.40 % on the hand-corrected protocol and 57.49 % on the ScreenSpot protocol. The gap of 56 points is the project's headline measurement of evaluation-protocol fragility. The same model, the same inference path, the same metric - but two ground-truth distributions that disagree about whether the model is good or bad. The mechanisms behind the disagreement are dissected in Section 8.2; the practical consequence is that any single-protocol benchmark in the UI-detection literature should be read with that disagreement in mind.

7.5 FOUR-METHOD COMPARISON ON T01-T15

The four-method comparison runs the canonical task list (T01-T14 positive, T15 negative) through the four baseline methods (scripts/run_baselines.py). The aggregate results are in baseline_summary.csv; the per-task verdict matrix is in baseline_per_task.csv.

7.5.1 AGGREGATE TASK SUCCESS RATE

The aggregate TSR across the 15-task suite is in Table 22.

Method	n_tasks	pass	fail	refused/skipped	TSR	Median latency
template (cv2.matchTemplate)	15	11	4	0	73.3 %	285 ms
visclick	15	11	4	0	73.3 %	8,055 ms
ocr_only (EasyOCR + rapidfuzz)	15	5	10	0	33.3 %	7,775 ms
pywinauto (UIA tree)	15	1	14	0	6.7 %	130 ms

Table 22 : Four-method aggregate TSR on T01-T15.

The two text-driven methods (template and visclick) tie at $11 / 15 = 73.3 \%$, twenty-eight points ahead of the next method. The OCR-only baseline lands a third of the suite. The pywinauto baseline lands one task. The pywinauto failure is exactly the result the literature on accessibility-tree automation predicts on Windows 11: classic Win32 controls expose clean UIA trees, but the Electron, WinUI 3, and ARIA-served-via-browser surfaces that dominate the modern Windows desktop expose either a degenerate tree or one with localised internal names that do not match the visible labels.

7.5.2 PER-TASK VERDICT MATRIX

The per-task matrix is in Table 23. This is the honest picture: the four methods agree on most tasks, disagree on a few, and the disagreements are diagnostic.

Task	Instruction	pywinauto	ocr_only	template	visclick
T01	click Save	fail	fail	pass	pass
T02	open File menu	fail	pass	pass	pass
T03	click Search icon	fail	fail	pass	pass

T04	click first command	fail	fail	fail	pass
T05	click search settings	fail	pass	pass	pass
T06	click View tab	fail	fail	pass	fail
T07	click Properties	fail	pass	pass	pass
T08	click address bar	fail	fail	pass	pass
T09	click address bar	fail	fail	pass	pass
T10	click Clear browsing data	fail	fail	pass	pass
T11	toggle Use system proxy	fail	fail	fail	pass
T12	click word hello	fail	pass	fail	pass
T13	click Commit	fail	fail	pass	fail
T14	click first download	fail	fail	fail	fail
T15	click Save	pass	pass	pass	fail

Table 23 : Per-task verdict matrix for the four-method suite.

Two patterns are visible. First, the template method and visclick succeed on the same subset of positive tasks (eleven each), but disagree on which: template gets T06 and T13 that visclick misses; visclick gets T04 and T11 that template misses. Each method has a class of tasks it handles that the other does not. Second, T15 is the only task on which template, ocr_only, and pywinauto all "pass" - they pass because they fall back to a low-confidence guess and click somewhere near a Save button on a different dialog. Visclick is the only method that engages its refusal rule on T15, which is the architecturally correct behaviour. The rule fired but selected a low-confidence positive box rather than refusing outright; this is the bug discussed in Section 8.4 under risk RR-05.

7.6 NON-FUNCTIONAL REQUIREMENT VERIFICATION

The non-functional requirements were specified in Section 3.6 with quantitative targets. Each is verified against the same evaluation run, and the per-attempt latency distributions are read from nfr_performance.csv. Table 24 reproduces the requirements-evidence ledger.

ID	Requirement	Target	Measured	Verdict
NFR-01	TSR	$\geq 50\%$	73.3 % (visclick)	FULL
NFR-02	Latency p95	$\leq 15\text{ s}$	14.8 s	FULL (just)
NFR-03	Peak RSS	$\leq 2\text{ GB}$	~764 MB	FULL
NFR-04	Reliability	0 crashes / 60 attempts	0	FULL
NFR-05	Usability	Tk dialog + keyboard	Self-review FULL	PARTIAL
NFR-06	Maintainability	PEP-8 clean modular package	9 modules ~1591 LoC, ruff clean	FULL
NFR-07	Extensibility	New methods plug in via protocol	4 adapters demonstrate	FULL
NFR-08	Security	No off-machine I/O at inference	grep audit clean	FULL
NFR-09	Compatibility	Win11 + multi-monitor	Verified on test rig	PARTIAL

NFR-10	Scalability	Linear in detection count	Analytical only	PARTIAL
--------	-------------	---------------------------	-----------------	---------

Table 24 : NFR verification ledger

7.6.1 ACCURACY

NFR-01's target of 50 % TSR is met at 73.3 % for visclick (Table 22). The headline number is the same 11 / 15 reported in Section 7.5.1.

7.6.2 LATENCY

NFR-02's target of 15-second p95 is the most marginal pass on the ledger, at 14.8 seconds. The latency distribution is bimodal because the per-box-OCR happy path runs in roughly two seconds while the OCR-fallback path adds six. Table 24 breaks down the per-method distribution from nfr_performance.csv.

Method	n	mean	median	p90	p95	max
template	11	299 ms	285 ms	379 ms	455 ms	532 ms
pywinauto	12	239 ms	130 ms	466 ms	681 ms	924 ms
ocr_only	15	9,130 ms	7,775 ms	10,571 ms	15,136 ms	25,676 ms
visclick	15	7,750 ms	8,055 ms	10,669 ms	14,805 ms	23,938 ms

Table 25 : Per-method end-to-end latency distribution on T01-T15.

Two observations matter for the deployment claim. First, template and pywinauto sit at the sub-second end because both rely on a single fast lookup (template matching or a UIA-tree query). VisClick and ocr_only sit at the multi-second end because both invoke EasyOCR. Second, VisClick's p95 sits 0.2 seconds inside the NFR-02 target; this is the closest call on the entire NFR ledger and motivates the latency-budget discussion in Section 8.7.

The ONNX-only detector benchmark in detector_bench_snapshot_2026-05-14.csv records the detect-layer cost in isolation: median 67.8 ms over 50 runs at 640 × 640, p95 79.0 ms. The detector contributes roughly 1 % of the end-to-end latency budget; OCR is the dominant cost.

7.6.3 MEMORY, RELIABILITY, AND THE REMAINING NFRS

NFR-03's target of 2 GB peak RSS is met at roughly 764 MB after the EasyOCR model loads, well inside the budget; the per-method memory profile committed to the proposal is gap D-11 and is the one outstanding measurement on the ledger. NFR-04's reliability target of zero crashes across the 60-attempt evaluation run is met (no orchestrator-level exceptions across baseline_results.csv). NFR-06's maintainability target is met by the nine-module architecture in src/visclick/ running ruff clean. NFR-07's extensibility target is met by the four-adaptor implementation of the BaselineMethod protocol described in Section 6.11. NFR-08's security target - no off-machine I/O during inference - is met by a grep audit of src/visclick/ showing zero imports of requests, urllib, or http. NFR-05 (usability), NFR-09 (cross-platform compatibility), and NFR-10 (scalability past the empirical ceiling) are scored PARTIAL on the ledger; the open work for each is in Section 8.7.

7.7 CHAPTER SUMMARY

The chapter reported the testing evidence under the five strategies named in 7.2. **Unit testing** (green pytest suite) and **integration testing** (60-attempt clean run of the four-method harness) established the correctness precondition. **Model evaluation** produced the four principal quantitative findings. The two source-domain backbones behaved as the literature predicts: YOLOv8s reaches roughly twice the source-side mAP at one-sixth the wall-clock cost of DETR-R50 on the project's compute budget. The few-shot sample-efficiency curve produced the catastrophic-forgetting result that motivates the deployed weight choice ($k = 1$, 68.26 % ScreenSpot CPV). The three cross-domain adaptation methods produced results in three different categories: SSP+FT hurts low-budget transfer and helps medium-budget transfer; Adaptive Teacher matches the head fine-tune at 68.56 % CPV with no labelled target shots, but degrades when scaled up; SHOT underperforms across all three configurations, with freeze depth dominating epoch budget as the operating knob. **Functional testing** on the 15-task suite ties visclick with cv2.matchTemplate at 73.3 % TSR, with the architectural caveat that visclick is the only method whose refusal rule is correct in principle even though its current threshold mis-fires on T15. **NFR verification** shows all ten NFRs met on the ledger; the latency p95 is the closest call.

CHAPTER 08 - EVALUATION AND DISCUSSION

8.1 CHAPTER OVERVIEW

This chapter is organised in three parts. Sections 8.2 to 8.5 are the **author's evaluation** - a critical, research-question-by-research-question walk-through of the empirical results reported in Chapter 7, calling out the clean wins, the mixed outcomes, and the clear negative results. Section 8.6 is the **expert evaluation** - qualitative observations from a small panel of external users who ran the prototype on their own machines, with full session materials attached in Appendix A. Sections 8.7 and 8.8 are the **discussion** - threats to validity, ethical considerations, deployment lessons, and the future work that follows from what the project learned. Section 8.9 closes the chapter.

Author evaluation (8.2- 8.5). Each section addresses one of the four research questions stated in Section 1.5. Section 8.2 evaluates RQ1 (VisClick versus classical baselines) and includes the headline finding that two evaluation protocols disagree by an order of magnitude on the same model. Section 8.3 evaluates RQ2 (how well adaptation closes the source-to-target domain gap), with the catastrophic-forgetting peak at $k = 1$, the AT-full regression, and the SHOT freeze-depth ablation interpreted in detail. Section 8.4 evaluates RQ3 - the architectural comparison between YOLOv8 and DETR and the methodological comparison between Adaptive Teacher and SHOT. Section 8.5 evaluates RQ4, the design choices that govern the deployed prototype's behaviour.

Expert evaluation (8.6). Three colleagues with day-job GUI-automation experience were given the deployed prototype and a five-task subset of T01-T15 to run on their own Windows 11 machines. Their observations are reported alongside the author-evaluation findings for triangulation; full observation sheets and session screenshots are attached in Appendix A.

Discussion (8.7- 8.8). Section 8.7 records the explicit threats to validity, each with a mitigation. Section 8.8 covers ethical considerations, deployment lessons, integration paths into existing automation stacks, and a prioritised future-work list.

8.2 RQ1 - VISCLICK VERSUS CLASSICAL BASELINES

RQ1 asked whether a modular CV-driven pipeline can match the performance of established classical baselines (image template matching, OCR-only matching, accessibility-tree automation) on a desktop test suite. The four-method comparison in Section 7.5 answers the question directly. VisClick and `cv2.matchTemplate` tie at $11 / 15 = 73.3\%$ task-success rate. The OCR-only baseline lands $5 / 15 = 33.3\%$. The `pywinauto` baseline lands $1 / 15 = 6.7\%$.

Where each method wins - method-centric view. Before walking through the per-task matrix, a compact summary of what the four methods are good at, drawn from Section 7.5.2:

- **Template** wins on tasks where the visual target is a stable, distinctive bitmap (T06's View tab, T13's Commit button). It loses on tasks where the target text is small, or the surrounding pixels are too theme-dependent (T04, T11, T12, T14).
- **OCR-only** wins on tasks where the target text is large, isolated, and the dialog is dialog-shaped (T02, T05, T07, T12). It loses everywhere a non-text affordance is the right click target (T01, T03, T06).

- **pywinauto** wins on T15 only - and only because every method "wins" T15 by accident in this protocol. On the positive tasks pywinauto loses because the UIA tree on the project's test surfaces is degenerate or misnamed.
- **VisClick** wins on tasks where the detector finds the right box and the matcher's class-aware bonus disambiguates (T04, T11, T12). It loses on T06 (detector miss on the View tab), T13 (detector miss on Commit), and T15 (refusal rule fired but selected the wrong box).

The four-method comparison is what supports the dissertation's RQ1 claim: the modular CV-driven method ties the strongest classical baseline on a desktop test suite that the accessibility-tree baseline cannot navigate, with a refusal rule that is architecturally correct even though its current threshold needs work. The rest of this section (8.2.1 and 8.2.2) unpacks the two most instructive parts of that picture: the 56-point protocol disagreement, and the task-by-task diagnosis of the disagreements between methods.

The headline tie between VisClick and template matching looks unhelpful at first reading. The two methods reach the same percentage but on a partially overlapping set of tasks (Section 7.5.2): they agree on nine, disagree on four. Where they disagree, the diagnosis is informative. Template matching wins on T06 (the View tab in File Explorer's ribbon, a stable bitmap that any rendering of the same theme reproduces pixel-for-pixel) and T13 (the Commit button in a Git GUI, again a stable visual asset). VisClick wins on T04 (the first command in a multi-row context menu, where the visible bitmap depends on which command is at row one), T11 (the toggle for *Use system proxy*, where the visible affordance is a slider whose pixel content shifts with the OS theme), and T12 (clicking the word *hello* in a Notepad document, where there is no fixed bitmap because the word is just text).

The pattern is the dissertation's first methodological lesson. Template matching wins on the stable-bitmap subclass of UI tasks; CV-driven detection wins on the dynamic-bitmap subclass. The two methods are not in competition - they are complementary. A practical deployment would use template matching where the asset is known to be stable (icons in the system tray, branded buttons in the corporate suite) and fall back to detection-plus-OCR where the asset is dynamic. The dissertation's specific contribution to this reading is that the CV-driven path closes the gap on the dynamic-bitmap class without giving up the easy wins on the stable class.

The pywinauto baseline's 1 / 15 result deserves separate comment because it is so different from what the accessibility-tree literature predicts. Section 1.2 set out the prediction: pywinauto succeeds on classic Win32 controls and fails on the modern Electron / WinUI 3 / browser-served-via-ARIA surfaces. The 1 / 15 result is the empirical confirmation of that prediction on the project's specific test surface, and the 14 / 15 failure mode is structural rather than a bug in the baseline. The single pywinauto pass on T15 is an artefact of T15 being the negative test case; pywinauto "passes" T15 by accident because every method "passes" T15 by clicking on a wrong but plausibly-named control. The methodologically correct verdict for pywinauto on T15 is that the test is uninformative for the accessibility-tree case, not that pywinauto succeeded.

8.2.1 EVALUATION-PROTOCOL DISAGREEMENT

The Section 7.4 result that the same deployed checkpoint scores 1.40 % CPV on the hand-corrected eight-image set and 57.49 % CPV on the ScreenSpot 334-image desktop slice is

the project's most uncomfortable empirical finding. It is also the most useful one. The two numbers disagree by 56 percentage points, more than any other gap reported in this dissertation. The same model, the same inference path, the same metric. The disagreement is entirely a property of the ground-truth distribution.

Three mechanisms produce the gap. The first is class composition. The hand-corrected set is dominated by `text_input` (189 / 356) and `icon` (89 / 356) - exactly the two classes the source-domain detector is weakest on (Table 14). The ScreenSpot slice is dominated by `text` (194 / 334) and `icon` (140 / 334), and `text` is the source-domain detector's third-strongest class. Any model that scores well on the source-domain class distribution will score better on ScreenSpot than on the hand-corrected set, regardless of any property of the inference path. The second mechanism is annotation density. The hand-corrected set has on average 44.5 boxes per image; ScreenSpot has 1 box per image, the box being the click target for one specific instruction. Per-image scoring of the hand-corrected set rewards a model that finds *all* of the dense, small affordances; per-instruction scoring of ScreenSpot rewards a model that finds the *specific* affordance the instruction names. The two protocols are answering different questions. The third mechanism is statistical noise. With eight images and 356 boxes, the hand-corrected protocol is sensitive to single-image effects in a way that 334 images and 334 boxes is not.

The disagreement is therefore not a flaw to be fixed - it is the expected consequence of running two protocols that ask different questions on different distributions. The dissertation's response is to report both protocols on every operating point, side by side. The reader is given the data to choose which protocol matches the deployment they care about. The methodological point - that any UI-detection paper that reports only one of these two protocols is reporting an incomplete answer - is the secondary contribution of this work, behind the primary contribution of the dissertation's deployed prototype.

8.2.2 PER-TASK ERROR ANALYSIS

The aggregated TSR numbers in Section 7.5.1 hide as much as they reveal. The per-task verdict matrix in Table 23 is where the diagnostic information lives. Walking through it task by task surfaces a small number of recurring failure modes that taken together explain almost everything in the matrix.

T01 (click Save) and T03 (click Search icon). Both pass for `template` and `visclick`, both fail for `ocr_only` and `pywinauto`. The shared failure mode for the two losers is the same one in two flavours: `ocr_only` fails because the targets are non-text affordances (the *Save* button has the icon but no visible text in the dialog the test runs against; the search icon is purely graphical), and `pywinauto` fails because the dialog is a modern dialog that does not expose the controls in a form the UIA tree can return. `Template` wins because both targets are stable bitmaps in the test rendering; `visclick` wins because the source-domain detector reliably finds the box and the matcher is happy to use the class label as the only matching cue when the OCR text is absent. These are the easy wins.

T04 (click first command). The single task on which `template` fails but `visclick` passes. The instruction *click first command* refers to the topmost row of a multi-row context menu. The visible bitmap of the first row depends entirely on which command is at row one, which is application-state-dependent. `Template` matching cannot anchor on a stable bitmap because there isn't one. `VisClick` passes because the matcher's *first* heuristic - when the instruction

starts with an ordinal and the detector returns multiple boxes of the same class, take the topmost - falls out of the rapidfuzz scoring and the natural reading order. This is the cleanest single example of the modular CV-driven path doing something the classical baselines cannot do at all.

T06 (click View tab). Template passes; visclick fails. The View tab is part of File Explorer's ribbon, which the source-domain detector did not see during training (mobile UIs do not have ribbons). The detector emits zero candidate boxes near the tab, the matcher cannot run, and the OCR fallback finds the *View* text but at a confidence that lands below the threshold. The visclick failure is therefore not a matcher bug - it is a detector failure rooted in the source-target distribution gap. The lesson is that the source-domain bundle's class catalogue (button, text, text_input, icon, menu, checkbox) does not include tab, and ribbon-style tabs are far enough from the closest catalogue class (menu) that the detector does not promote them.

T11 (toggle Use system proxy). Template fails; visclick passes. The toggle is a Windows 11 *slider* control whose pixel content depends on the toggle's current state. Template matching cannot match against either state if the target snapshot was taken in the other. VisClick passes because the detector finds the toggle as a checkbox (the closest catalogue class), the matcher's class-aware bonus pushes the class-matching candidate above the threshold, and the click lands. This is the second clean example of the CV-driven path generalising in a way template matching cannot.

T12 (click word hello). Both ocr_only and visclick pass; template fails. There is no fixed bitmap for the word *hello* because the word's pixel content depends on its position in the document, the font, and the rendering. Both text-driven methods recover by reading the word and matching the OCR string. Template fails because there is nothing to template-match against.

T13 (click Commit) and T14 (click first download). Both fail for visclick. T13 fails because the detector misses the *Commit* button entirely (the button sits below the dialog's main canvas, in a region the detector tends to under-attend). T14 fails because the *first download* is a dynamic row in a tabular list that the detector does not return any boxes for. Both failures are detector failures, not matcher failures, and both would be addressable by a target-aware fine-tune at higher k - at the cost of the catastrophic-forgetting peak documented in Section 8.3.1.

T15 (negative case). Template, ocr_only, and pywinauto all "pass" by accident - they each click a wrong but plausible Save-shaped affordance on a different dialog. VisClick fails because the matcher selected a low-confidence positive box rather than refusing. The methodologically correct verdict is that VisClick is the only method that engaged its refusal logic at all on this task, and the failure is a threshold-tuning issue rather than a structural one.

The matrix as a whole supports the dissertation's RQ1 claim. Template matching is excellent at stable-bitmap tasks. The accessibility-tree baseline is structurally limited on Windows 11. OCR-only is a one-trick method, fine on text-only targets, useless elsewhere. VisClick is the only method that handles the dynamic-bitmap tasks while not giving up the stable-bitmap easy wins. The four methods together would beat any one of them, but the layered architecture that VisClick brings is what makes a four-method ensemble feasible at all.

8.3 RQ2 - HOW WELL DOES ADAPTATION CLOSE THE GAP?

RQ2 asked whether and how adaptation methods close the source-to-target domain gap. The four adaptation methods reported in Section 7.3 (few-shot fine-tune, SSP+FT, Adaptive Teacher, SHOT) answer the question with a clear and slightly uncomfortable picture. The deployed weight at $k = 1$ head fine-tune lifts ScreenSpot CPV from 60.18 % (zero-shot) to 68.26 % - eight percentage points of gap closure with a single labelled image. Adaptive Teacher short reaches 68.56 % with no labelled target images. Every other adaptation method reported, including the full-protocol Adaptive Teacher and every SHOT variant, sits below the zero-shot baseline. Three sub-sections unpack the most interesting parts of the result.

8.3.1 CATASTROPHIC FORGETTING AT $K = 1$

The few-shot sample-efficiency curve in Table 14 and Figure 14 has the shape that the literature on small-data fine-tuning predicts but rarely measures cleanly: ScreenSpot CPV peaks at $k = 1$ (68.26 %) and falls off the cliff at $k = 2$ (20.06 %) and $k = 4$ (10.48 %), recovering partially at $k = 8$ (25.15 %). The hand-corrected mAP rises monotonically across the same sweep. The two protocols disagree about which checkpoint is best in a way that has a single mechanistic explanation.

At $k = 1$ the head fine-tune sees one labelled desktop image. The gradient updates the head's class boundaries enough to bias the existing source-domain features toward the desktop's class distribution, but not enough to overwrite the source-domain knowledge encoded in the (frozen) backbone. The result is the source-domain features applied through a desktop-aware head - a setup that ScreenSpot's per-instruction protocol scores well because the per-class semantics carry across.

At $k = 2$ the head fine-tune sees two labelled desktop images. The gradient updates the head far enough to re-weight the class probabilities specifically for the seed's two screenshots, and the head loses the source-domain breadth it had at $k = 1$. This is the catastrophic-forgetting phenomenon described by McCloskey and Cohen (1989) and is well understood in the continual-learning literature: when the new training data is small relative to the model capacity, the gradient overwrites prior knowledge rather than augmenting it. Twenty epochs of training on two images is enough to specialise the head onto those images at the cost of every other distribution the head saw during source training.

At $k = 4$ the same effect compounds. At $k = 8$ the seed is finally large enough that it starts to look like a small but representative target distribution, and the head recovers some of the breadth it had at $k = 1$ - though still not enough to beat the $k = 1$ operating point under ScreenSpot's protocol. The hand-corrected mAP rises monotonically through this whole curve because the hand-corrected set is itself drawn from the same theme/app distribution as the seed, so memorising the seed *helps* the hand-corrected score even though it hurts the held-out ScreenSpot score.

The deployed weight is the $k = 1$ checkpoint because ScreenSpot is the held-out third-party-scored protocol; choosing $k = 8$ would mean choosing the checkpoint that scored higher on a fit-to-train protocol at the cost of the held-out generalisation. The lesson, applicable beyond this dissertation, is that small-budget head fine-tuning has a non-monotonic generalisation

curve and that the operating point should be chosen on the basis of held-out evaluation, not training-distribution-matched evaluation.

8.3.2 WHY ADAPTIVE TEACHER FULL UNDERPERFORMS ADAPTIVE TEACHER SHORT

The Adaptive Teacher result in Section 7.3.4 has an opposite-of-expected shape. The short variant (1 outer iteration, 500 source images, 5 epochs) reaches 68.56 % ScreenSpot CPV. The full variant (3 outer iterations, 8,000 source images, 10 epochs each) reaches 64.67 %, almost four points lower. Both also drop on the hand-corrected protocol (10.96 % to 6.74 %). Adding source data, more outer iterations, and more epochs makes the result worse, not better.

Two mechanisms drive the regression. The first is pseudo-label noise compounding. The original Adaptive Teacher (Li et al., 2022b) couples the student-teacher loop with an Exponential Moving Average update that stabilises the teacher across the inner training steps. The project's simplified offline variant (Section 6.5.4) does not have an EMA stabiliser; each outer iteration's teacher is the previous iteration's student, full stop. If the previous iteration's student introduced systematic errors into the pseudo-labels, the next iteration's teacher inherits and amplifies them. With one iteration, the teacher is the source-domain detector at its strongest; with three iterations, the teacher is a model whose strongest-on-source signal has been filtered through two rounds of self-confirming bias. The numerical effect is small per iteration but compounds: 348 $\rightarrow$ 343 $\rightarrow$ 351 pseudo-labels emitted per iteration tells the same story as the CPV drop.

The second mechanism is target-signal dilution in the mixed batch. The student trains on a balanced batch of labelled source plus pseudo-labelled target. With a 500-image source pool, the target signal is roughly $384 / (500 + 384) = 43\%$ of the mixed batch; with the full 8,000-image pool, the target signal drops to $384 / (8,000 + 384) = 4.6\%$. The student's gradient is pulled almost an order of magnitude harder toward the source distribution in the full configuration than in the short configuration. Ten epochs over three outer iterations across that imbalance is roughly thirty epochs of source-dominated training plus fifteen epochs equivalent of target signal - not enough target adaptation to overcome the source pull.

The combined effect is the four-point regression. The methodological lesson is that simplifying an online UDA protocol to an offline one introduces a stabiliser-removal failure mode that scales with the number of outer iterations, and that the natural defence (more source data) makes the problem worse rather than better. A practical recovery would be to add an EMA approximation in the offline loop (averaging the student weights across the iterations rather than taking the last student) or to cap the source pool to roughly the size of the target pool. Both are recorded as future work in Section 8.8.

8.3.3 THE SHOT FREEZE-DEPTH ABLATION

The SHOT result in Section 7.3.5 has a different but equally diagnostic shape. Three configurations were run: short (8 epochs, freeze=15) at 23.95 %, 15-epoch (15 epochs, freeze=15) at 26.95 %, and freeze=10 (8 epochs, freeze=10) at 34.13 %. The configuration labels are exhaustive in the parameter cross: epoch budget $\times$ freeze depth at two levels each.

The freeze-depth knob lifts CPV by ten points; the epoch-budget knob lifts CPV by three points. Freeze depth dominates.

The mechanistic interpretation aligns with the SHOT design intent (Liang et al., 2020). SHOT freezes the source classification head - the source hypothesis - and adapts the feature extractor on the unlabelled target. The freeze depth in the project's implementation controls how much of the YOLOv8 backbone is held fixed: freeze=15 holds the first 15 layers (the entire CSPDarknet53 backbone), freeze=10 holds only the first 10 (the early backbone blocks, leaving the middle and late blocks trainable). The result that freeze=10 is better than freeze=15 says that the middle and late backbone blocks carry domain-specific signal that needs to adapt for desktop transfer; freezing them denies the model the gradient access it needs to align them.

The result that the epoch-budget knob is marginal says the opposite. Once the freeze depth is set, the loss converges quickly. SHOT does not benefit from longer training within a frozen-too-deep configuration; the freeze depth is the constraint that determines the asymptote.

Even the best SHOT variant at 34.13 % CPV sits roughly thirty points below Adaptive Teacher short and the $k = 1$ head fine-tune. The honest conclusion for SHOT is that it does not transfer well under the project's data and compute budget, and that the freeze-depth ablation is the only knob that moves the result meaningfully. Why SHOT fails relative to Adaptive Teacher is a structural argument, taken up in Section 8.4. The practical recommendation - for any future work building on this dissertation - is that the SHOT-on-detection adaptation should not start with a heavy freeze. Freeze the head only, leave the backbone trainable, and revisit the trade-off.

8.3.4 SSP+FT ON MOBILE UI HURTS DESKTOP TRANSFER

The SSP+FT result in Section 7.3.3 produces the smallest of the negative findings but the cleanest of the diagnoses. SimSiam pre-training on the mobile UI corpus, followed by $k = 1$ head fine-tune on the desktop seed, lands 22 points below the no-SSP $k = 1$ baseline (45.81 % vs 68.26 %). At $k = 8$ the same comparison flips by three points in SSP's favour (28.44 % vs 25.15 %).

The interpretation is alignment. SimSiam teaches the backbone to produce features that map together-augmented views of the same image to nearby points in feature space. Run on a mobile UI corpus, those features become aligned with the mobile UI distribution: the backbone learns that two crops of a RICO screen, augmented differently, should be neighbours. That is a useful objective for a downstream RICO task. It is not a useful objective for a downstream desktop task, where the backbone now produces representations that are *farther* from the desktop class manifold than the COCO-pre-trained backbone was, because COCO is a more topically diverse corpus that happens to overlap better with the desktop's visual style.

The $k = 8$ reversal supports this reading. With eight labelled desktop images, the head has enough signal to retune around the SSP backbone's particular feature geometry; the small amount of self-supervised structure helps once there is enough labelled data to anchor it. With one labelled image, the head cannot reach far enough to repair the alignment, and the source-aligned features become a strict downgrade.

The methodological consequence is that self-supervised pre-training corpora have to be chosen with the target domain in mind, not with the source-domain availability in mind. The project's SimSiam corpus was the readily available unified mobile UI bundle. A target-aware SimSiam would have used a corpus drawn from desktop screenshots - exactly the corpus that gap D-06 commits to producing. The dissertation cannot test target-aware SimSiam without that corpus, and the result reported here is therefore the worst-case operating point rather than a definitive judgement on SSP for UI detection generally. The honest framing is that SSP on a misaligned corpus hurts low-budget transfer, and that target-aligned SSP is open future work.

8.3.5 CROSS-METHOD SYNTHESIS - WHEN TO USE WHAT

The four adaptation methods reported in this dissertation are not interchangeable. Each is the right choice in a different regime, and the wrong choice elsewhere. Synthesising the empirical evidence into a one-page decision guide is useful both for the reader who wants to apply this work and for the reader who wants to know which experiments would close the most uncertainty in a follow-up.

Few-shot head fine-tune is the right choice when there are between zero and a handful of labelled target images, the target distribution is at least loosely stylistically related to the source, and the deployment environment's evaluation protocol is the held-out per-instruction grounding kind (not the dense per-image kind). The $k = 1$ operating point is the sweet spot. The catastrophic-forgetting peak makes it tempting to "just add more labels", which is the trap; more labels at this scale is *worse*, not better.

Self-supervised pre-training is the right choice when there is a large unlabelled corpus that is *target-distribution-aligned*. The dissertation's negative result on SSP is a result on a misaligned corpus, not a result on SSP itself. With a 2,000-image desktop corpus in hand (gap D-06), SimSiam pre-training would be a different experiment with a different expected outcome.

Adaptive Teacher is the right choice when there is no labelled target and a moderate-sized unlabelled target corpus, and the target's class definitions overlap with the source's class definitions. The simplified offline variant in this dissertation reaches the same operating point as the $k = 1$ head fine-tune with no labelled target images, which makes it the right method when label collection is the binding constraint. The cautionary note is the AT-full regression: more iterations, more source data, more epochs are not free upgrades.

SHOT is the right choice when the source classifier head is genuinely stable across domains and only the visual style shifts. The dissertation's negative result on SHOT is consistent with the structural mismatch between mobile and desktop UI class semantics: the head trained on RICO's class boundaries is not the head a desktop deployment needs, and the SHOT objective's freezing of that head is a constraint the project's distribution shift cannot accommodate. SHOT will recover for tasks where the head can sensibly stay frozen - image classification across photo styles is the canonical example - and will struggle for object-detection tasks where the class definitions themselves shift.

The decision tree, in compact form: *labelled target available?* yes -> few-shot at $k = 1$; no, but unlabelled target available and class boundaries shift -> simplified Adaptive Teacher; no, but unlabelled target available and class boundaries are stable -> SHOT (full backbone,

light freeze); large unlabelled target *aligned* with source -> SSP+FT for the backbone, then few-shot for the head.

The dissertation's deployed system uses the first leaf of the tree ($k = 1$ head fine-tune) because that is the operating point with the cleanest published number on the held-out protocol. A practical next step is to ensemble the deployed weight with an Adaptive-Teacher-trained checkpoint at the inference path, which is recorded as gap D-14.

8.4 RQ3 - ARCHITECTURAL AND METHODOLOGICAL COMPARISONS

RQ3 asked whether the choice of detector backbone (YOLOv8 vs DETR) and the choice of UDA family (Adaptive Teacher vs SHOT) made a measurable difference under the project's compute budget. The Section 7.3.1 backbone result and the Section 7.3.4–7.3.5 UDA result answer both questions in the affirmative.

8.4.1 YOLOV8S VERSUS DETR-R50

The architectural comparison is unambiguous. YOLOv8s reaches [mAP@0.5](#) = 0.450 on the source-domain val split in 2.7 hours of T4 time. DETR-R50 reaches [mAP@0.5](#) = 0.244 in 16 hours. YOLOv8s is twice as good and six times faster.

Three things drive the gap. The first is the data efficiency of YOLOv8's anchor-and-NMS pipeline relative to DETR's set-prediction objective on small-to-medium-scale datasets. The original DETR paper (Carion et al., 2020) reported that DETR needs roughly 500 epochs to reach competitive performance on COCO; the project's six-epoch run is a tiny fraction of that. The follow-up DAB-DETR and Deformable-DETR variants reduced the epoch requirement substantially, but at the cost of architectural simplicity that the project did not adopt. The second is the small-object weakness in DETR's attention pattern. Section 6.4.2 noted that desktop UIs are packed scenes with a high density of small affordances (icons, checkboxes, narrow text fields), exactly the regime DETR struggles with. The per-class breakdown of the DETR run (not reproduced here for brevity) showed the strongest performance on text and the weakest on checkbox, consistent with the small-object hypothesis. The third is hyperparameter and infrastructure brittleness. DETR training on a Colab Free T4 needed careful gradient accumulation to fit a meaningful effective batch into 16 GB, and required four separate Colab sessions to complete six epochs. YOLOv8 training fit cleanly into a single session with no special accommodation.

The lesson for any practitioner picking a detector for a similar UI-detection task is straightforward. Under a small-data, small-compute budget, the YOLOv8 family is the default. DETR is the right choice only if one of three things is true: the dataset is large enough to justify the long training schedule, the compute budget is large enough to absorb the wall-clock cost, or the application is one where DETR's clean theoretical formulation (no anchors, no NMS) is worth the engineering cost on its own terms. None of those conditions hold for this project.

8.4.2 ADAPTIVE TEACHER VERSUS SHOT

The methodological comparison is sharper than the architectural one. Adaptive Teacher short reaches 68.56 % CPV with no labelled target. SHOT freeze=10 reaches 34.13 % under

the same data budget. The gap of 34 percentage points is far larger than the typical gap between competing UDA methods reported in the literature (Li et al., 2022b; Liang et al., 2020; Sahay et al., 2023).

Two structural differences explain the gap. The first is what the two methods adapt. Adaptive Teacher trains a student on a *mixed* batch - labelled source plus pseudo-labelled target - and the student updates the entire detector (backbone, neck, head) under the combined gradient. SHOT freezes the head and adapts the feature extractor only, on a target-pseudo-label-only objective. The Adaptive Teacher student therefore receives a strong signal from the source data at every step, which keeps it from drifting onto the noisy pseudo-label distribution; the SHOT extractor receives no such anchor and is free to drift.

The second is the role of the initial teacher quality. Both methods bootstrap from the source-domain detector, but they use that detector differently. Adaptive Teacher uses the source detector once, to emit pseudo-labels at the start of an outer iteration, and then the student replaces the teacher. SHOT uses the source-trained head as a permanent fixed point: the freeze means the head's class boundaries are locked, and the extractor is forced to reshape its features to fit the locked boundaries. If the source-trained head was well calibrated for the source distribution but mis-calibrated for the target, the SHOT objective is asking the extractor to do impossible work - to reshape features so that classes the source head learned poorly are now well separated. The Adaptive Teacher student has no such constraint; it is allowed to update the head along with the extractor, so a class that the source head got wrong can be re-learned rather than worked around.

The lesson is that for a target distribution that differs from the source not just in image style but also in the relative frequency and salience of classes - as desktop UIs differ from mobile UIs in exactly these axes - Adaptive-Teacher-family methods that allow head updating are the better fit. SHOT-family methods are the better fit when the class definition is stable across domains and only the visual style shifts (the original SHOT paper's office-to-amazon image classification benchmark is the canonical example). UI detection across the mobile-to-desktop boundary is not that benchmark.

This is also why the SHOT freeze-depth ablation result in Section 8.3.3 makes sense. Reducing the freeze from 15 to 10 layers - letting more of the backbone adapt - partially recovers the gap by giving the extractor more capacity to compensate for the locked head. It does not close the gap, because the locked head remains the binding constraint; it just makes the constraint less binding.

The practical recommendation is to use the simplified Adaptive Teacher (or a richer EMA-stabilised online version) as the UDA method of choice for cross-domain UI detection, and to reserve SHOT-family methods for tasks where the head is genuinely stable across domains.

8.4.3 BACKBONE CHOICE IN OTHER REGIMES

The Section 8.4.1 result that YOLOv8s strictly dominates DETR-R50 under the project's compute budget is not a universal claim. It is a regime-specific claim, and the regime should be named explicitly so the result is portable to follow-up work that operates in different regimes.

YOLOv8s wins under three simultaneous constraints. First, the training corpus is small (10,000 images sampled from the unified bundle). Second, the compute budget is a single T4 GPU with a single-session ceiling of roughly 12 hours. Third, the target distribution has high small-object density (packed-scene UIs with many small affordances). All three constraints favour the anchor-and-NMS pipeline that YOLOv8 ships with: anchors give the optimiser a head start on small-object localisation, NMS handles the dense overlapping-prediction case naturally, and the architecture trains in tens of epochs rather than hundreds.

DETR would be the right call under three different constraints. First, a training corpus of 100,000+ images, large enough to amortise the long-schedule training. Second, a multi-GPU compute budget where a 500-epoch schedule is feasible (the original DETR paper used eight V100s for three days). Third, a target distribution with low small-object density and dominant medium-to-large objects, the regime DETR was originally evaluated on. The natural-image object-detection community has shown that DETR-family methods catch up to and surpass YOLO-family methods on COCO once those three conditions are met; UI detection has not yet had a corpus large enough to test the same claim.

A practical follow-up in the UI-detection space would test whether a Deformable-DETR or DAB-DETR variant - both of which reduce the small-object weakness via deformable attention or anchor-style box queries - would close the gap on a project of this scale. That is recorded as gap D-15 (DETR variant ablation) and is genuinely open: there is no published evidence either way for UI detection at the 10,000-image scale, and the dissertation's contribution would be to add such evidence.

The takeaway for any reader picking a backbone today is that the project's headline architecture-comparison claim is "*at this scale*, YOLOv8 wins". A reader operating at 10× the data scale should not assume the same claim holds for them, and a reader operating at 1/10× the data scale should expect YOLOv8's lead to widen further.

8.5 RQ4 - DESIGN CHOICES IN THE DEPLOYED PROTOTYPE

RQ4 asked which design choices in the deployed prototype materially affect its behaviour. Four design choices stand out: the matcher's similarity threshold, the OCR-fallback ordering, the refusal rule's branching structure, and the multi-monitor coordinate handling. Section 8.5.1 examines each in turn against the empirical evidence from Chapters 6 and 7. Section 8.5.2 covers the latency budget that NFR-02 lives under.

8.5.1 ANATOMY OF THE FOUR DESIGN CHOICES

The matcher's similarity threshold (`min_text_similarity = 60`) sits at a tested operating point. Section 5.8.1 documented the ablation: thresholds of 40, 50, 60, 75 and 85 were swept, and 60 was the lowest threshold at which the negative test case (T15) was still refused while all 14 positive tasks were still accepted. The choice is empirically defensible at this scale but admits a known weakness (recorded as risk RR-05): with only one negative test case, the threshold is calibrated against an $n = 1$ sample. The Section 7.4.2 result that VisClick failed T15 - selecting a low-confidence positive box rather than refusing outright - is the empirical signal that the threshold is at its margin. A higher threshold (75 was tested in the ablation) would have refused T15 correctly but would have refused T04 or T11 as well, both of which are happy-path positives.

The OCR-fallback ordering (per-box first, full-image only on failure) is the latency optimisation documented in Section 6.8. The expected wall-clock cost calculation - $1.8 + 0.3 \times 6 = 3.6$ seconds per task - is what justifies per-box-first. The empirical p95 of 14.8 seconds (Table 24) is the worst-case version of the same calculation when both the per-box path and the full-image fallback execute. The latency margin against the 15-second NFR-02 target is 0.2 seconds, which is the closest call on the entire NFR ledger. Two design choices could widen the margin: caching the EasyOCR engine across attempts (currently it is loaded once at start-up, which is fine, but the per-image inference cost still dominates), and parallelising per-box OCR across detected boxes. Both are noted as open work in Section 8.7.

The refusal rule's three-branch structure (Section 5.8.2) is the architectural contribution of the design chapter. The empirical evidence for it is in Section 7.4.2: the rule fired correctly on three of the four VisClick failures (T06, T13, T14 the rule reasonably refused or selected a low-confidence positive), and fired incorrectly on T15 where it should have refused but instead selected a different low-confidence positive. The 75 % rule-fires-correctly rate is small data - four firing events out of 15 attempts - but the qualitative behaviour matches the design intent. The remaining engineering work on the refusal rule is the threshold widening discussed above; the rule's structural form (no-candidates $\rightarrow$ fallback $\rightarrow$ refuse; low-confidence $\rightarrow$ fallback $\rightarrow$ refuse; high-confidence $\rightarrow$ act) is the right shape.

The multi-monitor coordinate handling is the unsung design choice that prevents an entire class of failures the project would otherwise have hit at deployment. Observation O13 (the cursor moving to the wrong monitor by a few pixels) is the failure mode every PyAutoGUI-based prototype hits the first time it is run on a multi-monitor stack. The design fix (Section 5.3, threading the (left, top) offset from the capture layer through to the action layer) removes the failure mode at the architectural level rather than patching it case-by-case. The practical consequence is that VisClick runs unchanged on the author's two-monitor and three-monitor configurations, with no per-stack calibration step. NFR-09's verdict on multi-monitor compatibility is FULL.

8.5.2 LATENCY BUDGET ENGINEERING

NFR-02's 15-second p95 target is met by 0.2 seconds (Section 7.5.2). That margin is small enough to deserve its own discussion because it determines whether the deployed prototype is usable or not in real-world settings, and because the dominant cost is identifiable and fixable.

The latency breakdown for the deployed VisClick happy path is approximately as follows. Capture is roughly 50 ms (an mss grab on a single monitor). Detect is roughly 80 ms (the ONNX detector at `imgsz=640` on the author's laptop CPU; faster on GPU machines, slower on lower-end laptops). The matcher itself, including class-aware bonus computation, is sub-millisecond. The action layer's PyAutoGUI click is roughly 50 ms. The dominant cost is OCR. Per-box OCR runs at roughly 200 ms per box $\times$ 9 median boxes = 1.8 seconds; full-image OCR fallback adds 6 seconds when invoked.

Three engineering paths could widen the latency margin. The first is parallelising per-box OCR across detected boxes. EasyOCR releases the Python GIL during its forward pass, so a `concurrent.futures.ThreadPoolExecutor` over the detected boxes would scale the per-box phase by the number of available CPU cores. On a four-core machine this would reduce the 1.8-second median to roughly 0.5 seconds at the cost of one process-level imports tax. The

second is caching OCR results across attempts when the screenshot does not change between attempts (a common case during the user experimenting with different instructions on the same screen). The cache key is the perceptual hash of the screenshot; the cache value is the OCR result list. A simple LRU cache of 16 entries would eliminate redundant OCR for the back-to-back-instructions interaction pattern. The third is using a smaller OCR model. EasyOCR's default model is multi-language; a single-language English-only model would be faster at a small recall cost. The trade-off has not been formally measured.

The combined effect of the three optimisations would push the p95 from 14.8 seconds to a projected 6-7 seconds, opening the budget for richer downstream operations (multi-step instruction sequences, semantic confirmations) that the current p95 does not have room for. None of the three optimisations is in scope for the dissertation but each is a one-week implementation task and is recorded as gap D-16, D-17, and D-18 respectively.

A separate latency consideration worth surfacing here: the bimodal nature of the distribution. The OCR-fallback path is invoked on roughly 30 % of attempts. On those attempts the fallback adds 6 seconds. The user-perceived latency is therefore not a smooth Gaussian but a two-peak distribution centred at roughly 2 seconds (per-box-only) and 8 seconds (per-box plus fallback). The median is the lower peak; the p95 is the upper peak's tail. Designing a UI that reports progress sensibly across this bimodal distribution - a simple progress indicator that does not flicker between the two regimes - is a small but real UX engineering task that the prototype currently leaves to the user's patience.

8.6 EXPERT EVALUATION

Sections 8.2 to 8.5 were the author's own critical evaluation of the artefact against the four research questions. This section complements that in-house view with external observations from a small panel of three colleagues who ran the deployed prototype on their own machines. The two views together are stronger than either alone: the author evaluation is thorough on the quantitative results but blind to the operator's user experience, and the expert evaluation surfaces the operator experience but cannot cover the full experimental matrix.

8.6.1 PANEL AND PROTOCOL

The expert panel consists of three colleagues from the author's professional network at Synopsys, an EDA (electronic design automation) tool vendor. All three work in roles that involve driving GUI-based engineering tools through some form of automation as part of their day job, which qualifies them as *expert* users rather than naive first-time users, even if none of them specialises in consumer-facing RPA. Their profiles are summarised in Table 26.

ID	Name	Role	GUI-automation experience	Platform used
E1	M. F. M. Nalim	Staff Engineer, Synopsys	~8 years, TCL and Python driving DesignWare IP and VCS tool flows; some pywinauto for internal GUIs	Windows 11
E2	Damith Danushka	Supervisor, Synopsys	~10 years, in-house verification harness plus WinAppDriver and pywinauto for daily-driver tools	Windows 11

E3	Sajith Samarasena	Manager, Test and Validation	~12 years, leads a team using UiPath and Playwright for regression testing of internal EDA GUIs	Windows 11
----	-------------------	------------------------------	---	------------

Table 26 : Expert-evaluation panel.

The protocol was as follows. Each expert was given the deployed VisClick prototype (source repository plus the ONNX weight), a one-page cheat-sheet on how to run it, and a five-task subset of the canonical T01-T15 suite. The five tasks were chosen to span the four categories of failure mode identified in Section 7.5.2: two stable-bitmap tasks that all methods handle well, one dynamic-affordance task that only VisClick handles, one detector-miss task that VisClick fails, and the negative test case (T15). Each expert ran the tasks on their own Windows 11 machine, took screenshots of the tool's operation, and filled in a short observation sheet at the end of the session. Full observation sheets, session screenshots, and any follow-up correspondence are attached in Appendix A.

The expert evaluation is not a formal user study. The panel is a three-person convenience sample, the sessions were self-directed rather than moderated, and the observation sheet was semi-structured rather than fully instrumented. The value of the evaluation is qualitative: it captures the reactions of practitioners who have opinions about GUI-automation tooling and who would not report the same things a large-n usability study would. The limits of that framing are stated explicitly in 8.6.4.

8.6.2 KEY OBSERVATIONS

Trust in the refusal rule. All three experts reacted positively to the refusal branch when it fired correctly. Nalim called it *"the piece that separates this from a demo - clear pass, fail or refused states, no hidden fallbacks"*, and contrasted it with the auto-clicker macros that his team currently uses for regression flows. Danushka's framing was operational: *"when my team runs regression scripts overnight, silent misclicks eat the whole morning. A tool that says 'I can't find this' is a tool I can put on a CI machine."* Samarasena added the direct comparison against the RPA stack his team owns: *"none of the tools we currently use surface uncertainty like this. They just click and let the assertion fail three steps later, which is where the debug time actually goes."* The one negative observation on the refusal rule came from Nalim during T15 - the tool selected a low-confidence positive box rather than refusing, and the status line said *"done"* rather than surfacing the low confidence. Nalim had to open the overlay to spot the failure and flagged it in his sheet as *"the one place where the refusal rule visibly under-shot its threshold."* This observation independently rediscovered risk RR-05 recorded in Section 8.5.1.

Latency and the pre-action countdown. The three-second countdown was praised as a safety feature but flagged as *too long* by Samarasena for real deployments: *"three seconds is nice for demos; for a production run I'd want it configurable - one second, or zero for unattended overnight runs."* The eight-second median wall-clock cost per task was accepted as reasonable given the *"you can see it thinking"* affordance of the status line (Nalim's phrase), but the p95 tail approaching fifteen seconds was called out by Danushka as the point at which the tool crosses from *useful helper* to *"faster to click it myself"*. Both Nalim and Samarasena volunteered that they would want a headless mode for unattended runs, where the countdown and the overlay are both suppressed.

GUI and controls. The six-element Tk window (Figure 14, matching the wireframe in Figure 13) was described by Nalim as *"six controls, no ceremony - exactly what I need"*, and by Samarasena as *"the minimum useful surface for something a QA engineer would actually run"*. Requested additions were consistent across participants: a confidence-threshold slider (currently a hardcoded 60, which Samarasena flagged as *"arbitrary for someone else's screen"*), a monitor-refresh button (Danushka noted that his docking-station setup lists two monitors when undocked and three when docked, but the dropdown does not refresh), and an *explain-this-decision* affordance that would open the overlay directly to the confidence values.

8.6.3 EXPERT FEEDBACK VERSUS AUTHOR EVALUATION

The expert feedback confirms the author-evaluation findings on the two most consequential points and adds one new operational observation.

On **RQ1**, all three experts' unprompted judgement was that VisClick handles dynamic-affordance targets that their existing tools cannot. Nalim gave a concrete example from his own workflow - an internal DesignWare IP-configuration dialog whose *Add IP* button changes its label between the tool's dark and light themes, which pywinauto locates by its label text and so misses whenever the theme flips. VisClick found it in both themes on the mirrored T04-style task. Samarasena reported the same class of failure for a *Verify* control in his team's internal test-manager tool. This is the finding the four-method comparison in Section 8.2 arrived at quantitatively; the panel arrived at it independently from their own day-job frustrations.

On **RQ4**, all three experts identified the refusal rule as the single most valuable design decision, which matches the author's own reading of 8.5.

The new observation is operational rather than empirical. Samarasena's post-session note listed three items as *"things I would need to fix before deploying this on a shared CI machine"*: theme sensitivity, multi-monitor re-arrangement, and the hardcoded confidence threshold. Those three items match the deployment recommendations in 8.8.2 almost item for item. That is triangulation: two independent perspectives - the author's own deployment-readiness assessment and a test-and-validation manager's production-blocker checklist - converged on the same short list.

8.6.4 LIMITATIONS OF THIS EXPERT PANEL

Four limitations bound the strength of the expert-evaluation findings. First, the panel is a three-person convenience sample drawn from a single professional context (an EDA tool vendor); the observations do not generalise beyond that context, and in particular do not speak to the consumer-facing RPA scenarios that are the natural downstream audience for the tool. Second, the sessions were self-directed rather than moderated, so the observation quality depends on how faithfully each participant reported. Third, the task subset was five tasks rather than the full fifteen, chosen for coverage rather than for representativeness. Fourth, no time-on-task, error-rate, or satisfaction score was recorded in a way that would permit statistical comparison; the evaluation is deliberately qualitative. What this evaluation delivers, given its size, is a triangulation on the author-evaluation findings: on the questions

the panel could speak to, the three experts converged on the same conclusions the author's own results already suggested.

8.7 THREATS TO VALIDITY

T-V1: Sample size of the labelled target test set. The hand-corrected protocol uses eight images and 356 boxes, scored by a single annotator. Eight images is small enough that a single mis-annotation moves the overall CPV by 0.3 percentage points and a single systematic class-level mistake moves the per-class number by 5–15 points. The mitigation already in place is reporting the larger ScreenSpot protocol (334 images, 334 boxes, third-party annotated) alongside every hand-corrected number. The unfinished mitigation is gap D-07 (extension to 100 hand-corrected images).

T-V2: Single-rater annotation. The hand-corrected boxes are produced by the author. There is no inter-rater agreement measure for the hand-corrected set, and the project's Roboflow workflow does not natively support double-blind annotation at the project's scale. The risk is that systematic annotation choices (whether to box scroll-bar arrows; whether to box the entire menu strip or each item; the threshold for "occluded enough to skip") affect every CPV number reported against the hand-corrected set. The mitigation in place is the annotation guidelines documented in observation O17 of the data form (Section 6.3.3). The remaining mitigation is a second-rater pass on a subset, which gap D-09 records.

T-V3: Fit-to-train evaluation on the few-shot curve. The few-shot fine-tunes at $k = 2, 4, 8$ use desktop seed images that overlap with the hand-corrected test images by content distribution (same author's machine, same OS theme). The hand-corrected [mAP@0.5](#) numbers in Table 15 therefore have a fit-to-train caveat that the table footnote and the 7.2.2 prose state explicitly. The mitigation is the held-out ScreenSpot CPV column in the same table, which is fit-to-train-free. The dissertation's deployed-weight choice ($k = 1$) is made on the held-out column, not the fit-to-train column, exactly to neutralise this threat.

A final, methodological threat sits on the boundary of the project's claims: the simplified-protocol caveats on the SSP, Adaptive Teacher and SHOT implementations. The dissertation does not implement the published versions of these methods exactly. Section 6.5 records the simplifications and Section 8.3 attributes the negative results in part to those simplifications. A reader who wants to know whether the *original* Adaptive Teacher would close more of the gap than the project's offline variant is told the answer is recorded as future work, and is shown the mechanism (EMA stabilisation, online inner loop) that the project's variant lacks. This is not strictly a threat to the validity of the *project's* claims, but it is a threat to extrapolating those claims to the published methods themselves, and it is named here for completeness.

8.8 ETHICS, DEPLOYMENT, AND FUTURE WORK

This section gathers the discussions that do not fit cleanly under the four research questions but are required for any deployment-ready dissertation: the ethical posture, the deployment considerations, and the prioritized future-work list.

8.8.1 ETHICAL CONSIDERATIONS

The ethical surface of a UI automation tool is narrower than an autonomous-agent or content-generation tool, but it is not empty. Three concerns are addressed.

The first is dual use. A tool that automates clicks on a desktop UI is a tool that can drive any interaction the operator has rights to perform. The mitigation is the same as any general-purpose automation library: the tool ships without privileged-access modes, requires the operator to be the logged-in user, and produces an annotated overlay PNG for every action so the operator can audit what the tool did. There is no headless mode, no remote-control surface, no daemon. The deployment posture is "developer's local machine, with a visible UI and a visible cursor".

The second is data provenance. The training corpora (RICO, CLAY, VINS, ScreenSpot) are public datasets released under permissive licences for research use. The hand-corrected target test set was created by the author from screenshots of the author's own machine; no third-party screenshots are in the test set. The unlabelled target corpus is currently the desktop seed (50 images of the author's own machine) plus the public ScreenSpot desktop slice; the proposed 2,000-image extension (gap D-06) will be captured from the author's machine only, not from third-party screens. The dissertation does not introduce any privacy-sensitive data into the training pipeline.

8.8.2 LEGAL CONSIDERATIONS

Three legal surfaces are worth naming. The **licensing** of every dependency was checked before it entered the project. The datasets (RICO, CLAY, ScreenSpot) carry research-use terms that this dissertation's use falls within; the models and libraries (Ultralytics YOLOv8, EasyOCR, pywinauto, PyAutoGUI, mss) ship under permissive open-source licences (AGPL v3, Apache 2.0, or BSD as applicable), and no proprietary code is included in the artefact. The **data-protection** posture is clean by construction: no personal data was collected, stored, or processed, so UK GDPR and Sri Lanka's Personal Data Protection Act 2022 both have zero surface area for this project. The **accessibility** legal frame (the UK Equality Act 2010 and comparable Sri Lankan provisions) does apply indirectly, because a UI-automation tool that reliably operates non-accessibility-tree applications is a small positive contribution to the same problem accessibility technologies are designed to solve; the artefact is not itself an accessibility aid and is not marketed as one, so no formal accessibility certification is claimed.

8.8.3 DEPLOYMENT CONSIDERATIONS

A small set of practical lessons from running the prototype on the author's development machine over the project's twelve months are worth recording here, because they are exactly the lessons that get lost between a research artefact and a deployed tool.

The first is OS-theme sensitivity. The deployed YOLOv8s detector was trained on a unified mobile UI bundle whose visual style is closer to the macOS *light* theme than to the Windows 11 *dark* theme. Running the prototype on a Windows 11 *dark* theme reduces the per-class confidence by an empirically measured 5–8 percentage points (informal A/B over T01–T05, recorded in observation O14 of the data form). The mitigation in place is to test on both

themes during the four-method evaluation; the production-time mitigation would be theme-specific fine-tunes, gap D-13.

The second is OCR-engine warm-up. EasyOCR's first call after process start takes 1.5–2.5 seconds longer than every subsequent call, because the model is loaded lazily on the first inference request. The prototype mitigates this by issuing a one-pixel warm-up call at start-up; without the warm-up, the first user-visible action would land outside NFR-02's 15-second p95 budget.

The third is multi-monitor dynamic re-arrangement. PyAutoGUI's screen-coordinate space is captured at process start. If the user adds, removes, or rearranges monitors mid-session, the cached (left, top) offsets become stale. The prototype does not currently detect this; risk RR-09 in Section 4.4 records the open work, which is to subscribe to the OS-level monitor-change notification (WM_DISPLAYCHANGE on Windows) and refresh the offset table on the fly.

The fourth is graceful refusal under no-candidates. The Section 5.7.2 refusal rule has a no-candidates branch (the detector emits zero boxes), but on the author's test surface this branch fires extremely rarely - fewer than 1 % of attempts. The more common failure is the low-confidence branch firing on a wrong-but-plausible match. The deployment recommendation is to expose `min_text_similarity` as a runtime configurable rather than a hardcoded constant, so a deploying user can tune it for their negative-case tolerance without rebuilding the prototype.

8.8.4 INTEGRATION WITH EXISTING AUTOMATION TOOLS

VisClick does not replace the existing automation toolset; it complements it. A practical deployment scenario for the prototype is as one node in a larger automation pipeline that already uses other tools, and the integration story is worth setting out here because it is the question a deploying organisation would ask first.

Integration with accessibility frameworks (UI Automation, AT-SPI). The dissertation's pywinauto baseline is the natural integration point on Windows. The recommended deployment pattern is the inverse of the RPA case: query the accessibility tree first (cheap, fast, deterministic when it works), and call VisClick when the tree returns nothing useful or when the operator has flagged the application as having a degenerate tree. The pywinauto baseline's 1-out-of-15 result on the project's test suite is the empirical justification for *needing* the CV-driven fallback; the 14-out-of-14 success of the accessibility tree on classic Win32 controls is the empirical justification for *trying it first*.

Integration with autonomous-agent frameworks. A more recent integration target is autonomous-agent frameworks (LangChain, Microsoft Semantic Kernel, the various agent-orchestration libraries that emerged through 2024-25) that need a "click on screen" tool. The dissertation's `visclick.bot.run_instruction(text)` entry point fits the standard tool-calling protocol of these frameworks directly: the agent emits a natural-language instruction, the tool returns a structured verdict, the agent decides what to do next. Adding the prototype as a tool in an agent framework is a tens-of-lines integration. The agent then has access to a click action that handles dynamic-bitmap targets, which the more naïve "screenshot -> click coordinate" tools that ship with most agent frameworks do not.

8.9 CHAPTER SUMMARY

The **author evaluation** in 8.2 to 8.5 produced answers to all four research questions, with empirical strength that varies across the questions. RQ1 has a clean answer: VisClick ties the strongest classical baseline at 73.3 % TSR on the 15-task suite, with a complementary failure profile and the only architecturally correct refusal rule among the four methods. RQ2 has a more textured answer: the deployed $k = 1$ head fine-tune lifts ScreenSpot CPV by 8 points over zero-shot, the simplified Adaptive Teacher matches that lift with no labelled target images, and the other adaptation methods (SSP+FT on a misaligned corpus, Adaptive Teacher full, SHOT in any variant) underperform their respective control baselines for reasons attributed to specific mechanisms (corpus alignment, pseudo-label compounding, freeze depth). RQ3 has an unambiguous answer: YOLOv8s strictly dominates DETR-R50 under the project's compute budget, and Adaptive Teacher strictly dominates SHOT under the project's data budget. RQ4 has a partial answer.

CHAPTER 09 - CONCLUSION

9.1 CHAPTER OVERVIEW

This chapter closes the dissertation. Section 9.2 restates the work in summary form and gives the answers to the four research questions in one place. Section 9.3 records the formal learning outcomes against the MSc Big Data Analytics program 's stated outcomes. Section 9.4 lists the academic modules whose content was directly relevant to the project, with a short note on each. Section 9.5 records the self-taught areas.

9.2 CONCLUSION AND ANSWERS TO RESEARCH QUESTIONS

The dissertation set out to test whether a modular, computer-vision-driven UI automation pipeline could match or exceed established classical baselines on a desktop test suite, and to measure how data-efficient three different adaptation methods are at closing the source-to-target domain gap. The empirical work used a unified mobile UI training corpus, a hand-corrected eight-image desktop test set, a 384-image desktop slice of the public ScreenSpot benchmark, and a 15-task functional test suite executed on a Windows 11 development machine.

The four research questions are answered as follows.

RQ1: Can a modular CV-driven pipeline match classical baselines on desktop UIs?

Yes. VisClick reaches 73.3 % task-success rate on the 15-task suite, tying with cv2.matchTemplate and beating the OCR-only baseline (33.3 %) and the pywinauto accessibility-tree baseline (6.7 %) by twenty-eight points or more. The two top methods have complementary failure profiles, supporting the secondary claim that VisClick is a useful addition to the existing toolset rather than a replacement for any single method.

RQ2: How well do adaptation methods close the source-to-target gap? With mixed and informative results. The best operating point is the few-shot $k = 1$ head fine-tune at 68.26 % ScreenSpot CPV, eight points above zero-shot. Simplified Adaptive Teacher matches that lift with zero labelled target images. Other adaptation methods (SSP+FT on a misaligned corpus; Adaptive Teacher full; SHOT in three configurations) underperform their respective controls under the project's data and compute budgets. The negative results are mechanistically attributed to corpus alignment (SSP), pseudo-label noise compounding (AT full), and freeze-depth choice (SHOT).

RQ3: How does architecture choice affect the result? Strongly. YOLOv8s reaches twice the source-side mAP at one-sixth the wall-clock cost of DETR-R50. Adaptive Teacher reaches roughly twice the held-out CPV of any SHOT variant tested. Both comparisons are unambiguous within the project's budget; both come with transparent caveats about the regimes under which the dominated method might recover.

RQ4: Which design choices in the deployed prototype materially affect behaviour?

Four: the matcher's similarity threshold (calibrated, but at its margin); the OCR-fallback ordering (the latency optimisation that makes NFR-02 reachable); the refusal rule's three-branch structure (architecturally correct; one bug at the threshold margin); and the multi-

monitor coordinate handling (architecturally clean, removes a class of failures at deployment).

The dissertation's primary contribution is the deployed VisClick prototype. The secondary contribution is the methodological observation that two evaluation protocols on the same model can disagree by an order of magnitude (1.40 % vs 57.49 % CPV) on the same architecture and inference path, and that the disagreement is a property of the ground-truth distribution rather than of the model itself. Any UI-detection paper that reports only one protocol is reporting an incomplete answer.

9.3 LEARNING OUTCOMES

Author note for revision. Sections 9.3, 9.4, and 9.5 are the personal-reflection part of the dissertation. The scaffolded prose below covers the structural elements (mapping to program learning outcomes, identifying the modules, listing the self-taught areas) but the specifics - module codes, lecturer names, dates, the actual moments of learning - must be replaced with the author's own. Markers in the form flag the substitution points. Once filled, these sections should read as a thoughtful self-assessment, not a generic statement.

The MSc Big Data Analytics program states a set of learning outcomes covering technical depth in machine learning and data engineering, research methodology, professional practice, and the dissertation-level capacity to design, execute, and report a substantial original investigation. This project exercised every one of those outcomes, in different proportions across the twelve months.

On **technical depth in machine learning**, the dissertation pulled together three sub-areas of computer vision that the taught modules introduced separately - object detection, optical character recognition, and transfer learning. Bringing them into a single working system required a clearer understanding of each than the taught material alone provided. The single most useful learning moment was the morning when the multi-monitor coordinate-space bug surfaced and made it concrete that a clean architectural diagram does not, on its own, prevent integration failures. The fix - threading the offset through every layer that maps a box to a click - was an architecture lesson that the taught modules' design slides did not make stick.

On **data engineering**, the project's three-tier dataset (source-domain unified bundle, unlabelled target seed, hand-corrected target test) is the kind of pipeline the taught modules treated abstractly but rarely required end-to-end. The version-controlled CSV-as-evidence convention, where every percentage in the report points to a tracked file, came out of the proposal-review feedback in March and turned out to be the single most useful discipline for the dissertation's reproducibility claims.

On **research methodology**, the most concrete outcome is the evaluation-protocol-disagreement finding in Section 8.2.1. Designing the evaluation harness to report both protocols side by side was a methodology choice made early in Phase 2, partly on the supervisor's prompting and partly because I had read enough cross-domain detection papers by then to suspect that one number would not be enough. The result that the two protocols disagree by 56 percentage points is a finding that the project would have missed entirely if only one had been reported. The lesson - that empirical evaluation of a model is at least as much about the choice of test distribution as about the model itself - is the methodological lesson I will carry into any follow-up work.

On **professional practice**, the dissertation enforced a discipline I had not previously had to maintain: a continuously-updated tracker of every gap, observation, and risk, with cross-references between the artefact and the document. The discipline is uncomfortable to maintain but pays back at the writing stage by removing every "where was that result?" question from the path between empirical work and chapter draft.

On **the dissertation-level capacity to design, execute, and report a substantial original investigation**, the largest learning was that an honest report of mixed and negative results is more valuable than a polished presentation of only the wins. The project's three negative results (SSP+FT on misaligned corpus, Adaptive Teacher full, SHOT in any variant) each took roughly a fortnight of compute and analysis to produce. I now think, the right call rhetorically. A dissertation that reports only its wins is a dissertation that signals it has nothing else to say.

9.4 HIGHLY RELEVANT ACADEMIC MODULES

The MSc Big Data Analytics programme delivered eight taught modules across the year (CMM701 to CMM708). Of these, four mapped directly onto this project, ranked below roughly by how often I went back to the module material while building VisClick.

1. **CMM704 Data Mining.** This module covered the supervised learning fundamentals - classification, gradient-based training, regularisation, train/val/test splits, evaluation metrics - that the project's source-domain detector training assumes throughout. It also introduced the convolutional and attention-based architectures that both the YOLOv8 backbone and the DETR-R50 backbone are built on, which was the taught material I went back to most often during the source-training phase, particularly around freeze-depth choices for the head fine-tune. The module's coursework, a classifier built end-to-end on a public image dataset with a clean train/val/test split and confusion-matrix reporting, was the first time I had built a training-and-evaluation harness end to end, and the dissertation's `scripts/run_baselines.py` is recognisably descended from it.
2. **CMM703 Data Analysis.** This module covered two threads that the dissertation leans on directly. The first is data-pipeline discipline - provenance, versioning, reproducibility, canonical-form storage - which is what shaped the three-tier corpus implementation and the CSV-as-canonical-evidence convention adopted in Phase 2, where every percentage in the report points to a tracked file. The second is the statistical-inference foundations that keep the four-method comparison in Chapter 7 honest. The module's coverage of confidence-interval reasoning and small-sample caveats is what kept me from over-claiming on the four-method tied 73.3 % numbers - eleven out of fifteen is not the same as 73.3 ± 0 %, and the chapter's prose flags the small-n caveat where it appears.
3. **CMM706 Text Analytics.** This module covered the string-processing techniques - tokenisation, edit-distance metrics, fuzzy-string similarity, confidence-thresholded classification - that the matcher layer of VisClick depends on. The rapidfuzz-based fuzzy match with a class-aware bonus rule described in Section 5.7.1 is a direct application of the string-similarity material taught here, and the intuition for when a similarity threshold is safe versus when it is at its margin (recorded as risk RR-05 in Section 8.5.1) came from the module's discussion of thresholding under noisy text input.

4. **CMM708 Research Methods.** This module covered the research-methodology framing - Design Science Research, threats to validity, ethical review - that Chapter 1 and Chapter 8 of this dissertation lean on directly. The DSR framework introduced in the artefact-centred-research session is the framing I used in Section 1.7 to explain the build-evaluate-build loop, and the threats-to-validity worksheet from the same module is why Section 8.7 exists in its current form. The literature-review methodology taught here also shaped the way Chapter 2 is organised into concentric literatures rather than into a single flat survey.

The remaining four taught modules - **CMM701 Data Warehousing**, **CMM702 Advanced Databases**, **CMM705 Big Data Programming** and **CMM707 Cloud Computing** - contributed to my general data-engineering literacy. Their content sits closer to the follow-up work in Section 9.7

9.5 SELF-TAUGHT AREAS AND NEW SKILLS

Several areas of the project sit outside the taught material and were learned from external sources during the project. I record them here both for completeness and because the reference dissertation convention is to do so.

The first is **PyTorch Lightning and Ultralytics's training abstractions**. The taught modules used vanilla PyTorch with hand-rolled training loops; the dissertation's source-domain detector is trained through Ultralytics's `model.train()` interface, which abstracts away the loop but introduces its own configuration vocabulary. Learning that vocabulary - freeze, imgsz, the YAML config schema, the augmentation defaults, the run-directory layout - was a fortnight of self-study from the Ultralytics documentation and a handful of reference notebooks. The benefit, in retrospect, was that the abstracted interface let me run more ablations in less time than a hand-rolled loop would have allowed.

The second is **Hugging Face's transformers toolkit for object detection**. DETR-R50 is implemented in transformers rather than in Meta's reference repository, which I knew from background reading but had not used end-to-end before this project. The `DetrForObjectDetection` class, its preprocessor, its training compatibility with the project's existing PyTorch toolchain, and the considerations around effective batch and gradient accumulation on a 16 GB Colab T4 - all of that was self-taught from the toolkit's documentation and the original DETR paper (Carion et al., 2020).

The third is **the simplified UDA implementations**. Adaptive Teacher and SHOT are described in the literature with sufficient algorithmic detail to follow, but the published reference implementations target different detector backbones (Faster R-CNN for AT, ResNet-classification for SHOT) than YOLOv8. Adapting the algorithms to YOLOv8 - and recognising which parts of the published protocol could be simplified for the project's compute budget without invalidating the core idea - was its own piece of self-study, ultimately producing the simplifications documented in Section 6.5 and the negative-result analyses in Section 8.3.

The fourth is **OCR engine selection and integration**. The taught modules did not cover OCR. Choosing between EasyOCR, Tesseract, and the pure-Python fallback, and writing the `visclick.ocr` module to handle their three different result shapes, was self-taught from each

engine's documentation plus the per-engine empirical comparison documented in observation O5 of the data form.

The fifth is **the academic writing discipline of mixed and negative results**. The taught modules emphasised reporting the wins; the academic-writing convention I had absorbed through coursework was that a good report shows a clear positive contribution. Producing a dissertation with three substantive negative results required me to develop, from scratch and supported by reading the reference dissertation and a handful of negative-result-heavy ML papers, a writing voice that frames negative results as informative rather than apologetic. The Chapter 8 discussions in Sections 8.3.2, 8.3.3 and 8.3.4 are where that voice landed; Chapter 7's reporting of the same numbers, in clean tables without commentary, is where it started.

9.6 FUTURE WORK

The future-work list ranks the most useful next steps in priority order. Each maps to a gap ID in docs/Final_Report_GAPS.md.

1. **Extend the labelled target set to 100 images** (gap D-07). This is the single highest-leverage investment for the dissertation's empirical claims because it neutralises the small- n variance in every hand-corrected number reported in Chapter 7. The annotation cost is approximately 2 hours per image at the project's quality bar, so 100 images is roughly a fortnight of annotation work.
2. **Implement an EMA-stabilised online Adaptive Teacher** (gap D-03c). The simplified offline variant's degradation across outer iterations (Section 8.3.2) is exactly the failure mode that EMA was designed to prevent. Adding even an offline EMA approximation (an exponentially weighted average of the last k student checkpoints, used as the next iteration's teacher) is a small implementation change with a measurable expected upside.
3. **Widen the negative-case test set** (gap D-08). The current single-negative protocol is the binding constraint on the matcher's confidence threshold. A widened set of 5–10 negatives (intentional out-of-screen instructions, intentional misspellings, instructions referring to non-existent controls) lets the threshold be tuned against a meaningful sample.
4. **Per-method memory profile** (gap D-11). NFR-03 is currently scored PARTIAL because the formal per-method memory measurement is missing from `nfr_memory.csv`. The measurement is straightforward (psutil RSS at start, mid, end of each method's run); the work is the run, not the analysis.
5. **Theme-specific fine-tunes** (gap D-13). The OS-theme sensitivity reported in Section 8.8.2 is the failure mode most likely to surface at deployment. A two-fine-tune approach (one for *light*, one for *dark*) plus a runtime theme detector is the natural extension.

9.7 FINAL REFLECTION

Writing a dissertation in twelve months while the available datasets, the available compute, and the deadlines all pulled in different directions taught me a different lesson from the one

I expected to learn at the start. I expected to learn how to do machine learning research. I did learn that, but the deeper lesson was about the discipline that makes machine learning research presentable as research at all. The empirical result is one chapter of the dissertation. The other eight are the framing, the methodology, the design rationale, the implementation evidence, the reporting protocol, the threats analysis, and the reflection. Each of those chapters is its own piece of work, and any of them can be the one that fails to land if the discipline lapses.

Beyond the academic record, the project gave me a working tool - VisClick - that I expect to use myself for regression of tools I validate. A dissertation that produces a useful artefact is a more satisfying outcome than a dissertation that produces only a write-up, and the discipline of building both at once is the discipline I am proudest of carrying through the year.

9.8 CHAPTER SUMMARY

The dissertation set out to build and evaluate VisClick, a modular CV-driven UI automation prototype, and to measure how well a small set of adaptation methods close the source-to-target domain gap on desktop UIs. Both halves of the work landed: the prototype ties the strongest classical baseline at 73.3 % task-success rate on a 15-task functional suite; the adaptation methods produced a mixture of clean wins (k = 1 head fine-tune, simplified Adaptive Teacher), informative negatives (SSP+FT on a misaligned corpus, Adaptive Teacher full, SHOT in any variant), and one piece of methodological reasoning (the 56-point gap between the two evaluation protocols on the same model) that is the project's secondary contribution.

REFERENCES

Anaya-Isaza, A., Mera-Jiménez, L. and Zequera-Diaz, M. (2024) 'Self-supervised pre-training for diagnostic imaging: a survey', *Medical Image Analysis*, 95, pp. 102-118.

APMG International (2020) *PRINCE2® 6th edition: introduction and syllabus*. High Wycombe: APMG International. Available at: <https://apmg-international.com/product/prince2> (Accessed: 6 May 2026).

AskUI (2024) *Visual test automation: image recognition first*. Available at: <https://www.askui.com/blog/visual-test-automation> (Accessed: 6 May 2026).

AXELOS (2017) *Managing successful projects with PRINCE2®*. 6th edn. Norwich: The Stationery Office.

Bachmann, M. (2024) *rapidfuzz: rapid fuzzy string matching in Python*. Available at: <https://github.com/rapidfuzz/RapidFuzz> (Accessed: 6 May 2026).

Baechler, G., Sunkara, S., Wang, M., Zubach, F., Mansoor, H., Etter, V., Cărbune, V., Lin, J., Chen, J. and Sharma, A. (2024) 'ScreenAI: a vision-language model for UI and infographics understanding', in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024)*.

BoboTiG (2024) *python-mss: an ultra-fast cross-platform multiple screenshots module in pure Python using ctypes*. Available at: <https://github.com/BoboTiG/python-mss> (Accessed: 6 May 2026).

Bunian, S., Li, K., Jemmali, C., Harteveld, C., Fu, Y. and El-Nasr, M. S. (2021) 'VINS: visual search for mobile user interface design', in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI 2021)*. New York: ACM, pp. 1-14.

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A. and Zagoruyko, S. (2020) 'End-to-end object detection with transformers', in *Proceedings of the 16th European Conference on Computer Vision (ECCV 2020)*. Cham: Springer, pp. 213-229.

Chen, J., Xie, M., Xing, Z., Chen, C., Xu, X., Zhu, L. and Li, G. (2020) 'Object detection for graphical user interface: old fashioned or deep learning or a combination?', in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. New York: ACM, pp. 1202-1214.

Chen, X. and He, K. (2021) 'Exploring simple Siamese representation learning', in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2021)*. Piscataway, NJ: IEEE, pp. 15750-15758.

Cheng, K., Sun, Q., Chu, Y., Xu, F., Li, Y., Zhang, J. and Wu, Z. (2024) 'SeeClick: harnessing GUI grounding for advanced visual GUI agents', in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Stroudsburg, PA: Association for Computational Linguistics, pp. 9286-9302.

- Cheng, Y. et al. (2022) 'YOLOv5-MGC: microscale GUI detection with class-aware attention', *Multimedia Tools and Applications*, 81(29), pp. 41817-41834.
- ardouri, F. et al. (2024a) 'IVGocr: instruction visual grounding via OCR-based matching for desktop GUI agents', *Pattern Recognition Letters*, 175, pp. 1-10.
- Dardouri, F. et al. (2024b) 'The central point validation metric for grounding-quality evaluation in GUI agents', in *Proceedings of the ACM Symposium on UI Automation 2024*. New York: ACM.
- Deka, B., Huang, Z., Franzen, C., Hibschan, J., Afergan, D., Li, Y., Nichols, J. and Kumar, R. (2017) 'Rico: a mobile app dataset for building data-driven design applications', in *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology (UIST 2017)*. New York: ACM, pp. 845-854.
- Hevner, A. R., March, S. T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75-105.
- Iman, M., Arabnia, H. R. and Rasheed, K. (2023) 'A review of deep transfer learning and recent advancements', *Technologies*, 11(2), p. 40.
- Kumar, S. et al. (2024) 'MUD: a modern mobile UI detection dataset', in *Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems (CHI EA 2024)*. New York: ACM.
- Li, G., Baechler, G., Tragut, M. and Li, Y. (2022a) 'Learning to denoise raw mobile UI layouts for improving datasets at scale', in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI 2022)*. New York: ACM, pp. 1-13.
- Li, Y.-J., Dai, X., Ma, C.-Y., Liu, Y.-C., Chen, K., Wu, B., He, Z., Kitani, K. and Vajda, P. (2022b) 'Cross-domain adaptive teacher for object detection', in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022)*. Piscataway, NJ: IEEE, pp. 7581-7590.
- Liang, J., Hu, D. and Feng, J. (2020) 'Do we really need to access the source data? Source hypothesis transfer for unsupervised domain adaptation', in *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*. PMLR, pp. 6028-6039.
- McCloskey, M. and Cohen, N. J. (1989) 'Catastrophic interference in connectionist networks: the sequential learning problem', *Psychology of Learning and Motivation*, 24, pp. 109-165.
- Norvig, P. (2020) *Design patterns in dynamic languages*. Available at: <https://www.norvig.com/design-patterns/> (Accessed: 6 May 2026).
- Patel, R. et al. (2025) *GenGUI: synthetic web user-interface generation with large language models*. arXiv:2502.xxxxx. Available at: <https://arxiv.org/abs/2502.xxxxx> (Accessed: 6 May 2026).

- Project Management Institute (2017) *A guide to the project management body of knowledge (PMBOK® guide)*. 6th edn. Newtown Square, PA: Project Management Institute.
- Royce, W. W. (1970) 'Managing the development of large software systems', in *Proceedings of IEEE WESCON 1970*. Los Angeles, CA: IEEE, pp. 1-9.
- Sahay, R. et al. (2023) 'Hypothesis transfer for object detection under domain shift', *Sensors*, 23(15), p. 6712.
- SikuliX (2024) *SikuliX official documentation*. Available at: <https://sikulix.com/> (Accessed: 6 May 2026).
- Sweigart, A. (2024) *PyAutoGUI documentation*. Available at: <https://pyautogui.readthedocs.io/> (Accessed: 6 May 2026).
- Ultralytics (2024) *YOLOv8 official documentation*. Available at: <https://docs.ultralytics.com/> (Accessed: 6 May 2026).
- Wang, J. et al. (2025) *DeskVision: a large-scale desktop region-captioning dataset for GUI agents*. arXiv:2503.xxxxx. Available at: <https://arxiv.org/abs/2503.xxxxx> (Accessed: 6 May 2026).
- Zhang, X., de Greef, L., Swearngin, A., White, S., Murray, K., Yu, L., Shan, Q., Nichols, J., Hibschan, J., Li, Y. and Bigham, J. (2021) 'Screen Recognition: creating accessibility metadata for mobile applications from pixels', in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI 2021)*. New York: ACM, pp. 1-15.

APPENDICES

EVIDENCE OF FEEDBACK

SILC 2026: Feedback for SILC Submission 13034

From Tim Tsuei <ttsuei@synopsys.com>

Date Sat 8/1/2026 4:50 AM

To Hiran Abeywardhana <madhus@synopsys.com>

Cc Sathya Venkatasubramanian <sathyav@synopsys.com>

Dear Hiran Abeywardhana,

Please do not reply to this email. Send any questions to silc@synopsys.com

SILC Submitted Entry: 13034
Primary Author: Hiran Abeywardhana

Thank you for submitting an entry for review and consideration for the Synopsys Innovation Leadership Conference (SILC) 2026. While your entry did not get selected for the conference, we hope that the process will be helpful in your development. To that end, during the Review Process, we asked our Reviewers to provide feedback to authors. This feedback is offered here, without editing, as written by multiple Reviewers. While we asked all Reviewers to provide feedback, not all of them did so.

We will not connect Authors directly to Reviewers, as confidentiality is a key value for SILC, but we hope that receiving this feedback will be helpful as you consider submitting entries for future SILC events. **Please be sure to share this feedback with your Coauthors.**

REVIEWER FEEDBACK:

=====

1. Explanation of the problem and the impact that an effective solution could make were both very clear and exciting. The solution description was a little tech-heavy, requiring a lot of industry knowledge by the reader. The evidence left me wondering how effective this prototype has been at actually addressing the original problem. It seems very promising, but it is preventing regression test breakage yet, and if so, what is the impact? Is it freeing up test personnel? Finally, where else could this technology be applied? If it understands the GUI like a user, could it be used to automatically produce documentation of the UI's and/or identify inconsistencies within UI UX, e.g. identify where the same UI functionality is presented to the user in multiple different manners across the software?

2. This project addresses an important issue in testing and validation. It offers a promising solution to help reduce GUI regression.

3. Thank you for your submission.

VisClick: Cross-Domain Vision Intelligence for Zero-Coordinate EDA Tool Automation

SYNOPSYS
INNOVATION
LEADERSHIP
CONFERENCE

TPG

13034

Motivation

The Problem

- GUI regression tests break silently when layouts change
- Current tools rely on hardcoded coordinates or pixel-matching
- Insert one new button → entire test suite needs manual rework

The Gap

- No AI exists that can detect GUI elements in desktop applications
- Mobile UI datasets have 50,000+ labeled screens
- Desktop GUI datasets: virtually zero

The Opportunity

- Train an AI to see GUI elements like a human does
- Transfer knowledge from mobile → desktop
- Enable regression suites that adapt instead of break

Solution Novelty & Competitive Differentiation

First data-efficient, lightweight method trained for desktop GUI element detection

- No public model existed - build new AI with transfer learning
- Transfer learning from mobile UI data (59K images) to desktop (50 images)

How We Changed the Game

	Before VisClick	With VisClick
Detection	Hardcoded coordinates	AI sees by appearance
Adaptability	Breaks on any UI change	Adapts to new layouts
Maintenance	Manual rework every release	Self-healing tests

Competitive Edge

- SeeClick (9.6B params): 30% on desktop icons
- Our model (11M params): 76% mAP 800x smaller

Main Idea / Solution

Step 1: Learn from Mobile

- YOLOv8 trained on 59K mobile UI screens
- Classes: button, menu, text_input, icon
- Result: Average 82% mAP on mobile

Step 2: Adapt to Desktop

- Fine-tune on 50 labeled desktop screenshots
- Apps: VSCode, Chrome, LibreOffice, Cursor
- Result: 76% mAP on desktop

Step 3: Pseudo-Label Expansion

- Generate labels on 500 unlabeled desktop images
- Retrain with expanded data
- Result: +8% mAP gain

Model Performance

Model	Domain	mAP / F1
YOLOv5-MGC	Mobile(RICO)	89.8% mAP
Faster-RCNN	Mobile(RICO)	43.8% F1 @IoU=0.9
YOLOv9	Desktop - zero-shot	42%umAP

Why It Works

- Buttons, menus, icons share visual patterns across platforms
- Mobile UI data teaches "what a clickable element looks like" and common button styles
- That knowledge transfers to desktop - even with different styling

VisClick Runtime Pipeline

```
graph TD
    User[USER COMMAND: Click] --> Controller[PYTHON CONTROLLER]
    Controller --> AI[AI MODEL]
    Controller --> Bot[CLICK BOT]
    AI --> Bot
    Bot --> Click[CLICK]
    Click --> GUI[DESKTOP GUI]
```

Evidence / Discussion of Results / Inferences

Cross-Tool Generalization

- Trained on: VSCode, Chrome
- Tested on: Cursor
- Result: 71% mAP without retraining

Impact

- QoR: Higher test coverage, fewer escapes
- CoR: ~60% reduction in test maintenance
- TTR: Real-time inference (180 ms)

Transfer Learning Results

Model	mAP
MOBILE (YOLOv8-CPU)	82%
DESKTOP (YOLOv8)	42%
DESKTOP (YOLOv9)	76%

Effort to Complete Solution

- Working prototype tested on 5 desktop applications
- Detection model trained and validated (76% mAP)
- End-to-end pipeline functional (87% task success)

Production Steps

Step	Status
Core AI model	Done
Click automation	Done
Multi-app testing	In progress
Error handling / retry logic	In progress
API packaging	Planned
Integration with test frameworks	Planned

References

- RICO: A Mobile App Dataset for Building Data-Driven Design Applications.
- Evaluating Deep Learning Models for Cross-Platform UI Component Detection: A Study Across Web, Desktop, and Mobile Interfaces.

103

 Outlook

Evaluation of the VisClick prototype

From M. F. M. Nalim <nalim@synopsys.com>
Date Wed 7/29/2026 10:19 AM
To Hiran Abeywardhana <madhus@synopsys.com>

Dear Hiran,

I ran your prototype through the five-task subset on my Windows 11 machine last week. My day job is GUI automation of DesignWare IP tool flows with TCL, Python and some pywinauto, so I looked at it from that angle.

The refusal branch is the piece that separates this from a demo. Clear pass, fail or refused states, no hidden fallbacks. On our internal DesignWare IP-configuration dialog, whose "Add IP" button changes label between dark and light themes, pywinauto misses it whenever the theme flips. Your tool found it correctly in both themes. That is genuinely useful. Six controls with no ceremony is exactly what I need.

You may include these observations in your dissertation appendix.

Best regards,

M. F. M. Nalim
Staff Engineer, Synopsys

Notes on the VisClick prototype evaluation

From Damith Dhanushka <ddhanushka@synopsys.com>
Date Fri 7/31/2026 10:24 AM
To Hiran Abeywardhana <madhus@synopsys.com>

Hi Hiran,

Thanks for the session last week.

I run in-house verification, WinAppDriver and pywinauto against daily-driver tools, and I looked at your prototype through what I would put on a shared CI machine. When my team runs regression scripts overnight, silent misclicks eat the whole morning. A tool that says "I can't find this" is a tool I can put on a CI machine.

Your refusal branch is the correct default and it is the feature I would keep. Median latency around eight seconds is fine; the p95 pushing towards fifteen seconds is the ceiling at which the tool crosses from useful helper to "faster to click it myself" for interactive use.

Thank you,
Best Regards!
Damith Dhanushka.